

SQIsign2D²

Algorithm specifications and supporting documentation

Version 1.0

- Zheng Xu, *Hefei National Laboratory, Hefei, China*
- Keteng Huang, *Xingtang Telecommunications Technology Co.,Ltd, Beijing Key Laboratory of Post-Quantum Cryptography, Beijing, China*
- Yi Ouyang, *University of Science and Technology of China, Hefei National Laboratory, Hefei, China*
- Jiankuo Dong, *University of Science and Technology of China, Hefei, China*
- Liang Bai, *Xingtang Telecommunications Technology Co.,Ltd, Beijing Key Laboratory of Post-Quantum Cryptography, Beijing, China*
- Yijing Ning, *University of Science and Technology of China, Hefei, China*
- Wei Wei, *University of Science and Technology of China, Hefei, China*
- Honggang Hu, *University of Science and Technology of China, Hefei, China*

April 26, 2026

xuzheng1@mail.ustc.edu.cn

Contents

Chapter 1. Introduction	5
1.1 Background	5
1.2 Algorithmic Properties	5
1.2.1 Innovation	5
1.2.2 Performance properties	7
1.2.3 Security properties	8
1.2.4 Other properties	8
1.3 Hard Problems of SQIsign2D ²	8
1.4 High Level Description of SQIsign2D ²	9
1.5 Notation	9
Chapter 2. Mathematical Background	12
2.1 The finite field $\mathbb{F}_p$ and $\mathbb{F}_{p^2}$	12
2.2 Elliptic curves	13
2.2.1 Montgomery curves	13
2.2.2 Addition formulas on Montgomery curves	14
2.3 Points on elliptic curves	15
2.3.1 Torsion Points	15
2.3.2 Discrete logarithm	19
2.3.3 Pairings on elliptic curves	19
2.4 Isogenies between elliptic curves	21
2.5 Principally polarized abelian surfaces and isogenies	23
2.5.1 Implementation of $(2, 2)$ -isogenies.	24
2.5.2 Chains of $(2, 2)$ -isogenies between products of elliptic curves	25
2.6 Lattice	26
2.6.1 Big integers	26
2.6.2 Floating-point support	27
2.6.3 Basic integer linear algebra	27
2.6.4 Lattices	29
2.6.5 Close vector enumeration in 2-dimensional lattices	31
2.7 Quaternions	34
2.7.1 Quaternions algebra	34
2.7.2 Lattices in quaternion algebras	34
2.7.3 Quaternion orders and ideals	36
2.7.4 Solving norm equations	37
2.8 Relation between isogenies and ideals	40
2.9 Representation of isogenies between elliptic curves via $(2, 2)$ -isogenies	43
2.9.1 Generating a random isogeny from E_0	43
2.9.2 Generating a random auxiliary isogeny	44
Chapter 3. SQIsign2D²	46
3.1 Digital signatures	46

3.1.1	Σ -Protocols	46
3.1.2	The Fiat–Shamir Transform	47
3.2	Parameter	47
3.2.1	Scheme parameters	47
3.2.2	Algorithmic parameters	48
3.3	Key generation	48
3.4	Signing	49
3.4.1	Commitment	50
3.4.2	Challenge	50
3.4.3	Response	51
3.4.4	Verification	52
3.5	Compact SQIsign2D ²	53
3.5.1	Compact key generation	53
3.5.2	Compact signing	54
3.5.3	Compact verification	56
Chapter 4.	Parameter setting	57
4.1	Parameter requirements	57
4.2	Parameter sets	58
Chapter 5.	Known answer test values	60
Chapter 6.	Performance analysis	61
6.1	The storage and transmission overhead	61
6.2	Reference implementation	61
6.3	Optimized implementation	62
6.4	Performance evaluation	62
Chapter 7.	Implementation details	63
7.1	Finite field arithmetic	63
7.1.1	Reference implementation	64
7.1.2	x86-64 optimized implementation	65
7.2	Elliptic curve arithmetic	65
7.2.1	Projective x -only coordinates	65
7.2.2	Curve arithmetic	66
7.2.3	Projective point difference	71
7.2.4	Jacobian coordinates	71
7.3	Computing pairings	76
7.3.1	The Miller loop approach	76
7.3.2	The cubical arithmetic approach	76
7.4	Isogenies	80
7.5	Theta coordinates	87
7.5.1	Theta coordinates of level 2	87
7.5.2	Doubling formulas using theta coordinates	88
7.5.3	Generic $(2, 2)$ -isogeny computation	89
7.5.4	Generic $(2, 2)$ -isogeny evaluation	91
7.5.5	Gluing $(2, 2)$ -isogeny computation	91
7.5.6	Gluing $(2, 2)$ -isogeny evaluation	92
7.5.7	Splitting change of coordinates	92
7.5.8	Computing a $(2, 2)$ -isogeny chain between products of elliptic curves	93
Chapter 8.	Heuristics and failure cases	103

8.1	Heuristics on lattices and ideals	103
8.2	Chains of $(2, 2)$ -isogenies	103
8.3	Failures in SQIsign2D ²	104
8.3.1	FullRepresentInteger and FullStrongApproximation	104
8.3.2	Response ideal sampling	105
Chapter 9.	Security analysis	106
9.1	Security reductions	106
9.1.1	Security of signature from Fiat-Shamir transform	106
9.1.2	Special soundness	107
9.1.3	Zero knowledge	108
9.1.4	Security proof of SQIsign2D ²	109
9.2	Resistance to known attacks	110
9.2.1	The pure endomorphism ring problem and general isogeny finding	110
9.2.2	Key recovery	111
9.2.3	Signature forgery	111
9.2.4	Attacking the Fiat-Shamir transform	111
9.3	Concrete security of parameter sets	112

Introduction

1.1. Background

Isogeny-based cryptography constitutes a significant part of post-quantum cryptography. This document provides a digital signature scheme SQIsign2D², whose security relies on the hardness of computing isogenies between supersingular elliptic curves or computing endomorphism ring of a supersingular elliptic curve. For SQIsign variants, the foundational framework was proposed in [DKL+20], a novel isogeny-based digital signature scheme, which has emerged as one of the most promising candidates and is currently under consideration in the NIST PQC standardization process [AAA+25]. With the breaking of SIDH and SIKE [JD11a; SIKE; CD23; MMP+23; Rob23], isogeny-based signature leverages high-dimension isogeny to verify and some schemes was proposed, such as SQIsignHD [DLR+24], SQIsign2DWest [BDF+25], SQIsign2DEast [NOC+25], SQIPrime [DF25]. The design of this scheme builds upon these SQIsign variants. Notably, SQIsign variants is conjectured to offer post-quantum security, meaning it can resist attacks from quantum computers.

The formal specifications of SQIsign2D² are structured as follows: Chapter 2 first present a comprehensive overview of the mathematical objects employed in the scheme, descriptions of the corresponding manipulation algorithms. Chapter 3 then elaborates on the core components of SQIsign2D², including key generation, signature generation, and verification. Other chapters cover a range of critical topics: parameter selection (Chapter 4), known answer tests (Chapter 5), performance analysis (Chapter 6), implementation details (Chapter 7), heuristic analysis and failure cases (Chapter 8), and a discussion of the scheme’s security properties (Chapter 9).

1.2. Algorithmic Properties

1.2.1. Innovation

We design SQIsign2D², a novel digital signature scheme within the SQIsign2D family, as well as the most efficient scheme in SQIsign family to date.

New parameters: Unlike other SQIsign2D variants, which use primes of the form $p = f \cdot 2^a - 1$, SQIsign2D² adopts a prime of the form $p = f \cdot 2^a \cdot 3^b - 1$ as the field characteristic, where $2^a \approx 3^b \approx \sqrt{p}$. Under this parameter setting, SQIsign2D² leverages efficiently computable 3^b -isogenies to significantly reduce the degree requirements for two-dimensional isogeny computations, thus lowering the overall computational cost compared to other SQIsign2D variants. Accordingly, we further optimize the finite-field arithmetic implementation for the prime field defined by our new parameter set.

New algorithms: Zheng Xu [XLZ+25], a member of our team, proposed two new algorithms

for the efficient generation of non-smooth isogenies. Based on these algorithms, the technique of reducing high-dimensional isogeny computations via one-dimensional isogenies was applied to accelerate schemes in SQIsign family.

- **ImRanIso** can be used to generate a non-smooth isogeny starting from E_0 . **ImRanIso** adapts a $2^{a'}3^b$ -isogeny to reduce the degree requirement for two-dimensional isogenies.
- **GenImRanIso** exploits the accessible 3^b -isogenies to reduce the degree requirements for two-dimensional isogenies.

Building on these algorithms, we design SQIsign2D². The proposed scheme substantially reduces the number of high-dimensional isogeny computations required during key generation and auxiliary isogeny generation. Compared with the previous best scheme, SQIsign2D-East, SQIsign2D² reduces the high-dimensional isogeny computations in key generation and auxiliary isogeny generation to one quarter and one third, respectively. At NIST security level 1, on Julia, the implementation achieves a significant performance improvement compared to SQIsign2D-East: in the uncompressed-signature setting, key generation and signing are accelerated by factors of 2.63 and 2.15, respectively; in the compressed-signature setting, the corresponding speedups are 2.75 and 1.77, respectively.

We provide size and efficiency comparisons between SQIsign2D² and other SQIsign2D variants in Table 1.1 and Table 1.4, demonstrating that SQIsign2D² is the fastest SQIsign2D-family variant to date.

TABLE 1.1. Key and Signature Sizes of 128-bit Secure SQIsignHD Schemes (Unit: bytes)

Scheme	Public Key Size	Compressed Signature Size
SQIsign	64	177
SQIsignHD	64	109
SQIPrime	192	240
SQIsign2D-West	66	148
SQIsign2D-East	64	150
SQIsign2DPush	66	151
PRISM	66	189
SQIsign2D²	64	154

TABLE 1.2. Isogeny-Degree Requirements for Key Generation in SQIsign2D Variants

Key Generation	degree of 2-isogenies	degree of 3-isogenies	degree of (2, 2)-isogenies
SQIsign2D-East	0	0	$\approx p$
SQIsign2D-West (Qlapoti)	0	0	$\approx p$
SQIsign2D²	$\approx \mathbf{p}^{1/4}$	$\approx \sqrt{\mathbf{p}}$	$\approx \mathbf{p}^{1/4}$

TABLE 1.3. Isogeny-Degree Requirements for Auxiliary Isogeny Computation in SQIsign2D Variants

Computation of Auxiliary Isogeny	degree of 2-isogenies	degree of 3-isogenies	degree of (2, 2)-isogenies
SQIsign2D-East	0	0	$\approx p^{3/2}$
SQIsign2D-West (Qlapoti)	0	0	$\approx p$
SQIsign2D²	0	$\approx \mathbf{p}^{3/2}$	$\approx \sqrt{\mathbf{p}}$

TABLE 1.4. Computational Costs of 128-bit Secure SQIsign-family Schemes

Scheme	Procedure	2-isogeny	3-isogeny	(2, 2)-isogeny
SQIsign2D-West	Key Generation	0	0	496
	Signing	248	0	992
	Verification	248	0	126
SQIsign2D-West + Qlapoti	Key Generation	0	0	243
	Signing	248	0	622
	Verification	248	0	126
SQIsign2D-East	Key Generation	0	0	253
	Signing	127	2	641
	Verification	127	2	129
SQIsign2DPush	Key Generation	393	234	0
	Signing	262	468	262
	Verification	0	78	131
PRISM	Key Generation	0	0	496
	Signing	0	0	496
	Verification	0	0	248
SQIsign2D²	Key Generation	65	78	66
	Signing	0	390	262
	Verification	0	78	131

1.2.2. Performance properties

The performance behavior of the SQIsign2D² scheme exhibits substantial differences when compared to other post-quantum signature candidates.

The most compact keys and signatures: SQIsign2D² delivers compact public keys and signatures sizes. To the best of our understanding, SQIsign2D² achieves the smallest combined size of public key and signature among all existing post-quantum digital signature schemes. Specifically, its total size is approximately 7 times smaller than that of Falcon and 16 to 17 times smaller than that of ML-DSA.

Practical and reasonable efficiency: Although SQIsign2D² runs slower than both ML-DSA and Falcon, its overall efficiency remains at a practical level, making it suitable for the vast

majority of real-world deployment scenarios. Notably, compared with previous SQIsign variants, SQIsign2D² shows a significant improvement in efficiency and, in its uncompressed form, is the fastest among the SQIsign family. **For instance, our optimized implementation at 128-bit security strength achieves runtimes of approximately 1.8 ms for key generation, 9.1 ms for signing, and 1.7 ms for verification on a 64-bit x86 platform with an effective clock frequency of about 4.3 GHz.** In comparison, under the same platform, SQIsign v2.0 [AAA+25] requires about 7.5 ms for key generation, 17.0 ms for signing, and 0.9 ms for verification.

A complex signing procedure: The primary limitation of SQIsign2D² lies in the complexity of its signing process, which relies on advanced mathematical foundations and the wide range of objects involved in operations. This complexity creates significant challenges for implementation, particularly when it comes to building resistance against side-channel attacks. It is important to note that the verification process of SQIsign2D² is far simpler and considerably faster.

1.2.3. Security properties

The security of SQIsign2D² is based on the hardness of a well-known number-theoretic problem: computing the endomorphism ring of a supersingular elliptic curve. This problem offers several key advantages for cryptographic design:

Reliable security parameter calibration: The computational complexity of the most efficient known attacks against SQIsign2D² or solving the endomorphism ring problem is thoroughly characterized and well-understood. This enables straightforward, accurate estimation of real-world attack costs, allowing for confident and precise calibration of security parameters.

Provable security: Under the hardness assumption of computing endomorphism ring problem and additional assumptions (for more detail, see chapter 9), SQIsign2D² is proven to satisfy EUF-CMA security (unforgeability under chosen message attacks).

1.2.4. Other properties

Parameter Simplicity: The algorithm’s parameter set is centered on a single main parameter: the characteristic p of the finite field. This parameter imposes few requirements, and all other parameters are deterministically derived from p . Such design minimizes configuration overhead and reduces the risk of insecure parameter choices.

Flexibility: The main parameter is of the form $f \cdot 2^a \cdot 3^b - 1$, where f is a small positive integer and $2^a \approx 3^b \approx 2^\lambda$ with λ denoting the security parameter. The algorithm supports flexible security levels by varying the security parameter λ , which determines the magnitudes of 2^a and 3^b . Parameters of the prescribed form are straightforward to generate for any desired λ .

Scalability: As computational power increases or application requirements change, the security strength can be scaled up by increasing λ , while preserving the same parametric structure $p = f \cdot 2^a \cdot 3^b - 1$. This allows a smooth transition to higher security levels without altering the algorithm’s core design.

1.3. Hard Problems of SQIsign2D²

Let E_1 and E_2 be two supersingular elliptic curves over $\mathbb{F}_{p^2}$. There are two hard problems of SQIsign2D². The foundational problem is essentially the following isogeny path problem: computing an isogeny $\varphi : E_1 \rightarrow E_2$. Another problem is computing endomorphism ring. Given an elliptic curve E , an endomorphism of E is an isogeny $\varphi : E \rightarrow E$. All endomorphisms of E consist a ring under addition and composition, and the ring is called the endomorphism ring of E , written $\text{End}(E)$. The endomorphism ring problem is the following: given E , compute $\text{End}(E)$. For supersingular elliptic curves, this problem is known to be equivalent to the isogeny path problem under polynomial time reductions [EHL+18; Wes22]. In fact, we have that

- given $E_1, E_2, \text{End}(E_1)$ and $\text{End}(E_2)$, one can find an isogeny $E_1 \rightarrow E_2$ in polynomial time, and
- given $E_1, E_2, \text{End}(E_1)$, and an isogeny $E_1 \rightarrow E_2$, one can compute $\text{End}(E_2)$ in polynomial time.

1.4. High Level Description of SQIsign2D²

SQIsign2D² is a digital signature scheme obtained from a sigma protocol via the Fiat–Shamir transform, and its security relies on hard computational problems on elliptic curves.

Now we describe the sigma protocol as follows. The prover sets an elliptic curve E_A as public key, and their secret is the associated endomorphism ring $\text{End}(E_A)$ or the isogeny $\tau : E_0 \rightarrow E_A$ and its degree. Since the curve E_A is public, the goal of the prover is to convince the verifier that they know $\text{End}(E_A)$. The procedure as follows.

1. For the commitment phase, the prover generates a random isogeny $\psi : E_0 \rightarrow E_{\text{com}}$, and sends E_{com} to the verifier as commitment.
2. For the challenge phase, the verifier generates a random isogeny φ from E_{com} to some other curve E_{chl} , and sends it to the prover.
3. From isogenies $\tau : E_0 \rightarrow E_A$, $\psi : E_0 \rightarrow E_{\text{com}}$ and $\varphi : E_{\text{com}} \rightarrow E_{\text{chl}}$, the prover can compute an isogeny $\sigma : E_A \rightarrow E_{\text{chl}}$ by knowing $\text{End}(E_A)$, and send it to the verifier.
4. The verifier checks that σ is indeed an isogeny from the public curve E_A to the challenge curve E_{chl} .

The idea is that to compute the response, the prover must use their knowledge of $\text{End}(E_A)$. Intuitively, the protocol asks for the prover to compute isogenies from E_A to a random curve E_{chl} , and that relates to knowledge of $\text{End}(E_A)$ thanks to the computational equivalence between the isogeny path problem and the endomorphism ring problem. However, this idea does not immediately work, and the protocol is insecure: a cheating prover could generate the commitment E_{com} by choosing a random isogeny $\varphi' : E_A \rightarrow E_{\text{com}}$. They may not be able to compute $\text{End}(E_A)$, but it does not matter. In response to $\varphi : E_{\text{com}} \rightarrow E_{\text{chl}}$, they would simply respond with $\sigma = \varphi \circ \varphi' : E_A \rightarrow E_{\text{chl}}$.

There is a simple fix to this issue, by ensuring that φ is not a “sub-isogeny” of σ . With this fix, one can actually prove that this protocol proves knowledge of *at least* some non-trivial part of $\text{End}(E_A)$. It is known that computing a non-trivial part of $\text{End}(E_A)$ is as hard as computing the full $\text{End}(E_A)$ [PW24].

The SQIsign2D² protocol described in detail in Chapter 3 follows the above outline.

1.5. Notation

We denote by $\lambda \in \mathbb{N}$ the security parameter. An event depending on λ is said to occur with *overwhelming probability* if its probability is at least $1 - \mu(\lambda)$ for some negligible function μ .

For a finite set S , the notation $x \xleftarrow{\$} S$ means that x is sampled uniformly at random from S . $x \leftarrow \mathcal{A}(\cdot)$ denotes that x is assigned the (possibly randomized) output of an algorithm $\mathcal{A}$. For positive integer m , denote by $[m]$ the set $\{0, 1, \dots, m-1\}$.

All algorithms and adversaries are assumed to be probabilistic polynomial-time (PPT), unless explicitly stated otherwise. A function $\mu : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is called *negligible* if for every constant $c > 0$ there exists λ_0 such that for all $\lambda \geq \lambda_0$, $\mu(\lambda) < \lambda^{-c}$. For any functions g , we will write $\tilde{O}(g)$ to mean $O(g \cdot \ell(g))$, where ℓ is any polylogarithmic function.

For a finite set S , we also denote by $\#S$ the number of elements of S .

TABLE 1.5. Notation and parameters of the SQIsign2D² scheme.

Scheme Parameters	
λ	Security parameter, one of 160, 256 or 512.
f	A small integer with $\gcd(f, 2) = 1$ and $\gcd(f, 3) = 1$.
p	Character of the base field, a prime of the form $f \cdot 2^a \cdot 3^b - 1$.
a'	An integer for key generation, $a' \approx a/2$.
Elementary Objects	
$\mathbb{Z}/d\mathbb{Z}$	Ring of integers modulo d , $d \in \mathbb{Z}^*$.
$\mathbb{F}_p$	Finite fields with character p and exactly p elements.
$\mathbb{F}_{p^2}$	Finite fields with character p and p^2 elements.
$\mathbb{F}_q$	Finite fields with character p and $q = p^n$ elements.
$\overline{\mathbb{F}_p}$	Algebraic closure of $\mathbb{F}_p$.
$\text{GL}_n(\mathbb{F}_q)$	Group of invertible matrices of size n over $\mathbb{F}_q$.
Elliptic Curve Objects	
E	An elliptic curve over some field $\mathbb{F}_q$.
$E \times E'$	A product of two elliptic curves E and E' .
A	An abelian variety over some field $\mathbb{F}_q$.
$\text{Jac}(C)$	The Jacobian of a hyperelliptic curve C .
$E_{A,B}$	An elliptic curve in Montgomery form with parameters $A, B \in \mathbb{F}_q$.
0_E	The point at infinity of an elliptic curve or abelian variety E .
pts	A subset of points contained in E .
P, Q	The canonical torsion basis of $E[2^a]$.
R, S	The canonical torsion basis of $E[3^b]$.
$E(\mathbb{F}_q)$	The $\mathbb{F}_q$ -rational points on E .
$E[n]$	The n -torsion points on E .
hint_2	Hints to generate a basis for $E[2^a]$.
hint_3	Hints to generate a basis for $E[3^b]$.
$j(E)$	The j -invariant of the elliptic curve E .
x_P and y_P	The x - resp. y -coordinate of a point $P \in E$.
$\varphi : E \rightarrow E'$	An isogeny between elliptic curves E and E' .
$\Phi : A \rightarrow A'$	A higher-dimensional isogeny between principally polarized abelian varieties A and A' .
$\text{End}(E)$	The endomorphism ring of an elliptic curve E .
Quaternion Objects	
$B_{p,\infty}$	A quaternion algebra, ramified at p and ∞ .
$B_{p,\infty}^*$	The space of linear functions $B_{p,\infty} \rightarrow \mathbb{Q}$.
$\mathcal{O}$	An order in $B_{p,\infty}$.
$\mathcal{O}_L(I), \mathcal{O}_R(I)$	The left, resp. right, order of an ideal $I \subset \mathcal{O}$.
$\text{Trd}(\alpha)$	The trace of an element $\alpha \in B_{p,\infty}$.
$\text{Nrd}(\alpha)$	The norm of an element $\alpha \in B_{p,\infty}$.
$\text{Nrd}(I)$	The norm of an ideal $I \subset \mathcal{O}$.
$[I]^*$	The pullback by an ideal I .
$[I]_*$	The pushforward by an ideal I .

TABLE 1.5. Notation and parameters of the SQIsign2D² scheme.

Protocol Objects	
pk	Public key; in SQIsign2D ² , pk contains the curve E_A ; in CompactSQIsign2D ² , pk contains the curve E_A and basis hints hint_2 and hint_3 .
sk	Secret key; in SQIsign2D ² , sk containing the secret ideal I_τ corresponding to secret isogeny τ , the points $\tau(P_0), \tau(Q_0), \tau(R_0), \tau(S_0)$, where P_0, Q_0 is the canonical basis of $E_0[2^a]$ and R_0, S_0 is the canonical basis of $E_0[3^b]$; in CompactSQIsign2D ² , sk containing the secret ideal I_τ corresponding to secret isogeny $\tau : E_0 \rightarrow E_A$, two basis hints hint_2 and hint_3 and two change-of-basis matrixs $\mathbf{M}_{\text{sk},2}, \mathbf{M}_{\text{sk},3}$.
msg	The message to be signed.
HASH	A hash function, obtained by repeated application of SM3 .
com	Commitment, containing a description of the commitment curve.
chl	Challenge, an integer between 0 and 3^b generated by hashing the public key pk, commitment curve and the message msg.
τ	Secret isogeny, containing a sescription of an isogeny from E_0 to secret curve E_A .
φ	Challenge isogeny, containing a sescription of an isogeny from commitment curve E_{com} to challenge curve E_{chl} of degree 3^b .
σ	Response, containing a description of an isogeny from E_A to the challenge curve E_{chl} .
E_{com}	Commitment curve.
E_{chl}	Challenge curve.
E_{aux}	The auxiliary curve required for the higher-dimensional response isogeny.
t	An integer between 0 and 3^b , a description of the dual of challenge isogeny.
bin	A bit, for fast verification.
$\mathcal{S}$	Signature; in SQIsign2D ² , $\mathcal{S}$ contains $E_{\text{com}}, E_{\text{aux}}$ together with the change-of-basis matrix $\mathbf{M}_c$, the challenge c , hint_{aux} and hint_c ; in CompactSQIsign2D ² , $\mathcal{S}$ contains E_{aux} together with the change-of-basis matrix $\mathbf{M}_c$, the challenge c , an integer t and a bit bin, hint_{aux} and hint_c .

Mathematical Background

SQIsign2D² relies on two mathematical tools, namely elliptic-curve isogenies over finite field and maximal orders and ideals of quaternion algebras, and the construction of SQIsign2D² is based on a correspondence between the two algebraic settings. This chapter reviews the definitions and basic properties for isogenies and quaternion algebras, and records the relations between these objects that are used throughout the implementation. For the conversion algorithms between isogenies and ideals, we follow [XLZ+25], mainly for the accelerated parts of the implementation.

Throughout this document, we work only over finite fields of characteristic p with

$$p \equiv 3 \pmod{4}.$$

2.1. The finite field $\mathbb{F}_p$ and $\mathbb{F}_{p^2}$

We follow the presentation in [AAA+25]. A finite field is a finite set of elements equipped with addition and multiplication operations and their inverses. From basic algebra, it is known that any finite field has cardinality $q = p^r$ for some prime p and some positive integer r , and is denoted by $\mathbb{F}_q$. For each such q , the finite field $\mathbb{F}_q$ is unique up to isomorphism, and its characteristic is p , written as $\text{char}(\mathbb{F}_q) = p$.

We represent $\mathbb{F}_p$ by the integers in $\{0, \dots, p-1\}$. The field operations are defined modulo p as follows:

- **Addition:** For $a, b \in \mathbb{F}_p$, the sum $c = a + b$ is the unique element $c \in \{0, \dots, p-1\}$ such that $c \equiv a + b \pmod{p}$.
- **Additive inverse:** For $a \in \mathbb{F}_p$, the additive inverse of a is the unique element $-a \in \{0, \dots, p-1\}$ satisfying $a + (-a) \equiv 0 \pmod{p}$.
- **Multiplication:** For $a, b \in \mathbb{F}_p$, the product $c = a \cdot b$ is the unique element $c \in \{0, \dots, p-1\}$ such that $c \equiv a \cdot b \pmod{p}$.
- **Multiplicative inverse:** For any nonzero $a \in \mathbb{F}_p$, its multiplicative inverse a^{-1} is the unique element $a^{-1} \in \{0, \dots, p-1\}$ satisfying $a \cdot a^{-1} \equiv 1 \pmod{p}$.
- **Quadratic residuosity:** For $a \in \mathbb{F}_p$, determining whether a is a quadratic residue amounts to checking whether there exists $b \in \mathbb{F}_p$ such that $b^2 \equiv a \pmod{p}$. This can be decided by evaluating $a^{(p-1)/2}$: the value is 1 when a is a quadratic residue and -1 otherwise.
- **Square root:** Let $a \in \mathbb{F}_p$ be a square. A canonical square root of a can be computed as $\sqrt{a} = a^{\frac{p+1}{4}} \pmod{p}$.

Since we use the finite fields with characteristic $p \equiv 3 \pmod{4}$, then we take $\mathbb{F}_{p^2} = \mathbb{F}_p(i)$, where $i^2 + 1 = 0$. Each element of $\mathbb{F}_{p^2}$ can be uniquely written by $a = a_0 + a_1i$, where $a_0, a_1 \in \mathbb{F}_p$. The field operations are then given as follows.

- **Addition:** For $a, b \in \mathbb{F}_{p^2}$, write $a = a_0 + a_1i$ and $b = b_0 + b_1i$. Their sum is $a + b = (a_0 + b_0) + (a_1 + b_1)i$, where the additions are performed in $\mathbb{F}_p$.
- **Additive inverse:** For $a = a_0 + a_1i \in \mathbb{F}_{p^2}$, the additive inverse is $-a = (-a_0) + (-a_1)i$, where the negations are taken in $\mathbb{F}_p$.
- **Multiplication:** For $a, b \in \mathbb{F}_{p^2}$, their product is $ab = (a_0b_0 - a_1b_1) + (a_0b_1 + a_1b_0)i$, using additions, negations, and multiplications in $\mathbb{F}_p$.
- **Multiplicative inverse:** For a nonzero element $a = a_0 + a_1i \in \mathbb{F}_{p^2}$, define $N = a_0^2 + a_1^2 \in \mathbb{F}_p$. Then $a^{-1} = (a_0N^{-1}) + (-a_1N^{-1})i$, where the inversion N^{-1} is carried out in $\mathbb{F}_p$.
- **Quadratic residuosity:** Let $a = a_0 + a_1i \in \mathbb{F}_{p^2}$. Then a is a square in $\mathbb{F}_{p^2}$ if and only if $a^{p+1} = a_0^2 + a_1^2$ is a square in $\mathbb{F}_p$.
- **Square root:** Let $a \in \mathbb{F}_p$. If a is a square in $\mathbb{F}_p$, its square root is defined as in $\mathbb{F}_p$. Otherwise, $-a$ is a square in $\mathbb{F}_p$, and we set $\sqrt{a} = \sqrt{-a}i$. Now let $a = a_0 + a_1i \in \mathbb{F}_{p^2} \setminus \mathbb{F}_p$ be a square in $\mathbb{F}_{p^2}$. A canonical square root of a is defined by $\sqrt{a} = (-i)^{\frac{1-\chi}{2}} S(a + \delta)$, where $\delta = \sqrt{a_0^2 + a_1^2}$, $S = [2(a_0 + \delta)]^{\frac{p-3}{4}}$, and $\chi = [2(a_0 + \delta)]^{\frac{p-1}{2}}$.

2.2. Elliptic curves

In this section, we assume that $\text{char}(\mathbb{F}_q) > 2$, which is ensured by the condition $p \equiv 3 \pmod{4}$. All elliptic curves considered here are smooth projective curves of genus 1 over finite field. For implementation purposes, we introduce the Montgomery form.

2.2.1. Montgomery curves

Montgomery curves Let $A, B \in \mathbb{F}_q$ satisfy $B(A^2 - 4) \neq 0$. The Montgomery curve $E_{A,B}$ over $\mathbb{F}_q$ is the elliptic curve defined by

$$By^2 = x^3 + Ax^2 + x. \quad (2.1)$$

If $B = 1$, we write $E_{A,B}$ as E_A . The curve has genus 1 and the condition $B(A^2 - 4) \neq 0$ makes that the curve is nonsingular. We denote by $E_{A,B}(\mathbb{F}_q)$ the set of $\mathbb{F}_q$ -rational points of $E_{A,B}$, including the point at infinity 0_E . The parameter A is often called **Montgomery coefficient**.

Two Montgomery curves are said to be **isomorphic** over $\mathbb{F}_q$ if there exists a linear change of coordinates $(x, y) \mapsto (Dx + R, Cy)$ between two curves, with $D, C, R \in \mathbb{F}_q$. If $E_{A,B}$ and $E_{A',B'}$ are isomorphic over a finite extension of $\mathbb{F}_q$, then they are said to be **twists** of each other. In particular, $E_{A,B}$ and $E_{A',B'}$ are isomorphic over $\mathbb{F}_q$ if and only if $C = \sqrt{B/B'} \in \mathbb{F}_q$. Moreover, if $C = \sqrt{B/B'} \in \mathbb{F}_{q^2} \setminus \mathbb{F}_q$, we say $E_{A,B}$ and $E_{A',B'}$ are **quadratic twists** of one another.

The j -invariant of Montgomery curve $E_{A,B}$ is defined as $j(E_{A,B}) = \frac{256(A^2-3)^3}{A^2-4}$. Two Montgomery curves have the same j -invariant if and only if they are isomorphic or quadratic twists. If $E_{A,B}$ and $E_{A',B'}$ are isomorphic, then the coordinate change $(x, y) \mapsto (Dx + R, Cy)$ sending the former to the latter is given by

$$D = \lambda^2 \frac{B'}{B}, \quad R = \lambda^2 \frac{AB'}{3B} - \frac{A'}{3}, \quad C = \frac{\lambda^3 B'}{B}, \quad \lambda := \sqrt{\frac{B(2A'^3 - 9A')(3 - A^2)}{B'(2A^3 - 9A)(3 - A'^2)}}. \quad (2.2)$$

Let $N = \#E_{A,B}(\mathbb{F}_q)$. If $N \equiv 1 \pmod{\text{char}(\mathbb{F}_q)}$, then $E_{A,B}$ is said to be **supersingular**. In the case $q = p^2$ with $p \equiv 3 \pmod{4}$, every supersingular curve with $B = 1$ has exactly $(p+1)^2$ points. Its quadratic twists $E_{A,\gamma}/\mathbb{F}_{p^2}$, where γ is any quadratic non-residue in $\mathbb{F}_{p^2}$, have exactly $(p-1)^2$ points and are all isomorphic. Moreover, if $E_{A,B}$ is supersingular, we have the trace of Frobenius map of $E_{A,B}$ is divided by p and $j(E_{A,B}) \in \mathbb{F}_{p^2}$.

Standardization of Montgomery curves. In the compressed version of SQIsign2D², one needs to choose a standard curve among the isomorphic class of a given montgomery curve. We focus on montgomery curves with $B = 1$. Let $E_A : y^2 = x^3 + Ax^2 + x$ be a montgomery curve with j -invariant $j(E_A) = \frac{256(A^2-3)^3}{A^2-4}$. There are six montgomery coefficients A' satisfying $E_{A'} \cong E_A$, given by

$$\left\{ \pm A, \pm \frac{A - 3\sqrt{A^2 - 4}}{\sqrt{2(A^2 - 4 - A\sqrt{A^2 - 4})}}, \pm \frac{A + 3\sqrt{A^2 - 4}}{\sqrt{2(A^2 - 4 + A\sqrt{A^2 - 4})}} \right\}.$$

To choose a standard one, first convert E_A to Short Weierstrass model $E_{a,b} : y^2 = x^3 + ax + b$ with

$$a = -\frac{12(A^2 - 3)^3}{(2A^3 - 9A)^2}, \quad b = -\frac{8(A^2 - 3)^3}{(2A^3 - 9A)^2},$$

then determinately select one of the three roots of $x^3 + ax + b = 0$ by lexicographic ordering. The root then determines a unique isomorphism to a montgomery curve. Indeed, let γ be a root of $x^3 + ax + b = 0$. Then $E_{a,b}$ is isomorphic to $y^2 = x^3 + 3\frac{\gamma}{\delta^2}x^2 + x$, where $\delta = \sqrt[4]{3\gamma^2 + a}$, and the isomorphism is given by

$$(x, y) \mapsto \left(\frac{x - \gamma}{\delta^2}, \frac{y}{\delta^3} \right).$$

The idea is implemented in [CurveStandardization](#)(Algorithm 7.9) described in Section 7.2.2.

2.2.2. Addition formulas on Montgomery curves

For any point $P \neq 0_E$, where 0_E is the infinite point on E , we write

$$P = (x_P, y_P).$$

It is easily to see that $-P = (x_P, -y_P)$. We now give the formulas for computing the group law on a Montgomery curve.

- **Point addition.** Let $E_{A,B}/\mathbb{F}_q$ be a Montgomery curve, and let $P = (x_P, y_P)$ and $Q = (x_Q, y_Q)$ be points on $E_{A,B}$ such that $P \neq \pm Q$. Write $R = P + Q = (x_R, y_R)$. Then the sum is given by

$$x_R = B\lambda^2 - (x_P + x_Q) - A, \tag{2.3}$$

and

$$y_R = \lambda(x_P - x_R) - y_P, \tag{2.4}$$

where

$$\lambda = \frac{y_P - y_Q}{x_P - x_Q}.$$

The point at infinity 0_E serves as the identity element, so

$$P + 0_E = 0_E + P = P.$$

- **Point doubling.** Let $E_{A,B}/\mathbb{F}_q$ be a Montgomery curve, and let $P = (x_P, y_P)$ be a point on $E_{A,B}$ with $P \neq -P$. Then

$$x_{[2]P} = \frac{(x_P^2 - 1)^2}{4x_P(x_P^2 + Ax_P + 1)}, \quad (2.5)$$

and

$$y_{[2]P} = y_P \cdot \frac{(x_P^2 - 1)(x_P^4 + 2Ax_P^3 + 6x_P^2 + Ax_P + 1)}{8x_P^2(x_P^2 + Ax_P + 1)^2}. \quad (2.6)$$

In the special case $P = -P$, one has

$$[2]P = 0_E.$$

- **Point difference.** Let $P, Q \in E(\mathbb{F}_{p^2})$. The x -coordinates of $P - Q$ and $P + Q$ can be recovered from x_P and x_Q through

$$x_{P-Q} = \frac{B - \sqrt{B^2 - AC}}{C}, \quad (2.7)$$

and

$$x_{P+Q} = \frac{B + \sqrt{B^2 - AC}}{C}, \quad (2.8)$$

where

$$\begin{aligned} A &= (x_P x_Q - 1)^2, \\ B &= (x_P x_Q + 1)(x_P + x_Q) + 2Ax_P x_Q, \\ C &= (x_P - x_Q)^2. \end{aligned} \quad (2.9)$$

- **Scalar multiplication** Using the abelian group law on an elliptic curve E , we define scalar multiplication by an integer $k \in \mathbb{Z}$. For $k > 0$, the map $[k] : E \rightarrow E$ sends a point $P \in E$ to

$$[k]P = \underbrace{P + P + \cdots + P}_{k \text{ times}}.$$

For $k < 0$, we define

$$[k]P = -[|k|]P,$$

and for $k = 0$, we set

$$[0]P = 0_E.$$

In practice, scalar multiplication is evaluated through a sequence of point doublings and point additions. In particular, when the Montgomery ladder is used, the number of elliptic-curve operations grows logarithmically with $|k|$.

2.3. Points on elliptic curves

2.3.1. Torsion Points

For a point $P \in E$, the **order** of P is defined as the smallest positive integer m such that

$$[m]P = 0_E.$$

For an integer $m \in \mathbb{Z}$ and an elliptic curve E over $\mathbb{F}_q$ where $\text{char}(\mathbb{F}_q) = p$, we write $E[m]$ for the subgroup of points P satisfying $[m]P = 0_E$.

Let E be an elliptic curve and let $m \in \mathbb{Z}$ with $m \neq 0$. If m is coprime to p , then $E[m] \cong (\mathbb{Z}/m\mathbb{Z}) \times (\mathbb{Z}/m\mathbb{Z})$. Furthermore, if $m = p^n$ for any positive integer n , then $E[m] = \{0_E\}$ or $E[m] \cong \mathbb{Z}/m\mathbb{Z}$.

Now let $m \in \mathbb{Z}$, $\gcd(m, p) = 1$ and let E be a supersingular elliptic curve defined over $\mathbb{F}_{p^2}$. Then there exist generators $R, S \in E[m]$, not uniquely determined in general, such that $\langle R, S \rangle = E[m]$. We call such a pair (R, S) a **basis** of $E[m]$. In the reference implementation, this basis is encoded more compactly by storing only the x -coordinates in the triple (x_R, x_S, x_{RS}) . This encoding is unchanged under the simultaneous sign change $(R, S) \mapsto (-R, -S)$, so both pairs produce the same triple. It also matches the use of the Ladder algorithm in Section 7.2, which computes the x -coordinate of $[n]R$ from x_R alone.

Throughout this work, a denotes the largest exponent such that $2^a \mid (p+1)$, and b denotes the largest exponent such that $3^b \mid (p+1)$ where p is the characteristic of the base field. In `SQIsign2D2`, one needs a basis (P, Q) of $E[2^a]$ and another basis (R, S) of $E[3^b]$. This problem has been studied extensively in the literature [CJL+17; CEM+24; ZSP+18]. For 2^a case, We use a deterministic construction in the procedure `TorsionBasisToHint2` (Algorithm 2.1), following [ZSP+18] with a small adaptation. The routine first checks the quadratic residuosity of A , and if A is a square in $\mathbb{F}_{p^2}$, then we choose a square h such that

$$x_P \leftarrow -A/(1 + i \cdot h)$$

is a non-square element of $\mathbb{F}_{p^2}$. Otherwise, we set $x_P = h \cdot A$. If x_P corresponds to a point of $E(\mathbb{F}_{p^2})$, then $x_Q = -x_P - A$ is also a non-square. Multiplying the corresponding points by the cofactor $(p+1)/2^a$ then gives a valid basis (P, Q) of $E[2^a]$.

The 3^b case is dealt with in `TorsionBasisToHint3` (Algorithm 2.3) using a trivial method. We sample x_P and x_Q in the form $1 + i \cdot h_1$ and $h_2 + i \cdot h_1$ respectively until they define a valid basis of $E[3^b]$ after multiplying the corresponding points by the cofactor $(p+1)/3^b$.

The value x_{PQ} must be computed deterministically from x_P and x_Q . From these two coordinates alone, however, one cannot distinguish whether x_{PQ} corresponds to $P + Q$ or to $P - Q$. This ambiguity is harmless as long as the implementation follows the reference convention and computes x_{PQ} with `ProjectiveDifference` (Algorithm 7.10), which fixes the choice deterministically.

On the verifier side, the sampling of a torsion basis for $E_A[2^a]$ can be accelerated by using hints carried in the signature. Concretely, for 2^a case, the verifier needs the quadratic residuosity of A together with the index used to derive x_P . These values are precomputed by the signer and included in the signature through `TorsionBasisToHint2` (Algorithm 2.1). Given these hints, the verifier reconstructs (x_P, x_Q, x_{PQ}) with `TorsionBasisFromHint2` (Algorithm 2.2). An important property of this routine is that the returned values x_P and x_Q are guaranteed to lie on the same twist of E , even before checking whether they represent points on E itself. This is sufficient in our setting, because every later use includes an order check, and that check implicitly confirms that the points do lie on E . For 3^b case, the verifier needs the valid h_1 and h_2 to obtain x_P and x_Q . These two values are precomputed by the signer and included in the signature through `TorsionBasisToHint3` (Algorithm 2.3). Given these hints, the verifier reconstructs (x_R, x_S, x_{RS}) with `TorsionBasisFromHint3` (Algorithm 2.4). The reference implementation proceeds in this way; see Chapter 7.

Remark 2.1. When A equals 0, Algorithm 2.1 fails. Under honest input generation, the procedure may fail with negligible probability.

Algorithm 2.1 TorsionBasisToHint₂(A, e)

Input: A non-zero affine Montgomery coefficient A and an integer e **Output:** An x -only basis (x_P, x_Q, x_{PQ}) of $E[2^e]$, together with two hints h_A, h

```

1: if  $A$  is a square then
2:    $h_A \leftarrow 1$ 
3: else
4:    $h_A \leftarrow 0$ 
5: end if
6:  $h \leftarrow 0$ 
7: if  $A$  is a square then
8:   repeat  $h \leftarrow h + 1, \quad x_P \leftarrow -1/(1 + i \cdot h) \cdot A$ 
9:   until  $(1 + h^2)$  is not a square and  $x_P \in E_A(\mathbb{F}_{p^2})$ 
10: else
11:   repeat  $h \leftarrow h + 1, \quad x_P \leftarrow h \cdot A$ 
12:   until  $x_P \in E_A(\mathbb{F}_{p^2})$ 
13: end if
14:  $x_{PQ} \leftarrow -x_P - A$ 
15:  $x_P \leftarrow \text{Ladder}((x_P : 1), (A + 2 : 4), \frac{p+1}{2^e})$ 
16:  $x_{PQ} \leftarrow \text{Ladder}((x_{PQ} : 1), (A + 2 : 4), \frac{p+1}{2^e})$ 
17:  $x_Q \leftarrow \text{ProjectiveDifference}(x_P, x_{PQ}, (A : 1))$ 
18: if  $h \geq 128$  then
19:    $h \leftarrow 0$ 
20: end if
21: return  $(x_P, x_Q, x_{PQ})$  and  $(h_A, h)$ 

```

Algorithm 2.2 TorsionBasisFromHint₂(E, h)

Input: A non-zero affine Montgomery coefficient A , an integer e , together with two hints h_A, h **Output:** An x -only basis (x_P, x_Q, x_{PQ}) of $E[2^e]$

```

1: if  $h_A = 1$  then
2:    $x_P \leftarrow -1/(1 + i \cdot h) \cdot A$ 
3: else
4:    $x_P \leftarrow h \cdot A$ 
5: end if
6:  $x_{PQ} \leftarrow -x_P - A$ 
7:  $x_P \leftarrow \text{Ladder}((x_P : 1), (A + 2 : 4), \frac{p+1}{2^e})$ 
8:  $x_{PQ} \leftarrow \text{Ladder}((x_{PQ} : 1), (A + 2 : 4), \frac{p+1}{2^e})$ 
9:  $x_Q \leftarrow \text{ProjectiveDifference}(x_P, x_{PQ}, (A : 1))$ 
10: return  $(x_P, x_Q, x_{PQ})$ 

```

Algorithm 2.3 TorsionBasisToHint₃(A, e)

Input: A non-zero affine Montgomery coefficient A and an integer e

Output: An x -only basis (x_R, x_S, x_{RS}) of $E[3^e]$, together with two hints h_1, h_2

```

1:  $h_1 \leftarrow 0$ 
2: repeat
3:    $h_1 \leftarrow h_1 + 1, \quad x_R \leftarrow 1 + i \cdot h_1$ 
4:   if  $x_R \notin E(\mathbb{F}_{p^2})$  then
5:     Goto step 2
6:   end if
7:    $x_R \leftarrow \text{Ladder}((x_R : 1), (A + 2 : 4), \frac{p+1}{3^e})$ 
8:    $R \leftarrow (x_R : 1)$ 
9:   for  $i$  from 0 to  $e - 2$  do
10:     $R \leftarrow \text{xTPL}(R, (A + 2 : A - 2))$ 
11:  end for
12: until  $R \neq 0_E$ 
13:  $h_2 \leftarrow 1$ 
14: repeat
15:    $h_2 \leftarrow h_2 + 1, \quad x_S \leftarrow h_2 + i \cdot h_1$ 
16:   if  $x_S \notin E(\mathbb{F}_{p^2})$  then
17:     Goto step 14
18:   end if
19:    $x_S \leftarrow \text{Ladder}((x_S : 1), (A + 2 : 4), \frac{p+1}{3^e})$ 
20:    $S \leftarrow (x_S : 1)$ 
21:   for  $i$  from 0 to  $e - 2$  do
22:     $S \leftarrow \text{xTPL}(S, (A + 2 : A - 2))$ 
23:  end for
24: until  $S \neq 0_E$  and  $S \neq \pm R$ 
25:  $x_{RS} \leftarrow \text{ProjectiveDifference}(x_R, x_S, (A : 1))$ 
26: return  $(x_R, x_S, x_{RS})$  and  $(h_1, h_2)$ 

```

Algorithm 2.4 TorsionBasisFromHint₃($E, (h_1, h_2)$)

Input: A non-zero affine Montgomery coefficient A , an integer e , together with two hints h_1, h_2

Output: An x -only basis (x_R, x_S, x_{RSQ}) of $E[3^e]$

```

1:  $x_R \leftarrow 1 + i \cdot h_1$ 
2:  $x_S \leftarrow h_2 + i \cdot h_1$ 
3:  $x_R \leftarrow \text{Ladder}((x_R : 1), (A + 2 : 4), \frac{p+1}{3^e})$ 
4:  $x_S \leftarrow \text{Ladder}((x_S : 1), (A + 2 : 4), \frac{p+1}{3^e})$ 
5:  $x_{RS} \leftarrow \text{ProjectiveDifference}(x_R, x_S, (A : 1))$ 
6: return  $(x_R, x_S, x_{RS})$ 

```

2.3.2. Discrete logarithm

Let G be a commutative group and $x, y \in G$ be elements such that y is in the subgroup generated by x , the **discrete logarithm problem** (DLP) is the problem of determining an integer $m \geq 1$ such that

$$x^m = y.$$

For elements with a large prime order, this problem is widely believed to be hard on classical computers, forming the foundational security assumption behind traditional Diffie–Hellman cryptography. However, if the prime factors of the order are sufficiently small primes or the order is smooth, then the DLP can be solved efficiently.

Let μ_n denote the group of n -th roots of unity, formally defined as $\mu_n := \{\zeta \in \overline{\mathbb{F}_p} \mid \zeta^n = 1\}$. SQIsign2D² restricts its choice of primes p to those where $\mu_n \subseteq \mathbb{F}_{p^2}^*$, with $n = 2^a, 3^b$ or $2^a \times 3^b$ where $p = f \cdot 2^a \cdot 3^b - 1$. Specifically, within SQIsign2D² we focus on elements $\zeta \in \mu_{l^e} \subseteq \mathbb{F}_{p^2}^*$, where $l = 2$ (respectively 3) and $e \leq a$ (respectively $e \leq b$).

Multiple algorithms exist for computing discrete logarithms in μ_{l^e} , where $e \leq a$ for $l = 2$ and $e \leq b$ for $l = 3$, all rooted in the classic Pohlig–Hellman method [PH03]. In our implementation, we employ the iterative **NormalizedDlog**(Algorithm 2.5) procedure, which implements the Pohlig–Hellman framework to compute the discrete logarithm between two such elements. SQIsign2D² only invokes this algorithm on results from pairing computations.

Algorithm 2.5 NormalizedDlog(ζ_0, ζ_1)

Input: Two values $\zeta_0, \zeta_1 \in \mu_{l^e}$ with $\mu_{l^e} \subseteq \mathbb{F}_{p^2}^*$

Output: The value $k \in [0, l^e]$ such that $\zeta_1 = \zeta_0^k$

```

1: if  $e = 1$  then
2:   return  $k$  such that  $\zeta_1 = \zeta_0^k$  by exhaustive search
3: end if
4:  $e' \leftarrow \lfloor e/2 \rfloor$ 
5:  $\zeta'_0 \leftarrow \zeta_0^{l^{e-e'}}$ 
6:  $\zeta'_1 \leftarrow \zeta_1^{l^{e-e'}}$ 
7:  $k' \leftarrow \text{NormalizedDlog}(\zeta'_0, \zeta'_1)$ 
8:  $\zeta''_0 \leftarrow \zeta_0^{l^{e'}}$ 
9:  $\zeta''_1 \leftarrow \zeta_1^{l^{e'}}$ 
10:  $k'' \leftarrow \text{NormalizedDlog}(\zeta''_0, \zeta''_1)$ 
11: return  $k = k' + l^{e'} k''$ 

```

2.3.3. Pairings on elliptic curves

In this subsection, we will introduce the pairing, which is a tool to compute discrete logarithm problem efficiently. This method has been used to SQIsign to achieve moderate acceleration in [LWX+24]. The main idea is that pairings allow us to translate the elliptic-curve discrete logarithm problem into a finite-field discrete logarithm problem, which can then be solved efficiently with a **NormalizedDlog**(Algorithm 2.5).

The Weil pairing. Let E be an elliptic curve over $\mathbb{F}_q$, and let $n \in \mathbb{N}$ be coprime to p . The Weil pairing of level n is the map

$$e_n : E[n] \times E[n] \rightarrow \mu_n,$$

which is bilinear, alternating, non-degenerate, and Galois invariant. Its construction is given in terms of rational functions attached to divisors and we can compute the Weil pairing by Miller’s algorithm [Mil04]. For the precise definition and further background on the Weil pairing, see [Wei40].

The Tate–Lichtenbaum pairing. Let E be an elliptic curve over $\mathbb{F}_q$, and let $n \in \mathbb{N}$ be coprime to p . The unreduced Tate–Lichtenbaum pairing is the map

$$T_n : E[n] \times E(\mathbb{F}_q)/nE(\mathbb{F}_q) \rightarrow \mathbb{F}_q^*/(\mathbb{F}_q^*)^n,$$

which is bilinear, non-degenerate, and Galois invariant. The reduced Tate–Lichtenbaum pairing is the map

$$t_n : E[n] \times E(\mathbb{F}_q)/nE(\mathbb{F}_q) \rightarrow \mu_n,$$

defined by

$$t_n(P, Q) = T_n(P, Q)^{(q-1)/n}.$$

It is again bilinear, non-degenerate, and Galois invariant. For further details on the Tate–Lichtenbaum pairing, see [Tat62; Lic69; FR94].

When $E/\mathbb{F}_{p^2}$ is a maximal supersingular elliptic curve and $n \mid (p+1)$, the reduced Tate–Lichtenbaum pairing is alternate.

Now we will introduce how to use the pairing to `SQIsign2D2`. The pairing t_n provides an efficient tool for solving discrete logarithm problems on torsion subgroups. We use the fact that, for any $P \in E[n]$, one has $t_n(P, P) = 1$. Suppose that $E[n]$ admits a basis $P_1, P_2 \in E(\mathbb{F}_q)$ such that $\zeta = t_n(P_1, P_2)$ is a primitive n -th root of unity. Let $Q = [a_1]P_1 + [a_2]P_2 \in E[n]$. By bilinearity, we obtain $t_n(P_1, Q) = t_n(P_1, [a_1]P_1 + [a_2]P_2) = t_n(P_1, P_1)^{a_1} t_n(P_1, P_2)^{a_2} = \zeta^{a_2}$. Similarly, $t_n(P_2, Q) = t_n(P_2, P_1)^{a_1} = \zeta^{-a_1}$. Therefore, once ζ is fixed, the coefficients a_1 and a_2 can be recovered by evaluating two Tate pairings of level n and solving the resulting discrete logarithms in μ_n via `NormalizedDlog` (Algorithm 2.5).

In `SQIsign2D2`, this idea is used for $n = 2^e$ with $e \leq a$ and for $n = 3^e$ with $e \leq b$ in order to compute the matrix of a basis transformation.

For the case $n = 3^e$, let (R_1, S_1) be a basis of $E[3^b]$, so that $\zeta = t_{3^b}(R_1, S_1)$ is a primitive 3^b -th root of unity, and let (R_2, S_2) be a basis of $E[3^e]$. Using Tate pairings, we compute coefficients x_1, x_2, x_3, x_4 such that

$$\begin{pmatrix} R_2 \\ S_2 \end{pmatrix} = \begin{pmatrix} x_1 & x_3 \\ x_2 & x_4 \end{pmatrix} \begin{pmatrix} R_1 \\ S_1 \end{pmatrix}.$$

This is the form used in the `ChangeOfBasis3` (Algorithm 2.6) routine. Alternatively, if one first

Algorithm 2.6 `ChangeOfBasis3e`($E, (R_1, S_1), (R_2, S_2)$)

Input: A basis (R_1, S_1) of $E[3^b]$ and a basis (R_2, S_2) of $E[3^e]$

Output: A change-of-basis matrix (x_i) ($1 \leq i \leq 4$) such that $R_2 = [x_1]R_1 + [x_2]S_1$ and $S_2 = [x_3]R_1 + [x_4]S_1$

- 1: $\zeta \leftarrow t_{3^e}(R_1, S_1)$
 - 2: $\zeta_1 \leftarrow t_{3^e}(R_2, S_1)$, $\zeta_2 \leftarrow 1/t_{3^e}(R_2, R_1)$, $\zeta_3 \leftarrow t_{3^e}(S_2, S_1)$, $\zeta_4 \leftarrow 1/t_{3^e}(S_2, R_1)$
 - 3: **for** i from 1 to 4 **do**
 - 4: $x_i \leftarrow 3^{b-e} \cdot \log_\zeta(\zeta_i)$
 - 5: **end for**
 - 6: **return** (x_1, x_2, x_3, x_4)
-

computes the matrix relating (R_2, S_2) to the basis $([3^{b-e}]R_1, [3^{b-e}]S_1)$, then the desired matrix is obtained by restricting to the 3^e -torsion level, or equivalently by applying the same procedure at the appropriate level. The corresponding implementation details are given in Chapter 7. The case $n = 2^e$ is handled in the same way in `ChangeOfBasis2` (Algorithm 2.7).

Algorithm 2.7 $\text{ChangeOfBasis}_{2^e}(E, (P_1, Q_1), (P_2, Q_2))$ **Input:** A basis (P_1, Q_1) of $E[2^a]$ and a basis (P_2, Q_2) of $E[2^e]$ **Output:** A change-of-basis matrix (x_i) ($1 \leq i \leq 4$) such that $P_2 = [x_1]P_1 + [x_2]Q_1$ and $Q_2 = [x_3]P_1 + [x_4]Q_1$

```

1:  $\zeta \leftarrow t_{2^e}(P_1, Q_1)$ 
2:  $\zeta_1 \leftarrow t_{2^e}(P_2, Q_1)$ ,  $\zeta_2 \leftarrow 1/t_{2^e}(P_2, Q_1)$ ,  $\zeta_3 \leftarrow t_{2^e}(P_2, Q_1)$ ,  $\zeta_4 \leftarrow 1/t_{2^e}(P_2, Q_1)$ 
3: for  $i$  from 1 to 4 do
4:    $x_i \leftarrow 2^{b-e} \cdot \log_{\zeta}(\zeta_i)$ 
5: end for
6: return  $(x_1, x_2, x_3, x_4)$ 

```

2.4. Isogenies between elliptic curves

Isogenies. Let E_1 and E_2 be elliptic curves defined over $\mathbb{F}_q$. An isogeny defined over $\mathbb{F}_q$ is a non-constant rational map $\varphi : E_1 \rightarrow E_2$ that sends the infinite point of E_1 to the infinite point of E_2 , that is, $\varphi(0_{E_1}) = 0_{E_2}$. Equivalently, φ can be written in the form $\varphi(x, y) = (f(x, y), g(x, y))$, where $f(x, y)$ and $g(x, y)$ are rational functions in $\mathbb{F}_q(E_1)$. Here $\mathbb{F}_q(E_1)$ is the fractional field of $\mathbb{F}_q[x, y]/\langle y^2 - f(x) \rangle$, where $E_1 : y^2 = f(x)$. As we known, non-constant isogeny is surjective and has finite kernel. Two elliptic curves are said to be **isogenous** if there exists an isogeny between them. From Tate's theorem, we have two curves E_1 and E_2 are isogenous over $\mathbb{F}_q$ if and only if $\#E_1(\mathbb{F}_q) = \#E_2(\mathbb{F}_q)$.

Let E_1, E_2 be Montgomery curves, then φ can be represented by a rational map of the form $\varphi(x, y) = \left(\frac{u(x)}{v(x)}, \frac{s(x)}{t(x)} y \right)$, where $u(x), v(x), s(x), t(x) \in \mathbb{F}_q[x]$, with $\gcd(u(x), v(x)) = 1$, $\gcd(s(x), t(x)) = 1$, and satisfying $v^3(x) \mid t^2(x), t^2(x) \mid v^3(x)f_1(x)$. Moreover, $\deg(\varphi) = \max\{\deg u(x), \deg v(x)\}$.

If $\varphi : E_1 \rightarrow E_2$ is an isogeny, then φ is a group homomorphism with respect to the group law on elliptic curves. Its kernel, denoted by $\ker(\varphi)$, is therefore a finite subgroup of E_1 . We next recall several standard facts about isogenies that will be used later.

Dual isogeny. Given an isogeny $\varphi : E_1 \rightarrow E_2$, there exists a unique isogeny $\hat{\varphi} : E_2 \rightarrow E_1$, such that $\varphi \circ \hat{\varphi} = [\deg(\varphi)]$, $\hat{\varphi} \circ \varphi = [\deg(\varphi)]$. The isogeny $\hat{\varphi}$ is called the **dual isogeny** of φ . If the kernel of φ is a cyclic group $\langle P \rangle$, and the order of P is n , for any $Q \in E_1[n]$ such that $\{P, Q\}$ is a basis of $E_1[n]$, we have $\ker(\hat{\varphi}) = \langle \varphi(Q) \rangle$.

Vélu's formula. Let G be a finite subgroup of $E(\overline{\mathbb{F}}_q)$. Then there exists a unique elliptic curve E' and an isogeny $\varphi : E \rightarrow E'$ such that $\ker(\varphi) = G$. For such group G , Vélu's formulas [Vél71; BDL+20] provide an explicit construction of the codomain curve E' and of the corresponding isogeny. The computational cost of Vélu's formulas, together with their variants, is $\tilde{O}(\sqrt{N})$, where $N = \#G$. As a result, these formulas are practical only when N is relatively small.

When N is large and composite, it is more efficient to decompose the isogeny into a sequence of lower-degree isogenies. Writing $N = \prod_i \ell_i^{e_i}$ and $s = \sum_i e_i$ for the prime factorization of N , we can decompose φ as $\varphi = \varphi_s \circ \cdots \circ \varphi_2 \circ \varphi_1$, where each φ_j has prime degree dividing N . This approach is effective when N is **smooth**, namely when all of its prime factors are sufficiently small.

In the following sections, we give explicit formulas for isogenies of degrees 2, 3, and 4 which will be used in SQIsign2D^2 . All elliptic curves considered there are written in Montgomery form.

2-Isogenies Let $Q \in E_{A,B}$ be a point of order 2 that generates the kernel of a 2-isogeny $\varphi : E_{A,B} \rightarrow E_{A',B'}$. We split the discussion into two distinct cases:

- **Case** $Q = (0, 0)$: The isogeny is given by

$$\varphi(x, y) = \left(\frac{1}{\sqrt{A^2 - 4}} \cdot \frac{x^2 + Ax + 1}{x}, \frac{1}{\sqrt[4]{A^2 - 4}} \cdot y \cdot \frac{x^2 - 1}{x^2} \right),$$

and the codomain curve has parameters

$$(A', B') = \left(-\frac{2A}{\sqrt{A^2 - 4}}, B \right).$$

- **Case** $Q = (x_Q, 0)$ **with** $x_Q \neq 0$: Following the constructions from [DLL+23; Ren18], we define isogeny as

$$\varphi(x, y) = \left(\frac{x_Q x^2 - x}{x - x_Q}, \sqrt{x_Q} \cdot y \cdot \frac{x_Q x^2 - 2x_Q^2 x + x_Q}{(x - x_Q)^2} \right),$$

with the codomain parameters

$$(A', B') = (2(1 - 2x_Q^2), B).$$

4-Isogenies To improve computational efficiency, some implementations may choose 4-isogeny formulas instead of computing 2-isogenies. For compatibility, these 4-isogenies must be equivalent to the composition of the 2-isogeny formulas presented above. Let $Q \in E_{A,B}$ be a point of order 4, and let $\varphi : E_{A,B} \rightarrow E_{A',B'}$ denote the associated 4-isogeny. We consider three distinct cases:

- **Case** $Q = (1, y_Q)$: Here $[2]Q = (0, 0)$. Composing the first and second 2-isogeny formulas yields

$$\varphi(x, y) = \left(\frac{1}{A - 2} \cdot \frac{(x + 1)^2(x^2 + Ax + 1)}{x(x - 1)^2}, \frac{1}{\sqrt[3]{A - 2}} \cdot y \cdot \frac{(x + 1)(x^4 - 4x^3 - 2x^2 - 4Ax^2 - 4x + 1)}{x^2(x - 1)^3} \right),$$

with the codomain curve parameters

$$(A', B') = \left(2 \cdot \frac{6 + A}{2 - A}, B \right).$$

- **Case** $Q = (-1, y_Q)$: Analogously, we obtain

$$\varphi(x, y) = \left(\frac{1}{A + 2} \cdot \frac{(x - 1)^2(x^2 + Ax + 1)}{x(x + 1)^2}, \frac{1}{\sqrt[3]{A + 2}} \cdot y \cdot \frac{(x - 1)(x^4 + 4x^3 - 2x^2 + 4Ax^2 + 4x + 1)}{x^2(x + 1)^3} \right),$$

and the codomain parameters

$$(A', B') = \left(2 \cdot \frac{6 - A}{2 + A}, B \right).$$

- **Case** $Q = (x_Q, y_Q)$ **with** $x_Q \neq \pm 1$: The isogeny is given by

$$\varphi(x, y) = \left(\frac{(x(x_Q^2 + 1) - 2x_Q)(xx_Q - 1)^2 x}{(2xx_Q - x_Q^2 - 1)(x - x_Q)^2}, x_Q^2 \sqrt{2(x_Q^3 + x_Q)} \cdot y \cdot \frac{(xx_Q - 1)(8x^2 x_Q^2 + (x_Q^2 + 1)(x^4 - 4x^3 x_Q + 2x^2 x_Q^2 - 4xx_Q + 1))}{(x - x_Q)^3 (2xx_Q - x_Q^2 - 1)^2} \right),$$

with codomain parameters

$$(A', B') = (2 - 4x_Q^4, B).$$

3-Isogenies Let $Q = (x_Q, y_Q) \in E_{A,B}$ be a point of order 3 that generates the kernel of a 3-isogeny $\varphi : E_{A,B} \rightarrow E_{A',B'}$. Then $E_{A',B'}$ can be computed as

$$(A', B') = \left((Ax_Q - 6x_Q^2 + 6)x_Q, Bx_Q^2 \right),$$

and the isogeny is given by

$$\varphi(x, y) = \left(\frac{x(xx_Q - 1)^2}{(x - x_Q)^2}, y \cdot \frac{(xx_Q - 1)(x^2x_Q - 3xx_Q^2 + x + x_Q)}{(x - x_Q)^3} \right).$$

The details of these algorithms can be found in Chapter 7.

2.5. Principally polarized abelian surfaces and isogenies

In SQIsign2D², we use the method of computing 2 dimension isogenies to efficiently represent 1 dimension isogenies with non-smooth degree. The objects we consider are principally polarized superspecial abelian surfaces. Over an algebraically closed field, every such surface is isomorphic to one of the following [OU73]:

1. a product $E_1 \times E_2$ of supersingular elliptic curves;
2. the Jacobian $\text{Jac}(C)$ of a genus-2 hyperelliptic curve C .

The first case is called product case and the second Jacobian case. Both occur in our implementation.

Product case When the abelian surface is a product $A = E_1 \times E_2$, the group operation follows immediately from the addition of elliptic curves. For points $(P_1, P_2), (Q_1, Q_2) \in A$, $(P_1, P_2) \oplus (Q_1, Q_2) = (P_1 \oplus_1 Q_1, P_2 \oplus_2 Q_2)$, where $\oplus_1$ and $\oplus_2$ denote the group laws on E_1 and E_2 , and the identity element is $0_A = (0_{E_1}, 0_{E_2})$.

Jacobian case For $p > 5$, every genus-2 hyperelliptic curve over $\mathbb{F}_{p^2}$ can be represented by an equation $C : y^2 = f(x)$, where f is squarefree and $\deg(f) \in \{5, 6\}$. Since the Jacobian $\text{Jac}(C)$ is an abelian variety, there exists a group law on it [Mil86]. When E is an elliptic curve, we have $\text{Jac}(E) \cong E$. In the case of genus 2, the group law on $\text{Jac}(C)$ can be computed using Cantor's algorithms [Can87].

Isogenies Let A_1 and A_2 be principally polarized abelian surfaces over $\overline{\mathbb{F}}_q$. An isogeny $\Phi : A_1 \rightarrow A_2$ between abelian varieties is a surjective map defined coordinate-wise by rational fractions which is also a group homomorphism. There are three types of 2-dimensional isogenies considered in SQIsign2D²:

- if A_1 is a product and A_2 is a Jacobian, then Φ is called a **gluing isogeny**;
- if A_1 is a Jacobian and A_2 is a product, then Φ is called a **splitting isogeny**;
- if both A_1 and A_2 are Jacobians, then Φ is called **generic**.

$\Phi : A_1 \rightarrow A_2$ is called (N, N) -isogeny if the kernel of Φ is generated by two linearly independent points $P, Q \in A_1[N]$ where linearly independent means $[x]P + [y]Q = 0_{A_1}$ if and only if $N \mid x$ and $N \mid y$.

The independence condition is not sufficient by itself. A pair of independent N -torsion points defines an isogeny between principally polarized abelian surface when the subgroup they

generate is maximal N -isotropic subgroup. Equivalently, the subgroup is generated by N -torsion points P, Q and the N -Weil pairing must be trivial, that is $e_N(P, Q) = 1$.

If there is a maximal N -isotropic subgroup G , generalized V  lu formulas recover the corresponding isogeny from this kernel data in time $O(N^2)$ [LR23].

If $N = \prod_i \ell_i^{e_i}$, the map can be written as a composition of isogenies with sufficiently small degree,

$$\Phi = \Phi_{r,r_t} \circ \Phi_{r,1} \circ \cdots \circ \Phi_{1,r_1} \circ \Phi_{1,1},$$

with each $\Phi_{i,j}$ ($j = 1, \dots, j_t$) an (ℓ_i, ℓ_i) -isogeny. Then we can compute Φ with complexity $O\left(\sum_i e_i \ell_i^2\right)$.

SQIsign2D² only computes $(2^e, 2^e)$ -isogeny

$$\Phi : E_1 \times E_2 \rightarrow E_3 \times E_4$$

between products of elliptic curves over $\mathbb{F}_{p^2}$ and we can write it as a chain of $(2, 2)$ -isogenies of length e . Since the principally polarized superspecial abelian surfaces are almost jacobian, the chain in our implement can be represented as the following: the first step is gluing, the last step is splitting, and all steps in between are generic.

2.5.1. Implementation of $(2, 2)$ -isogenies.

The implementation described in Section 7.5 represents points on a principally polarized abelian surface by level-2 theta coordinates. This representation plays the same role for surfaces as the x -coordinate for elliptic curves. The routines used for a single $(2, 2)$ -step are listed as follows:

- **ThetaDBL**(Algorithm 7.35): input a point P in theta coordinates together with constants `consts`; output the theta coordinates of $[2]P$.
- **GenericCodomainWith8Torsion**(Algorithm 7.36): input theta coordinates of 8-torsion points T_1'', T_2'' on a surface A ; output the dual theta null point $(\alpha : \beta : \gamma : \delta)$, its inverse $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$, and the theta null point 0_B of the codomain B of the isogeny $\Phi : A \rightarrow B$ with kernel $\ker(\Phi) = \langle [4]T_1'' \rangle \oplus \langle [4]T_2'' \rangle$.
- **GenericCodomainWith4Torsion**(Algorithm 7.38): input the theta coordinates of a 4-torsion point T_1' on a surface A such that $[2]T_1' \in \ker(\Phi)$, together with the theta null point 0_A ; output the dual theta null point $(\alpha : \beta : \gamma : \delta)$, its inverse $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$, and the theta null point 0_B of the image of $\Phi : A \rightarrow B$.
- **GenericCodomain**(Algorithm 7.39): input the theta null point 0_A of a surface A ; output the dual theta null point $(\alpha : \beta : \gamma : \delta)$, its inverse $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$, and the theta null point 0_B of the codomain of a $(2, 2)$ -isogeny $\Phi : A \rightarrow B$.
- **GenericEval**(Algorithm 7.40): input a point P on the source surface A in theta coordinates and the inverse dual theta null point $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$; output the theta coordinates of $\Phi(P)$.
- **GluingCodomain**(Algorithm 7.44): input theta coordinates of 8-torsion points T_1'', T_2'' on a product surface; output the dual theta null point $(\alpha : \beta : \gamma : 0)$, its inverse $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : 0)$, the theta null point 0_B of the image surface A of the isogeny $\Phi : E_1 \times E_2 \rightarrow A$ with kernel $\ker(\Phi) = \langle [4]T_1'' \rangle \oplus \langle [4]T_2'' \rangle$, the dual J of the theta point $\Phi(T_1'')$, and a change-of-basis matrix $\mathcal{N}$ used again during evaluation.
- **GluingEval**(Algorithm 7.51): input a point $P \in E_1 \times E_2$, an 8-torsion point T_1'' satisfying $[4]T_1'' \in \ker(\Phi)$, the point J defined as the dual of the theta point $\Phi(T_1'')$ on A , and the matrix $\mathcal{N}$ returned by **GluingCodomain**; output the theta coordinates of $\Phi(P)$.

- **GluingEvalSpecial**(Algorithm 7.52): input a point $P \in E_1 \times E_2$ of the form $(P_1, 0)$ or $(0, P_2)$, the inverse $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : 0)$ of the dual theta null point on A , and the matrix $\mathcal{N}$ computed by **GluingCodomain**; output the theta coordinates of $\Phi(P)$.
- **SplittingIsomorphism**(Algorithm 7.46): input the theta null point 0_A on a surface $A \cong E_1 \times E_2$; output the isomorphism sending 0_A to the theta null point attached to the product theta structure.

Algorithmic details are deferred to Chapter 7.

2.5.2. Chains of $(2, 2)$ -isogenies between products of elliptic curves

Consider a $(2^e, 2^e)$ -isogeny

$$\Phi : E_1 \times E_2 \rightarrow E_3 \times E_4$$

and assume the generators P, Q of $\ker(\Phi)$ are known. The computation is organized as a chain

$$E_1 \times E_2 \xrightarrow{\Phi_1} A_1 \xrightarrow{\Phi_2} A_2 \longrightarrow \cdots \longrightarrow A_{e-1} \xrightarrow{\Phi_e} E_3 \times E_4.$$

For each step, we need to determine the theta null point of the codomain. Once this data is available, the corresponding isogeny can be evaluated. A direct construction of the chain is obtained as follows.

The first step is computing the gluing isogeny $\Phi_1 : E_1 \times E_2 \rightarrow A_1$. It is computed from the 8-torsion points $[2^{e-3}]P, [2^{e-3}]Q$ by **GluingCodomain**(Algorithm 7.44), where the torsion points lie above the kernel $\ker(\Phi_1) = \langle [2^{e-1}]P, [2^{e-1}]Q \rangle$. We can also compute the image points $\Phi_1(P), \Phi_1(Q)$ by **GluingEval**(Algorithm 7.51).

For every step $\Phi_i : A_{i-1} \rightarrow A_i$, $i \in \{2, \dots, e-2\}$, the input consists of the 8-torsion points $[2^{e-i-2}](\Phi_{i-1} \circ \cdots \circ \Phi_1(P)), [2^{e-i-2}](\Phi_{i-1} \circ \cdots \circ \Phi_1(Q))$, which lie above $\ker(\Phi_i)$. The codomain data is then computed with **GenericCodomainWith8Torsion**(Algorithm 7.36).

For the last two steps, we first consider $\Phi_{e-1} : A_{e-2} \rightarrow A_{e-1}$ which can be computed from the 4-torsion points $\Phi_{e-2} \circ \cdots \circ \Phi_1(P), \Phi_{e-2} \circ \cdots \circ \Phi_1(Q)$ by **GenericCodomainWith4Torsion**(Algorithm 7.38), where the 4-torsion points lie above $\ker(\Phi_{e-1})$.

The final map $\Phi_e : A_{e-1} \rightarrow E_3 \times E_4$ is determined from the 2-torsion kernel points $\Phi_{e-1} \circ \cdots \circ \Phi_1(P), \Phi_{e-1} \circ \cdots \circ \Phi_1(Q)$ using **GenericCodomain**(Algorithm 7.38).

After the last step, in order to express image points on $E_3 \times E_4$ in $(X : Z)$ -Montgomery coordinates on each component E_3 and E_4 , one can compute the change of coordinates from the system of theta coordinates on $E_3 \times E_4$ induced by Φ_e to product theta coordinates by **SplittingIsomorphism**(Algorithm 7.38).

There is a standard method to avoid the square-root computations that occur in the last two steps. Assume that one starts from isotropic points $P, Q \in (E_1 \times E_2)[2^{e+2}]$ lying above $\ker(\Phi)$, in the sense that $\ker(\Phi) = \langle [4]P, [4]Q \rangle$ and $e_{2^{e+2}}(P, Q) = 1$. Then, for every $i \in \{2, \dots, e\}$, the isogeny $\Phi_i : A_{i-1} \rightarrow A_i$ is computed from the 8-torsion points $[2^{e-i}](\Phi_{i-1} \circ \cdots \circ \Phi_1(P)), [2^{e-i}](\Phi_{i-1} \circ \cdots \circ \Phi_1(Q))$, which lie above $\ker(\Phi_i)$, by calling **GenericCodomainWith8Torsion**(Algorithm 7.38). This optimization is available only when the full 2^{e+2} -torsion is defined over $\mathbb{F}_{p^2}$; this condition is not automatic.

The implementation does not follow the direct method above literally. Instead, the chain is built with a balanced strategy. By storing intermediate points obtained during the doublings and pushing them through each isogeny, we can reduce the number of executions of the doubling **ThetaDBL**(Algorithm 7.35) to a quasi-linear number $O(e \log e)$. We use optimal strategies in the sense introduced for SIDH/SIKE [JD11b, Section 4.2.2].

Failure probability. A chain computation may fail to terminate if a splitting occurs before the last step. Chapter 8 shows that, for the chains arising in key generation and signing, the probability of this event is $\tilde{O}(1/p)$. For verification of an honest signature, this failure does not occur. So the verification procedure rejects whenever such a case is encountered.

Implementation of isogeny chains. Section 2.5.2 gives the algorithms used to build complete chains from the elementary $(2, 2)$ -procedures listed above. There are some algorithms:

- **Isogeny22Chain**(Algorithm 7.50): input isotropic points $P, Q \in E_1 \times E_2$ of order 2^e , together with an array `pts` of points on $E_1 \times E_2$; output a chain $\Phi = \Phi_e \circ \dots \circ \Phi_1$ of $(2, 2)$ -isogenies satisfying $\ker(\Phi) = \langle P, Q \rangle$, together with the evaluated images $\{\Phi(R) : R \in \text{pts}\}$, computed with optimal strategies.
- **Isogeny22ChainWithTorsion**(Algorithm 7.53): input isotropic points $P, Q \in E_1 \times E_2$ of order 2^{e+2} , together with an array `pts` of points on $E_1 \times E_2$; output a chain $\Phi = \Phi_e \circ \dots \circ \Phi_1$ of $(2, 2)$ -isogenies satisfying $\ker(\Phi) = \langle P, Q \rangle$ and the evaluated images $\{\Phi(R) : R \in \text{pts}\}$, computed with balanced strategies.

More details of the algorithms can be found in Chapter 7.

2.6. Lattice

2.6.1. Big integers

The implementation of `SQIsign2D2` requires arithmetic on integers of a priori unbounded size. The exact bit length depends on the chosen parameter set, and intermediate values may grow beyond what can be conveniently bounded in advance. For this reason, the implementation relies on a dynamic multiple-precision integer library. In practice, the code uses GMP¹. If explicit upper bounds for all intermediate quantities are derived, one may instead use a fixed-precision implementation.

The routines required here are standard and available in common big-integer libraries. In particular, the implementation assumes support for the following operations:

- Integer arithmetic, including addition, subtraction, multiplication, and related basic operations;
- Uniform sampling from an integer interval;
- Approximate and exact integer square-root computations;
- Probabilistic primality testing via Miller–Rabin;
- Extended greatest common divisor (XGCD): on input (a, b) , compute (g, u, v) such that $ua + bv = g = \gcd(a, b) > 0$, and, when both $a \neq 0$ and $b \neq 0$, the output additionally satisfies $1 \leq au \leq \frac{|ab|}{g}$, $-\frac{|ab|}{g} < bv \leq 0$;
- Arithmetic modulo an integer;
- computation of the Legendre symbol;
- evaluation of the dyadic valuation of an integer, denoted by `DyadicValuation`;
- extraction of square roots modulo prime moduli.

All of the above are directly available in GMP, except for modular square roots, for which the implementation uses the routine `ModularSQRT` (Algorithm 2.8) given below. One additional point is that the XGCD routine provided by GMP does not enforce the condition $u \neq 0$, whereas this condition is required by `SQIsign2D2`.

¹<https://gmplib.org/>

Algorithm 2.8 ModularSqrt(n, m)**Input:** An odd prime m and an integer n such that n is a quadratic residue modulo m .**Output:** An integer x satisfying $x^2 \equiv n \pmod{m}$.

```

1: if  $n \equiv 0 \pmod{m}$  then
2:   return 0
3: end if
4: if  $m \equiv 3 \pmod{4}$  then
5:   return  $n^{(m+1)/4} \bmod m$ 
6: else if  $m \equiv 5 \pmod{8}$  then
7:   if  $n^{(m-1)/4} \not\equiv 1 \pmod{m}$  then
8:     return  $n^{(m+3)/8} \bmod m$ 
9:   else
10:    return  $2n(4n)^{(m-5)/8} \bmod m$ 
11:   end if
12: end if                                     // At this point  $m \equiv 1 \pmod{8}$ ; use Tonelli–Shanks
13:  $w \leftarrow 2$ 
14:  $e \leftarrow \text{DyadicValuation}(m-1)$ 
15:  $q \leftarrow (m-1)/2^e$ 
16: while  $w$  is a square modulo  $m$  do
17:    $w \leftarrow w+1$ 
18: end while
19:  $z \leftarrow w^q \bmod m$ 
20:  $y \leftarrow n^q \bmod m$ 
21:  $x \leftarrow n^{(q+1)/2} \bmod m$ 
22:  $f \leftarrow 2^{e-2}$ 
23: for  $i = 0$  to  $e-1$  do
24:    $b \leftarrow y^f \bmod m$ 
25:   if  $b = m-1$  then
26:      $x \leftarrow xz \bmod m$ 
27:      $y \leftarrow yz^2 \bmod m$ 
28:      $z \leftarrow z^2 \bmod m$ 
29:   end if
30:    $f \leftarrow f/2$ 
31: end for
32: return  $x$ 

```

2.6.2. Floating-point support

The implementation uses floating-point arithmetic only in the lattice-reduction routines. Native platform formats are not always sufficient. A format with at least a 24-bit mantissa and an exponent range of at least 20 bits is sufficient for all parameter sets considered here.

The reference implementation realizes this format by storing the mantissa in a native floating-point type and the exponent separately in an integer type, using the “double plus exponent” header library [PZ24].

Because lattice reduction is numerically sensitive, the reduced basis may depend on details such as operation ordering, rounding behavior, and compiler settings. As a result, different implementations need not reproduce the Known Answer Tests of Chapter 5 exactly. This does not prevent interoperability: signatures produced in this way remain valid and are accepted by a correct implementation of `SQIsign2D2.Verify` (Algorithm 3.3).

2.6.3. Basic integer linear algebra

In the implementation of `SQIsign2D2`, several small integer matrices must be handled. Most of them have dimension 2 or 4, while some routines also involve matrices of size $4 \times n$ or $n \times 4$ with $n \leq 16$. We consider matrices having coefficients in $\mathbb{Z}$ or in $\mathbb{Z}/d\mathbb{Z}$ for some integer d .

Multiplication Elementary linear-algebra operations, such as matrix–vector and matrix–matrix multiplication, are implemented with the standard schoolbook method.

Determinant and inversion For 2×2 matrices, we use the standard formulae:

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc, \quad \begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

For 4×4 matrices, the determinant and adjugate can be computed using Laplace-like formulae [Ebe07], for a total cost of 78 ring operations.

Hermite normal form (HNF) The Hermite normal form (HNF) is the standard normal form used for integer matrices. A matrix $\mathbf{H}$ is in **column-style HNF** if:

- $\mathbf{H}$ is upper triangular, namely $h_{i,j} = 0$ for $j < i$, and all zero columns are placed on the left;
- in every non-zero row, the pivot is positive and occurs strictly to the right of the pivot in the row above;
- all entries to the left of a pivot are zero, and all entries to the right are non-negative and strictly smaller than the pivot.

For an integer matrix $\mathbf{A}$, its HNF is a matrix $\mathbf{H}$ such that

$$\mathbf{AU} = \mathbf{H}$$

for some unimodular matrix $\mathbf{U} \in \text{GL}_n(\mathbb{Z})$, with $\mathbf{H}$ in HNF. The matrices $\mathbf{A}$ and $\mathbf{H}$ therefore generate the same column space, and $\mathbf{H}$ is uniquely determined by $\mathbf{A}$. It thus serves as a canonical representation of the column space. Algorithms for computing HNF can be found in [Coh13, Section 2.4.2]. In SQIsign2D², this routine is used on full-rank integer matrices of size 4×4 , 4×8 , and 4×16 . HNF(Algorithm 2.9) gives one example; any efficient routine with the same output is equivalent for implementation purposes.

Algorithm 2.9 HNF($\mathbf{M}$)

Input: An integer matrix $\mathbf{M}$ with d rows and $c \geq d$ columns of rank d . Columns are $\mathbf{M}_1^t, \dots, \mathbf{M}_c^t$; entry (r, l) is $\mathbf{M}_{r,l}$.

Output: The HNF of $\mathbf{M}$.

```

1: for  $i \leftarrow d$  down to 1 do
2:   for  $j \leftarrow i - 1$  down to 1 do
3:     if  $\mathbf{M}_{i,i} = 0$  and  $\mathbf{M}_{i,j} = 0$  then
4:        $g, u, v \leftarrow 1, 1, 0$ 
5:     else
6:        $g, u, v \leftarrow \text{XGCD}(\mathbf{M}_{i,i}, \mathbf{M}_{i,j})$  //  $u\mathbf{M}_{i,i} + v\mathbf{M}_{i,j} = g, u > 0$ 
7:     end if
8:      $\mathbf{M}_i^t \leftarrow u\mathbf{M}_i^t + v\mathbf{M}_j^t$  //  $\mathbf{M}_{i,i} = g$ 
9:   end for
10:  for  $j \leftarrow i - 1$  down to 1 do
11:     $g \leftarrow \mathbf{M}_{i,j} / \mathbf{M}_{i,i}$ 
12:     $\mathbf{M}_j^t \leftarrow \mathbf{M}_j^t - g\mathbf{M}_i^t$  //  $\mathbf{M}_{i,j} = 0$ 
13:  end for
14:  for  $j \leftarrow i + 1$  to  $c$  do
15:     $r \leftarrow \mathbf{M}_{i,j} \bmod \mathbf{M}_{i,i}$  //  $0 \leq r < \mathbf{M}_{i,i}$ 
16:     $g \leftarrow (\mathbf{M}_{i,j} - r) / \mathbf{M}_{i,i}$ 
17:     $\mathbf{M}_j^t \leftarrow \mathbf{M}_j^t - g\mathbf{M}_i^t$  //  $\mathbf{M}_{i,j} = r$ 
18:  end for
19: end for
20: return  $\mathbf{M}$ 
```

2.6.4. Lattices

Let V be a finite-dimensional $\mathbb{Q}$ -vector space. We call V a **quadratic space** if it is equipped with a symmetric bilinear form $\langle \cdot, \cdot \rangle : V \times V \rightarrow \mathbb{Q}$, that is, a map satisfying, for all $\mathbf{x}, \mathbf{y}, \mathbf{z} \in V$ and $\lambda \in \mathbb{Q}$,

- $\langle \mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{y}, \mathbf{x} \rangle$;
- $\langle \mathbf{x} + \mathbf{y}, \mathbf{z} \rangle = \langle \mathbf{x}, \mathbf{z} \rangle + \langle \mathbf{y}, \mathbf{z} \rangle, \langle \mathbf{x}, \mathbf{y} + \mathbf{z} \rangle = \langle \mathbf{x}, \mathbf{y} \rangle + \langle \mathbf{x}, \mathbf{z} \rangle$;
- $\langle \lambda \mathbf{x}, \mathbf{y} \rangle = \lambda \langle \mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{x}, \lambda \mathbf{y} \rangle$.

Two vectors $\mathbf{x}$ and $\mathbf{y}$ are said to be **orthogonal** if $\langle \mathbf{x}, \mathbf{y} \rangle = 0$. Every quadratic space admits an **orthogonal basis**, namely a basis in which distinct basis vectors are pairwise orthogonal.

Associated with the bilinear form is the **quadratic form** $Q(\mathbf{x}) = \langle \mathbf{x}, \mathbf{x} \rangle$. If $Q(\mathbf{x}) > 0$ for every non-zero $\mathbf{x} \in V$, then Q is said to be **positive definite**; in that case, the bilinear form and the quadratic space are also called positive definite. We then refer to $Q(\mathbf{x})$ as the **length** of $\mathbf{x}$. A basic example is the standard inner product on $\mathbb{Q}^n$, whose associated quadratic form is the squared Euclidean norm $\|\mathbf{x}\|^2$.

A **lattice** in V is the set of all integer linear combinations of a basis of V . Two bases generate the same lattice if and only if the absolute value of the determinant of the corresponding change-of-basis matrix is 1. Given a basis $\mathbf{y}_0, \dots, \mathbf{y}_{d-1}$ of a quadratic space, the associated **Gram matrix** is the symmetric matrix whose (i, j) -entry is $\langle \mathbf{y}_i, \mathbf{y}_j \rangle$. This matrix determines the bilinear form on the lattice spanned by $\mathbf{y}_0, \dots, \mathbf{y}_{d-1}$. In particular, it is diagonal exactly when the basis is orthogonal. In general, however, a lattice may not admit an orthogonal basis. The purpose of **lattice reduction** is therefore to compute a basis that is, in a precise sense, as close to orthogonal as possible.

Lattice reduction Let $\mathbf{y}_0, \dots, \mathbf{y}_{d-1}$ be a basis of a positive definite quadratic space. Define the **Gram-Schmidt vectors** $\bar{\mathbf{y}}_i$ as

$$\bar{\mathbf{y}}_i = \mathbf{y}_i - \sum_{j=0}^{i-1} \mu_{i,j} \bar{\mathbf{y}}_j, \quad \text{where} \quad \mu_{i,j} = \frac{r_{i,j}}{r_{j,j}}, \quad r_{i,j} = \langle \mathbf{y}_i, \bar{\mathbf{y}}_j \rangle. \quad (2.10)$$

The basis is said to be (η, δ) -**reduced** for parameters $\frac{1}{2} < \eta < 1$ and $\frac{1}{4} < \delta < 1$ if:

- $|\mu_{i,j}| < \eta$ for $0 \leq j < i < d$, and
- $\langle \bar{\mathbf{y}}_i, \bar{\mathbf{y}}_i \rangle \geq (\delta - \mu_{i,i-1}^2) \langle \bar{\mathbf{y}}_{i-1}, \bar{\mathbf{y}}_{i-1} \rangle$ for $1 \leq i < d$.

A symmetric matrix is said to be (η, δ) -**reduced** if it is the Gram matrix of an (η, δ) -reduced basis.

L2(Algorithm 2.10) of [NS09] takes as input a basis of a lattice and its associated Gram matrix and outputs an (η, δ) -reduced basis for the same lattice, along with its associated Gram matrix. This algorithm only applies (multiple precision) integer operations on the lattice basis and the Gram matrix, however, it uses floating-point operations to keep track of the values $r_{i,j}$ and $\mu_{i,j}$ for $i \geq j$, called the **GSO family** in [NS09] (see also [NV10, Chapter 5]). We reproduce pseudocode for **L2**(Algorithm 2.10) below. Numbers, vectors and matrices that are stored as floating-point values are indicated by the type annotation `:: FLOAT`. Similarly, all casts from integer to floating point are explicitly indicated by the operator `FLOAT()`.

SQIsign2D² manipulates quadratic spaces and lattices of dimension 4. The elements of the space are integer column vectors of length 4. Thus a lattice basis can be represented by a 4×4 integer matrix, and the operations in **SizeReduce**(Algorithm 2.12) and **InsertBefore**(Algorithm 2.13) are easily understood as multiplying it by elementary unimodular matrices.

Algorithm 2.10 $L_{2,\eta,\delta}(\mathbf{b}_0, \dots, \mathbf{b}_{d-1}, \mathbf{G})$ **Input:** A basis $(\mathbf{b}_0, \dots, \mathbf{b}_{d-1})$ of a d -dimensional quadratic space, its associated $d \times d$ Gram matrix $\mathbf{G}$.**Input:** Parameters: $\frac{1}{2} < \eta < 1$, $\frac{1}{4} < \delta < 1$.**Output:** A (η, δ) -reduced basis of the same lattice, its associated Gram matrix.

```

1:  $\bar{\delta} \leftarrow \text{FLOAT}\left(\frac{\delta+1}{2}\right)$ ,  $\bar{\eta} \leftarrow \text{FLOAT}\left(\frac{\eta+0.5}{2}\right)$ 
2:  $r_{0,0} \leftarrow \text{FLOAT}(\mathbf{G}_{0,0})$ ,  $\mu_{0,0} \leftarrow \text{FLOAT}(1)$ ,  $T \leftarrow [\text{FLOAT}(0), \text{FLOAT}(0), \text{FLOAT}(0), \text{FLOAT}(0)]$ 
3:  $k \leftarrow 1$ 
4: while  $k < d$  do
5:    $(\mathbf{b}_0, \dots, \mathbf{b}_{d-1}), \mathbf{G}, r, \mu \leftarrow \text{SizeReduce}((\mathbf{b}_0, \dots, \mathbf{b}_{d-1}), \mathbf{G}, k, r, \mu, \bar{\eta})$ 
6:    $T_0 \leftarrow \text{FLOAT}(\mathbf{G}_{k,k})$ 
7:   for  $i$  from 1 up to  $k-1$  do
8:      $T_i \leftarrow T_{i-1} - \mu_{k,(i-1)} r_{k,(i-1)}$ 
9:   end for
10:   $s \leftarrow \min\{0 \leq i \leq k \mid \text{such that } T_j < \bar{\delta} r_{j,j} \text{ for all } i \leq j < k\}$ 
11:  if  $k \neq s$  then
12:     $(\mathbf{b}_0, \dots, \mathbf{b}_{d-1}), \mathbf{G}, k, s, r, \mu \leftarrow \text{InsertBefore}((\mathbf{b}_0, \dots, \mathbf{b}_{d-1}), \mathbf{G}, k, s, r, \mu)$ 
13:     $k \leftarrow s$ 
14:  end if
15:   $k \leftarrow k + 1$ 
16: end while
17: return  $(\mathbf{b}_0, \dots, \mathbf{b}_{d-1}), \mathbf{G}$ 

```

Algorithm 2.11 $\text{ExtendGSOFamily}(\mathbf{G}, k, r :: \text{FLOAT}, \mu :: \text{FLOAT})$ **Input:** A $d \times d$ Gram matrix $\mathbf{G}$, index $1 \leq k < d$, the GSO family r, μ up to row $k-1$.**Output:** The GSO family r, μ up to row k .

```

1: for  $j$  from 0 up to  $k$  do
2:    $r_{k,j} \leftarrow \text{FLOAT}(\mathbf{G}_{k,j})$ 
3:   for  $l$  from 0 up to  $j-1$  do
4:      $r_{k,j} \leftarrow r_{k,j} - r_{k,l} \mu_{j,l}$ 
5:   end for
6:   if  $j < k$  then
7:      $\mu_{k,j} \leftarrow \frac{r_{k,j}}{r_{j,j}}$ 
8:   end if
9: end for
10: return  $r, \mu$ 

```

The parameters η and δ have relatively little impact on the performance of SQIsign2D^2 , as long as they are taken close to $1/2$ and 1 respectively. Following standard practice [FPL23], they can be taken as $\eta = 0.51$, $\delta = 0.99$.

Regarding the implementation of floating-point operations, [NS09, Theorem 2] indicates that a 24-bit mantissa is generally adequate for all three security levels of SQIsign2D^2 . On the other hand, the exponent must be at least as large as the lattice volume, which is why we recommend employing an exponent of at least 20 bits in the implementation of floating-point operations.

Lattice enumeration and sampling Given a lattice Λ generated by a basis $(\mathbf{b}_0, \dots, \mathbf{b}_{d-1})$ of a positive definite quadratic space, two related tasks are enumerating all vectors of Λ of length less than a given bound and uniformly sampling from the same set. A common solution to both consists in determining a parallelogram containing all the vectors of interest and enumerating/sampling from the parallelogram, rejecting the vectors whose length surpasses the bound.

SQIsign2D^2 only needs to implement the sampling variant of the algorithm. For this, we adapt in [LatticeSampling](#) (Algorithm 2.14) an enumeration algorithm of Dieter [Die75].

Algorithm 2.12 $\text{SizeReduce}((\mathbf{b}_0, \dots, \mathbf{b}_{d-1}), \mathbf{G}, k, r :: \text{FLOAT}, \mu :: \text{FLOAT}, \bar{\eta} :: \text{FLOAT})$

Input: A basis $(\mathbf{b}_0, \dots, \mathbf{b}_{d-1})$ of a quadratic space, its $d \times d$ Gram matrix $\mathbf{G}$, index $1 \leq k < d$, the GSO family r, μ up to row $k - 1$, parameter $\frac{1}{2} < \bar{\eta} < 1$.

Output: The size-reduced basis $(\mathbf{b}_0, \dots, \mathbf{b}_{d-1})$ of the same lattice, its Gram matrix $\mathbf{G}$, the GSO family r, μ up to row k .

```

1: done  $\leftarrow$  false
2: while not done do
3:    $r, \mu \leftarrow \text{ExtendGSOFamily}(\mathbf{G}, k, r, \mu)$ 
4:   done  $\leftarrow$  true
5:   for  $i$  from  $k - 1$  down to 0 do
6:     if  $|\mu_{k,i}| > \bar{\eta}$  then
7:       done  $\leftarrow$  false
8:        $X \leftarrow \lfloor \mu_{k,i} \rfloor$  // Round to the closest integer
9:        $\mathbf{b}_k \leftarrow \mathbf{b}_k - X\mathbf{b}_i$  // Update basis
10:      for  $j$  from 0 up to  $d - 1$  do
11:         $\mathbf{G}_{k,j} \leftarrow \mathbf{G}_{k,j} - X\mathbf{G}_{i,j}$  // Update Gram matrix
12:      end for
13:      for  $j$  from 0 up to  $d - 1$  do
14:         $\mathbf{G}_{j,k} \leftarrow \mathbf{G}_{j,k} - X\mathbf{G}_{j,i}$  // Update Gram matrix
15:      end for
16:      for  $j$  from 0 up to  $i - 1$  do
17:         $\mu_{k,j} \leftarrow \mu_{k,j} - \text{FLOAT}(X)\mu_{i,j}$  // Update  $\mu$ 
18:      end for
19:    end if
20:  end for
21: end while
22: return  $(\mathbf{b}_0, \dots, \mathbf{b}_{d-1}), \mathbf{G}, r, \mu$ 

```

2.6.5. Close vector enumeration in 2-dimensional lattices

Suppose $L = \langle \beta_0, \beta_1 \rangle$ is a 2-dimension lattice over $\mathbb{Z}$ equipped with length $Q(x\beta_0 + y\beta_1) = x^2 + y^2$, where $\{\beta_0, \beta_1\}$ is a reduced basis and $Q(\beta_1) \leq Q(\beta_0)$. The corresponding bilinear form is denoted by $\langle \cdot, \cdot \rangle$. Algorithm [FullStrongApproximation](#) requires finding vectors in 2-dimension integral lattice L that are close to a given target vector v .

Finding the closest vector Given reduced basis β_0, β_1 of the lattice L , SQIsign2D^2 using Babai's nearest plane algorithm [[Bab86](#)] in [ClosestVector](#)(Algorithm 2.15) to find vectors close to the vector v . And there are two algorithms used in the implementation.

Enumerating close vectors Given $m \in \mathbb{N}$, $B \in \mathbb{R}$ and a vector u close to v , [EnumerateCloseVectors](#)(Algorithm 2.16) uses the Fincke-Pohst algorithm [[Coh13](#)] to enumerate short vectors c in the lattice L in order to find vectors of the form $u + c$ that are also close to v . It returns up to m such vectors whose distance to v is at most B .

Algorithm 2.13 InsertBefore($(\mathbf{b}_0, \dots, \mathbf{b}_{d-1}), \mathbf{G}, k, s, r :: \text{FLOAT}, \mu :: \text{FLOAT}$)

Input: A basis $(\mathbf{b}_0, \dots, \mathbf{b}_s, \dots, \mathbf{b}_k, \dots, \mathbf{b}_{d-1})$ of a quadratic space, its $d \times d$ Gram matrix $\mathbf{G}$, indices $0 \leq s < k < d$, the GSO family r, μ up to row k .

Output: The basis $(\mathbf{b}_0, \dots, \mathbf{b}_k, \mathbf{b}_s, \dots, \mathbf{b}_{d-1})$ where $\mathbf{b}_k$ has been inserted before $\mathbf{b}_s$, the associated Gram matrix, the GSO family r, μ up to row s .

```

1: for  $j$  from  $k$  down to  $s + 1$  do
2:   swap  $\mathbf{b}_j$  and  $\mathbf{b}_{j-1}$ 
3:   for  $i$  from 0 up to  $d - 1$  do
4:     swap  $\mathbf{G}_{i,j}$  and  $\mathbf{G}_{i,j-1}$ 
5:   end for
6:   for  $i$  from 0 up to  $d - 1$  do
7:     swap  $\mathbf{G}_{j,i}$  and  $\mathbf{G}_{j-1,i}$ 
8:   end for
9: end for
10:  $r_{s,s} \leftarrow \text{FLOAT}(\mathbf{G}_{s,s})$ 
11: for  $i$  from 0 up to  $s - 1$  do
12:    $\mu_{s,i} \leftarrow \mu_{k,i}$ 
13:    $r_{s,i} \leftarrow r_{k,i}$ 
14: end for
15:  $r_{s,s} \leftarrow r_{s,s} - \mu_{s,i} r_{s,i}$ 
16: return  $(\mathbf{b}_0, \dots, \mathbf{b}_{d-1}), \mathbf{G}, r, \mu$ 

```

Algorithm 2.14 LatticeSampling($(\mathbf{b}_0, \dots, \mathbf{b}_{d-1}), \mathbf{G}, B$)

Input: A basis $(\mathbf{b}_0, \dots, \mathbf{b}_{d-1})$ of a quadratic space, its $d \times d$ Gram matrix $\mathbf{G}$, an integer $B > 0$.

Output: A uniformly random vector of length $\leq B$ in the lattice generated by $(\mathbf{b}_0, \dots, \mathbf{b}_{d-1})$.

```

1: Initialize  $\mathbf{Id}$  to the  $d \times d$  identity matrix
2:  $\mathbf{U}, \mathbf{H} \leftarrow \text{L2}_{\eta, \delta}(\mathbf{Id}, \mathbf{G}^{-1})$  // Parse columns of  $\mathbf{U}$  and  $\mathbf{Id}$  as basis vectors
3: repeat
4:   for  $i$  from 0 up to  $d - 1$  do
5:      $B_i \leftarrow \lfloor \sqrt{B \mathbf{H}_{i,i}} \rfloor$ 
6:     Sample  $x_i$  uniformly from  $[-B_i, B_i]$ 
7:   end for
8:    $(y_0 \cdots y_{d-1}) \leftarrow (x_0 \cdots x_{d-1}) \mathbf{U}^{-1}$ 
9: until  $(y_0 \cdots y_{d-1}) \mathbf{G} (y_0 \cdots y_{d-1})^t \leq B$ 
10: return  $y_0 \mathbf{b}_0 + \cdots + y_{d-1} \mathbf{b}_{d-1}$ 

```

Algorithm 2.15 ClosestVector((β_0, β_1, q, v))

Input: q a positive integer, β_1, β_0 a reduced integral lattice basis such that $Q(\beta_1) < Q(\beta_0)$ and $Q(x\beta_0 + y\beta_1) = x^2 + qy^2$, v a vector with integer coefficients

Output: u vector in lattice generated by β_1, β_0 close to v for Q

```

1:  $\mu_1 := Q(\beta_1)\beta_0 - \langle \beta_0, \beta_1 \rangle \beta_1$ 
2:  $u := v - \left\lfloor \frac{\langle \mu_1, v \rangle Q(\beta_1)}{Q(\mu_1)} \right\rfloor \beta_0$ 
3:  $u := u - \left\lfloor \frac{\langle \beta_1, u \rangle}{Q(\beta_1)} \right\rfloor \beta_1$ 
4: return  $u$ 

```

Algorithm 2.16 EnumerateCloseVectors(L, q, v, u, m, B)

Input: q positive integer, $L = (b_0, b_1)$ a lattice, v target vector, $u \in L$ a vector close to v , m maximal number of tries, B norm bound

Output: A list of vectors $u' \in L$ close to v

```

1:  $i := 0$ 
2:  $w := v - u$ 
3:  $r, s, t := Q(b_0), 2\langle b_0, b_1 \rangle, Q(b_1)$ 
4: if  $4rt - s^2 \leq 0$  then
5:   Abort
6: end if
7:  $B_e := B$ 
8: if  $B - Q(w) > 0$  then
9:    $B_e := B - Q(w)$ 
10: end if
11:  $B_y := \left\lfloor \frac{\sqrt{4r^2 B_e + 1}}{\sqrt{4r^2 t - s^2}} \right\rfloor + 1$ 
12:  $y := -B_y - 1$ 
13: while ( $y < B_y$ ) and ( $i < m$ ) do
14:    $y := y + 1$ 
15:    $B_x := \left\lfloor \frac{2r(1 + \sqrt{4r^2 B_e + 4tr^2 y^2 - s^2 y^2}) - sy\sqrt{4r^3}}{2r\sqrt{4r^3}} \right\rfloor + 1$ 
16:    $x := - \left\lfloor \frac{2r(1 + \sqrt{4r^2 B_e + 4tr^2 y^2 - s^2 y^2}) + sy\sqrt{4r^3}}{2r\sqrt{4r^3}} \right\rfloor - 2$ 
17:   while ( $x < B_x$ ) and ( $i < m$ ) do
18:      $x := x + 1$ 
19:      $i := i + 1$ 
20:     if  $Q(w - xb_0 - yb_1) \leq B$  then
21:       return  $u + xb_0 + yb_1$ 
22:     end if
23:   end while
24: end while

```

2.7. Quaternions

This section reviews the necessary background on ideals and orders in quaternion algebras.

2.7.1. Quaternions algebra

The quaternion algebra used throughout this document is the algebra over $\mathbb{Q}$ ramified exactly at the prime p and at the infinite place, and we denote it by $B_{p,\infty}$. The parameter p is the same prime as in the finite field and elliptic curve setting, that is $p \equiv 3 \pmod{4}$, then we fix a presentation of $B_{p,\infty}$ with basis $1, i, j, k$ satisfying the relations

$$i^2 = -1, \quad j^2 = -p, \quad k = ij = -ji.$$

Hence, $B_{p,\infty}$ is a four-dimensional $\mathbb{Q}$ -algebra with non-commutative multiplication. For later reference, the multiplication table of the basis elements $1, i, j, k$ is given in Table 2.1.

	1	i	j	ij
1	1	i	j	ij
i	i	-1	ij	$-j$
j	j	$-ij$	$-p$	pi
ij	ij	j	$-pi$	$-p$

FIGURE 2.1. Multiplication table for $B_{p,\infty}$, the quaternion algebra ramified at a prime p and ∞ .

Hence any element of $B_{p,\infty}$ can be written uniquely as

$$\alpha = x_0 + x_1i + x_2j + x_3k, \quad x_0, x_1, x_2, x_3 \in \mathbb{Q}.$$

For implementation purposes, we represent such an element by integers $(a, b, c, d, r) \in \mathbb{Z}^5$, and the quaternion is

$$\alpha = \frac{a + bi + cj + dk}{r}.$$

When needed, this representation is normalized by dividing out any common factor in the numerator coefficients and denominator, so that the form is canonical.

The standard involution on $B_{p,\infty}$ is quaternion conjugation. For $\alpha = x_0 + x_1i + x_2j + x_3k$, its conjugate is $\bar{\alpha} = x_0 - x_1i - x_2j - x_3k$. Then the reduced trace and reduced norm of α are defined by $\text{Trd}(\alpha) = \alpha + \bar{\alpha}$, $\text{Nrd}(\alpha) = \alpha\bar{\alpha} = \bar{\alpha}\alpha$. With the above presentation, one obtains $\text{Trd}(\alpha) = \frac{2a}{r}$ and $\text{Nrd}(\alpha) = \frac{a^2 + b^2 + pc^2 + pd^2}{r^2}$. Moreover, we define a symmetric bilinear form by

$$\langle \alpha, \beta \rangle = \text{Nrd}(\alpha + \beta) - \text{Nrd}(\alpha) - \text{Nrd}(\beta) = \text{Trd}(\alpha\bar{\beta}), \quad (2.11)$$

and the bilinear form is positive-definite.

2.7.2. Lattices in quaternion algebras

In `SQIsign2D2`, we consider the quaternion elements $\alpha \in B_{p,\infty}$, as well as the full-rank lattices in $B_{p,\infty}$ generated by four elements over $\mathbb{Z}$. More precisely, such a lattice has the form $\mathbb{Z}\alpha_1 + \mathbb{Z}\alpha_2 + \mathbb{Z}\alpha_3 + \mathbb{Z}\alpha_4$, where $\{\alpha_1, \dots, \alpha_4\}$ is a $\mathbb{Q}$ -basis of $B_{p,\infty}$. We next specify the concrete representation of these objects and the basic operations on them.

Quaternion lattices. A lattice is determined by a basis $\{\alpha_1, \alpha_2, \alpha_3, \alpha_4\}$ consisting of $\mathbb{Q}$ -linearly independent quaternions. Throughout this specification, we adopt the convention that elements are represented as **columns** of a matrix L , so $(\alpha_1, \alpha_2, \alpha_3, \alpha_4) = (1, i, j, k)L$ is the row vector describing the basis.²

The dual lattice of Λ is defined by

$$\Lambda^* = \{f \in B_{p,\infty}^* \mid \forall x \in \Lambda, f(x) \in \mathbb{Z}\} \cup \{0\},$$

where $B_{p,\infty}^*$ denotes the space of nonzero linear maps from $B_{p,\infty}$ to $\mathbb{Q}$. Let $(1^*, i^*, j^*, k^*)$ denote the basis of the dual lattice $(\mathbb{Z} + \mathbb{Z}i + \mathbb{Z}j + \mathbb{Z}k)^*$. If Λ is generated by the basis $(1, i, j, k) \cdot L$, then its dual lattice Λ^* is generated by

$$L^{-1} \cdot (1^*, i^*, j^*, k^*)^t.$$

In the following, we slightly abuse notation by identifying the columns (respectively, rows) of a matrix defining a lattice basis with the lattice itself (respectively, its dual lattice). The basic operations are then computed as follows.

- **Equality:** Verify that L_1 and L_2 have the same HNF [Coh13, §2.4.3].
- **Sum:** For lattices L_1 and L_2 , concatenate their matrices L_1, L_2 and compute its HNF to obtain $L_1 + L_2$.
- **Intersection:** For lattices L_1 and L_2 , first compute the dual lattices L_1^* and L_2^* . Then $L_1 \cap L_2$ is the dual of $L_1^* + L_2^*$.
- **Multiplication:** For lattices L_1 and L_2 , the product $L_1 L_2$ can be computed, for example, by taking the right multiplication matrices $A_1, \dots, A_4$ associated with a basis $\alpha_1, \dots, \alpha_4$ of L_2 , and then computing the sum

$$A_1 L_1 + A_2 L_1 + A_3 L_1 + A_4 L_1.$$

- **Containment:** Given an element $\alpha \in B_{p,\infty}$ and a lattice L , check whether $\alpha \in L$ is a solution of the linear system $LX = \alpha$ and check whether the entries of X are integers.
- **Inclusion:** To determine whether a lattice L_1 is contained in a lattice L_2 , one may either test whether all four basis vectors of L_1 lie in L_2 , or check whether $L_1 + L_2$ equals L_2 .
- **Index:** When $L_1 \subset L_2$, the index of L_1 in L_2 , denoted $[L_2 : L_1]$, is the order of the finite quotient group L_2/L_1 . The value is equal to $|\det(L_1)/\det(L_2)|$, which can be computed using the dimension-4 determinant algorithm from [Ebe07].
- **Basis reduction:** A lattice is said to be **reduced** when its basis is reduced in the sense of Section 2.6.4, instantiated with the bilinear form of Equation 2.11. We use the L2 algorithm to compute a reduced basis. The Gram matrix of the basis $(1, i, j, ij)$ is the diagonal matrix $\mathbf{G}$ with entries $(2, 2, 2p, 2p)$, and for any other basis the corresponding Gram matrix is

$$\mathbf{G}' = \mathbf{M}^t \mathbf{G} \mathbf{M},$$

where $\mathbf{M}$ is the change-of-basis matrix.

²In practice, the reference implementation takes common denominators and represents L as an integer matrix $\mathbf{M}$ and a common denominator r , so that $L = \mathbf{M}/r$. This can be considered as an implementation detail.

2.7.3. Quaternion orders and ideals

An **order** is a lattice of $B_{p,\infty}$ that is also a subring. Elements of an order $\mathcal{O}$ are said to be **integral**, since they have reduced norm and trace in $\mathbb{Z}$. An order is called **maximal** if it is not contained in any other larger order.

The quaternion algebra used in SQIsign2D^2 contains a maximal order with basis $\{1, i, \frac{i+j}{2}, \frac{1+ij}{2}\}$ which will be denoted by $\mathcal{O}_0$ in the remainder of this section. Moreover, the maximal order is isomorphic to the endomorphism ring of supersingular elliptic curve $E_0 : y^2 = x^3 + x$. It is easy to see that the maximal order $\mathcal{O}_0$ contains a non-maximal suborder of basis $\{1, i, j, ij\}$.

Let $\mathcal{O}$ be an order in $B_{p,\infty}$. A **left (right) integral ideal** of $\mathcal{O}$, denoted by I , is a sublattice of $\mathcal{O}$ and satisfied $\mathcal{O}I \subseteq I$ ($I\mathcal{O} \subseteq I$). The **left order** of an ideal I is defined as $\mathcal{O}_L(I) = \{\alpha \in B_{p,\infty} \mid \alpha I \subseteq I\}$ and similarly for the right order $\mathcal{O}_R(I)$. In this case, I is clearly a left ideal of $\mathcal{O}_L(I)$. Furthermore, if I is a left ideal of maximal order $\mathcal{O}$, then $\mathcal{O}_L(I) = \mathcal{O}$ and $\mathcal{O}_R(I)$ is also a maximal order. An ideal with left order $\mathcal{O}_L$ and right order $\mathcal{O}_R$ is called a **connecting ideal** of $\mathcal{O}_L$ and $\mathcal{O}_R$.

In SQIsign2D^2 , we always consider the maximal order $\mathcal{O}$ and the left integral $\mathcal{O}$ -ideal. The reduced norm of I , denoted by $\text{Nrd}(I)$, is defined as $\text{Nrd}(I) = \gcd\{\text{Nrd}(\alpha) : \alpha \in I\}$. Moreover, we have $\text{Nrd}(I) = \sqrt{[\mathcal{O}_L(I) : I]} = \sqrt{[\mathcal{O}_R(I) : I]}$. The left integral ideal I can be generated by two elements $\alpha, \text{Nrd}(I)$, where $\alpha \in I$ for some α . So we write $I = \langle \alpha, \text{Nrd}(I) \rangle$. Similarly to the case I is a right integral ideal of $\mathcal{O}_R(I)$. If we want to compute a left integral $\mathcal{O}$ -ideal of reduced norm N , we can find an element $\alpha \in \mathcal{O}$ such that $N \mid \text{Nrd}(\alpha)$ and $\gcd(N, \text{Nrd}(\alpha)/N) = 1$. The conjugation of I is defined as $\bar{I} = \{\bar{\alpha} : \alpha \in I\}$ and we have $\bar{I}$ is a left integral $\mathcal{O}_R(I)$ -ideal, $I\bar{I} = \text{Nrd}(I)\mathcal{O}_L(I)$, $\bar{I}I = \text{Nrd}(I)\mathcal{O}_R(I)$. So we define the inverse of I to be $\frac{\bar{I}}{\text{Nrd}(I)}$.

We say two orders $\mathcal{O}_1, \mathcal{O}_2$ are equivalent if there exists $\beta \in B_{p,\infty}^*$ such that $\beta\mathcal{O}_1\beta^{-1} = \mathcal{O}_2$, two left integral $\mathcal{O}$ -ideals I_1, I_2 are equivalent if there exists $\beta \in B_{p,\infty}^*$ such that $I_2 = I_1\beta$. In this case, $\mathcal{O}_R(I_2) = \beta^{-1}\mathcal{O}_R(I_1)\beta$, and $\mathcal{O}_R(I_2)$ equivalent to $\mathcal{O}_R(I_1)$.

Now we will give some basic operations on ideals as follows.

Ideal from generators: For the generators $\alpha \in I$, $\text{Nrd}(I)$, we compute $\mathcal{O}\alpha$, $\mathcal{O}\text{Nrd}(I)$, then lattice sum computes $I = \mathcal{O}\alpha + \mathcal{O}\text{Nrd}(I)$.

Generators from ideal: Compute an element $\alpha \in I$ such that $\gcd(\text{Nrd}(I), \text{Nrd}(\alpha)/\text{Nrd}(I)) = 1$, and then $I = \langle \alpha, \text{Nrd}(I) \rangle$.

Compute the connecting ideal: For two maximal orders $\mathcal{O}_1, \mathcal{O}_2$, compute connecting ideal $I = N\mathcal{O}_1\mathcal{O}_2$, where N is $\sqrt{[\mathcal{O}_1 : \mathcal{O}_1 \cap \mathcal{O}_2]}$.

Inverse ideal: Compute the rank 4-lattice $\frac{\bar{I}}{\text{Nrd}(I)}$, which is the inverse of I .

Left (Right) order: The left or right order can be obtained from the inverse ideal, for $\mathcal{O}_L(I) = I\bar{I}^{-1}$ and $\mathcal{O}_R(I) = \bar{I}^{-1}I$.

Pullback and pushforward of ideal: For two left $\mathcal{O}$ -ideals I, J of the coprime norm, the pushforward of I from J , denoted by $[J]_*I$, is defined by $J^{-1}(J \cap I)$. For right $\mathcal{O}$ -ideal K , the pullback of I from K , denoted by $[K]^*I$, is defined by $[\bar{K}]_*I$. The idea of pullback and pushforward is from that of isogenies, which is used to construct a commutative diagram.

Finding equivalent ideal: In SQIsign2D^2 , the response isogeny is obtained from the response ideal, which is a random equivalent ideal of the composition of the dual of I_τ , I_{com} and I_{chl} . This means we need to compute a random equivalent ideal. Since every left integral $\mathcal{O}$ -ideal

equivalent to I has the form $I \frac{\bar{\alpha}}{\text{Nrd}(I)}$ for some $\alpha \in I$, we only compute a random element in I . The algorithm samples constants c_i and constructs $\alpha = \sum_{i=1}^4 c_i \alpha_i$, where $\{\alpha_1, \dots, \alpha_4\}$ is a reduced basis of I , and obtains $J = I \frac{\bar{\alpha}}{\text{Nrd}(I)}$. Heuristically, we can expect the norm of the output ideal to be $\approx \sqrt{p}$ which should be much smaller than the norm of response ideal in SQIsign.

Algorithm 2.17 IdealGenerator(I)

Input: I , an ideal.

Output: γ primitive in I such that its norm N_I and γ generate I as ideal of its left or right order.

```

1:  $n \leftarrow 0$ 
2: while true do
3:    $n \leftarrow n + 1$ 
4:   for  $a$  from  $-n$  up to  $n$  do
5:     for  $b$  from  $-n + |a|$  up to  $n - |a|$  do
6:       for  $c$  from  $-n + |a| + |b|$  up to  $n - |a| - |b|$  do
7:          $d \leftarrow n - |a| - |b| - |c|$ 
8:         if  $\text{gcd}(a, b, c, d) = 1$  then
9:            $\gamma \leftarrow$  Quaternion of coordinates  $a, b, c, d$  in the basis in HNF of  $I$ 
10:           $q \leftarrow \text{Nrd}(\gamma)/N_I$ 
11:          if  $\text{gcd}(q, N_I) = 1$  then
12:            return  $\gamma$ 
13:          end if
14:        end if
15:      end for
16:    end for
17:  end for
18: end while

```

Algorithm 2.18 RandomEquivalentQuaternion(L)

Input: A ideal lattice L .

Output: A uniformly sampled $\mathbf{b} \in L$ such that $\text{Nrd}(\mathbf{b})/\text{Nrd}(L) < 2^a$.

```

1: Let  $(\mathbf{b}_0, \mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3)$  be a basis of  $L$ 
2: Compute the Gram matrix  $\mathbf{G}$  of  $(\mathbf{b}_0, \mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3)$ 
3:  $\mathbf{b} \leftarrow \text{LatticeSampling}((\mathbf{b}_0, \mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3), \mathbf{G}, \text{Nrd}(I) \cdot 2^{a+1})$ 
4: if  $\mathbf{b}/3 \in \mathcal{O}_L(L)$  then
5:   repeat
6:      $\mathbf{b} \leftarrow \mathbf{b}/3$ 
7:   until  $\mathbf{b}/3 \notin \mathcal{O}_L(L)$ 
8: end if
9: return  $\mathbf{b}$ 

```

2.7.4. Solving norm equations

An important part of both the key generation and signing procedure in SQIsign2D², is the ability to solve norm equations in special quaternion orders, sometimes with some additional constraints. In original SQIsign, they consider the p -extremal maximal orders. Throughout this section, for our SQIsign2D², we only consider the special maximal order $\mathcal{O}_0 = \mathbb{Z} + \mathbb{Z}i + \mathbb{Z}\frac{i+j}{2} + \mathbb{Z}\frac{1+k}{2}$. The maximal order $\mathcal{O}_0$ contains j with $j^2 = -p$ and a distinguished quadratic subring $R = \mathbb{Z}[i]$ with $i^2 = -1$. Moreover, the two R and jR are orthogonal.

Cornacchia's algorithm. Cornacchia's algorithm [Cor07] allows us to efficiently find integer solutions for equations of the form $x^2 + y^2 = m$ with m a positive integer, under the factorization of m . For prime m , a pseudocode following [MN90] is given in Cornacchia(Algorithm 2.19).

For arbitrary m , factorize m into a product of some small primes and a larger integer m' , if m' is prime, then use Cornacchia's algorithm to solve the equation; see [extend_Cornacchia](#). The list `Prime_list` consists of small primes that are congruent to 1 modulo 4, and is fixed to be $[2, 5, 13, 17, 29, 37, 41, 53, 61, 73, 89, 97]$ in the implementation.

Algorithm 2.19 `Cornacchia( $m$ )`

Input: $m \in \mathbb{N}$ with m prime.

Output: $x, y \in \mathbb{Z}$ such that $x^2 + y^2 = m$ if such x, y exist, non-existence indicator $\perp$ otherwise.

```

1: if  $-1$  is not a square modulo  $m$  then
2:   return  $\perp$ 
3: end if
4: if  $m = 2$  then
5:   return  $1, 1$ 
6: end if // Now  $m$  is an odd prime and  $-1$  square modulo  $m$ 
7:  $r \leftarrow \text{ModularSqrt}(-1, m)$ 
8:  $s \leftarrow 1$ 
9:  $r, s \leftarrow s, (r \bmod s)$ 
10:  $r, s \leftarrow s, (r \bmod s)$ 
11: while  $s^2 < m \leq r^2$  do
12:    $r, s \leftarrow s, (r \bmod s)$ 
13: end while
14:  $x, y \leftarrow s, m - s^2$ 
15: if  $x^2 + y^2 = m$  then
16:   return  $r, s$ 
17: else
18:   return  $\perp$ 
19: end if
```

Representing integers by $\mathcal{O}_0$. Cornacchia's algorithm efficiently solves norm equations of the form $x^2 + y^2 = m$. This makes it straightforward to solve norm equations in $\mathcal{O}_0$. The norm of an element of coefficients (x, y, z, t) in basis $1, i, j, k$ is then

$$F(x, y, z, t) = x^2 + y^2 + p(z^2 + t^2).$$

We follow the method introduced in [DLL+23][§6]. The general idea for solving equations of the form $F(x, y, z, t) = M$ for some target norm M is to sample $z, t \in \{-\lfloor \sqrt{4M/p} \rfloor, \dots, \lfloor \sqrt{4M/p} \rfloor\}$ such that $p(z^2 + t^2) < 4M$, and then try to solve for x, y such that $x^2 + y^2 = 4M - p(z^2 + t^2)$ using [extend_Cornacchia](#)(Algorithm 2.20). If we obtain the solution (x, y, z, t) , we need to check $x \equiv z \pmod{2}$ and $y \equiv t \pmod{2}$. The constant `QUAT_repres_num_trial` is set such that the algorithm almost never fails. See Chapter 8 for more details on heuristics and failures.

Algorithm 2.20 `extend_Cornacchia( $m$ )`

Input: $m \in \mathbb{N}$.**Output:** $x, y \in \mathbb{Z}$ such that $x^2 + y^2 = m$ if such x, y exist, non-existence indicator $\perp$ otherwise.

```

1: Factor  $m$  as  $m = d \cdot m'$ , where the prime factorization of  $d$  only consists of primes in Prime_list, and  $m'$ 
   is not divisible by any prime in Prime_list.
2: if  $m$  is not prime or  $-1$  is not a square modulo  $m$  then
3:   return  $\perp$ 
4: end if
5:  $\text{res} \leftarrow \text{Cornacchia}(m')$ 
6: if  $\text{res} = \perp$  then
7:   return  $\perp$ 
8: end if
9:  $x, y \leftarrow \text{res}$ 
10: for  $\ell$  in Prime_list do
11:   Let  $e$  be the largest integer such that  $\ell^e \mid d$ .
12:   if  $e > 0$  then
13:      $s, t \leftarrow \text{Cornacchia}(\ell)$ 
14:     for  $j$  from 0 up to  $e - 1$  do
15:        $x, y \leftarrow xs - yt, xt + ys$ 
16:     end for
17:   end if
18: end for
19: return  $x, y$ 

```

Algorithm 2.21 `FullRepresentInteger $\mathcal{O}_0(M)$`

Input: $M \in \mathbb{Z}$ such that $M > p$.**Output:** A primitive $\gamma \in \mathcal{O}_0$ with $\text{Nrd}(\gamma) \mid M$, or raises an exception.**Output:** $d \in \mathbb{N}$ such that $\text{Nrd}(d\gamma) = M$.

```

1: Initialize counter  $\leftarrow 0$ , found  $\leftarrow \text{false}$ 
2: while (not found) and (counter < QUAT_repres_num_trial) do
3:   counter  $\leftarrow$  counter + 1
4:   Sample  $z$  uniformly from  $[1, \dots, m]$  for  $m = \lfloor \sqrt{\frac{4M}{p}} \rfloor$ 
5:   Sample  $t$  uniformly from  $[-m', \dots, m']$  for  $m' = \lfloor \sqrt{\frac{4M - pz^2}{p}} \rfloor$ 
6:   Set  $M' \leftarrow 4M - p(z^2 + qt^2)$ 
7:    $\text{res} \leftarrow \text{extend\_Cornacchia}(M')$ 
8:   found  $\leftarrow (\text{res} \neq \perp)$ 
9:   if found then
10:     $x, y \leftarrow \text{res}$ 
11:    if  $x \not\equiv t \pmod{2}$  then
12:      Swap  $x$  and  $y$ 
13:    end if
14:    found  $\leftarrow (x - t \equiv 0 \pmod{2})$  and  $(y - z \equiv 0 \pmod{2})$ 
15:    if found then
16:       $\gamma \leftarrow (x + yi + zj + tk)$ 
17:      Set  $d$  to be the biggest scalar such that  $\gamma/d \in \mathcal{O}_0$ 
18:      return  $\gamma/d, d/2$ 
19:    end if
20:  end if
21: end while
22: raise Exception: ("FullRepresentInteger failed")

```

Computing an element in Eichler order with fixed reduced norm. For two maximal orders $\mathcal{O}_0$ and $\mathcal{O}_1$, the connecting ideal of $\mathcal{O}_0, \mathcal{O}_1$ is I , that is $\mathcal{O}_L(I) = \mathcal{O}_0$, $\mathcal{O}_R(I) = \mathcal{O}_1$, then we define the Eichler order of $\mathcal{O}_0$ and $\mathcal{O}_1$ to be $\mathcal{O}_0 \cap \mathcal{O}_1 = \mathbb{Z} + I$. In SQIsign2D^2 , we need to compute an element in Eichler order $\mathcal{O}_0 \cap \mathcal{O}_1$ of fixed reduced norm, where I is the connecting ideal of $\mathcal{O}_0$ and $\mathcal{O}_1$ and $\text{Nrd}(I) = N$. The method is from original SQIsign [DKL+20], that is, find $(C : D) \in \mathbb{P}^1(\mathbb{Z}/N\mathbb{Z})$ such that $Cj + Dk \in \mathbb{Z} + I$, and use strong approximation to compute an element in $\mathcal{O}_0 \cap \mathcal{O}_1$ of target norm.

Computing $(C : D)$ by linear system. The idea of computing $(C : D)$ involves solving some linear system of equations. The general algorithm for this step is given in [EichlerModConstraint](#)(Algorithm 2.22). This algorithm uses linear algebra in $\mathbb{Z}/N\mathbb{Z}$ to find a quaternion of a special form, satisfying some constraint.

Algorithm 2.22 [EichlerModConstraint](#)($I, \gamma, \delta, \text{Divisible}$)

Input: A left $\mathcal{O}$ -ideal I of norm N .

Input: Elements $\gamma, \delta \in \mathcal{B}_{p,\infty}$.

Input: A boolean Divisible indicating whether $N \mid \text{Nrd}(\gamma)$.

Output: Integers $C, D \in \mathbb{Z}$ such that $\gamma j(C + iD)\delta \in \mathbb{Z} + I$.

```

1: Let  $d_1$  be the denominator of  $\gamma j i \delta$ 
2: Let  $d_2$  be the denominator of  $I$ 
3: Let  $d_3$  be the denominator of  $\gamma j \delta$ 
4: Let  $\beta_1, \beta_2, \beta_3, \beta_4$  denote a basis of  $I$  in Hermite normal form
5: if  $\text{Divisible}$  then
6:    $\mathbf{M} \leftarrow [\gamma j \delta d_1 d_2, \gamma j i \delta d_2 d_3, \beta_2 d_1 d_3, \beta_3 d_1 d_3] \in \mathbb{M}_{4 \times 4}(\mathbb{Z}/N\mathbb{Z})$ 
7:   Compute  $[C, D, \_, \_]^\top \in \ker \mathbf{M}$ 
8: else
9:    $\mathbf{M} \leftarrow [\gamma j \delta d_1 d_2, \gamma j i \delta d_2 d_3, d_1 d_2 d_3, \beta_2 d_1 d_3, \beta_3 d_1 d_3] \in \mathbb{M}_{4 \times 5}(\mathbb{Z}/N\mathbb{Z})$ 
10:  Compute  $[C, D, \_, \_, \_]^\top \in \ker \mathbf{M}$ 
11: end if
12: return  $C, D$ 
```

Strong approximation: The strong approximation is to go from an element $\mu_0 := j(C + iD)$ satisfying $\gamma \mu_0 \delta \in L$ where L is the (multiplicative) lattice in which we want to solve the norm equation, to an element $\mu := \lambda \mu_0 + N \mu_1$, where $\lambda \in \mathbb{Z}$, $\mu_1 \in \mathcal{O}_0$, which automatically satisfies the same constraint and also solves the norm equation. The original idea is to look for $\mu_1 := x_1 + y_1 i + j(z_1 + t_1 i) \in R + jR$, solving for the particularly simple norm form

$$\text{Nrd}(\mu) := N^2(x_1^2 + y_1^2) + p(C + Nz_1)^2 + p(D + Nt_1)^2,$$

by first solving it modulo N . [FullStrongApproximation](#)(Algorithm 2.23) shows how to do this, including an improvement by [PS18] for finding shorter solutions, and by [DLL+23] for finding μ_1 in the whole order.

It should be noted that we set $\gamma = \delta = 1$ in SQIsign2D^2 , so [EichlerModConstraint](#)(Algorithm 2.22) and [FullStrongApproximation](#)(Algorithm 2.23) are simpler than these in original SQIsign [DKL+20].

2.8. Relation between isogenies and ideals

The following procedures which are the conversion between ideals and isogenies is the main computational bottleneck in signing.

Algorithm 2.23 FullStrongApproximation $_{N_\mu}(\mathcal{O}_0, N, C, D, \lambda, \text{max_tries})$ **Input:** $\mathcal{O}_0$, N a prime, an odd integer N_μ , $C, D \in \mathbb{Z}$, λ satisfying $\lambda^2 = N_\mu/p(C^2 + D^2) \pmod{N}$ **Input:** max_tries a number bounding the number of tries**Output:** a primitive $\mu = \lambda j(C + Di) + N\mu_1$ with $\mu_1 \in \mathcal{O}_0$, such that $\text{Nrd}(\mu) \mid N_\mu$.**Output:** $d \in \mathbb{N}$ such that $\text{Nrd}(d\mu) = N_\mu$.

```

1:  $c_0 := 2p\lambda C$ 
2:  $c_1 := (2p\lambda D)^{-1} \pmod{N}$ 
3:  $c_2 := (4N_\mu - \lambda^2 p(C^2 + D^2))/N$ 
4:  $c_3 := -c_0 c_1 \pmod{N}$ 
5:  $\mathbf{L} := \begin{pmatrix} N & c_3 N \\ 0 & N^2 \end{pmatrix} \in M_2(\mathbb{Z})$ 
6:  $\mathbf{v} := [\lambda C, \lambda D + N c_1 c_2]$ 
7:  $\mathbf{v}' := \text{ClosestVector}(\mathbf{L}, 1, \mathbf{v})$ 
8:  $B := \min(\lceil \log_2(4N_\mu/p) \rceil, 3\lceil \log_2(N) \rceil + 10)$ 
9: for  $\mu$  in EnumerateCloseVectors( $\mathbf{L}, 1, \mathbf{v}, \mathbf{v}', \text{max\_tries}, B$ ) do
10:    $d = 1$ 
11:   if  $\mu/3$  is in  $\mathcal{O}_0$  then
12:      $\mu \leftarrow \mu/3, d \leftarrow 3d$ 
13:   Continue
14:   else
15:     return  $\mu/2, \text{TRUE}$ 
16:   end if
17: end for
18: return  $\_, \text{FALSE}$ 

```

Deuring correspondence. In this document, we only consider supersingular elliptic curves over $\mathbb{F}_{p^2}$. For such a curve E , the ring $\text{End}(E)$ is isomorphic to a maximal order $\mathcal{O}$ in the quaternion algebra $B_{p,\infty}$. Once an isomorphism $\mathcal{O} \cong \text{End}(E)$ has been fixed, we freely identify elements of $\mathcal{O}$ with endomorphisms of E . In particular, for $\alpha \in \mathcal{O}$ and $P \in E$, we write $\alpha(P)$ for the action of the corresponding endomorphism on P , and we write $\ker(\alpha)$ for its kernel.

With this notion, left $\mathcal{O}$ -ideals of norm coprime to p correspond to finite subgroup schemes of E , equivalently to kernels of separable isogenies from E . For a left ideal $I \subseteq \mathcal{O}$, we write

$$E[I] := \{P \in E \mid \alpha(P) = 0_E \text{ for all } \alpha \in I\}.$$

This is the subgroup corresponds to I . When I is represented as $I = \mathcal{O}\langle\alpha, N\rangle$, the subgroup is $E[I] = \ker(\alpha) \cap E[N]$. Given such an ideal I , we denote by φ_I the separable isogeny with kernel $E[I]$.

Conversely, if φ is a separable isogeny from E , we write I_φ for the corresponding ideal, namely

$$I_\varphi = \{\alpha \in \mathcal{O} \mid \alpha(P) = 0_E \text{ for all } P \in \ker(\varphi)\}.$$

All constructions in SQIsign2D² use a prime p satisfying $p \equiv 3 \pmod{4}$. Under this assumption, we fix the base curve $E_0 : y^2 = x^3 + x$, which is supersingular over $\mathbb{F}_{p^2}$. Its endomorphism ring is identified with the maximal order $\mathcal{O}_0 = \mathbb{Z} \oplus \mathbb{Z}i \oplus \mathbb{Z}\frac{i+j}{2} \oplus \mathbb{Z}\frac{1+k}{2}$.

The fixed identification $\text{End}(E_0) \cong \mathcal{O}_0$ is specified by sending j to the Frobenius endomorphism

$$(x, y) \mapsto (x^p, y^p),$$

and i to the automorphism

$$(x, y) \mapsto (-x, \sqrt{-1}y)$$

of E_0 . This choice is used throughout the implementation. Unless stated otherwise, E_0 and $\mathcal{O}_0$ always refer to the objects above, while $\mathcal{O}$ denotes an arbitrary maximal order.

Whenever an isogeny $\varphi : E_0 \rightarrow E$ and the corresponding $(\mathcal{O}_0, \mathcal{O})$ -ideal I are known, then φ corresponds to I and $\mathcal{O}_R(I) \cong \text{End}(E)$. Moreover, if I is a principal ideal, then φ is an endomorphism; if I equivalent to J , then the isogenies induced from I and J have the same image curve. This is the Deuring's correspondence, which will be used in `SQIsign2D2`.

The action of endomorphism on torsion points. In the following algorithms, we will use a prime p of the form $f \cdot 2^a \cdot 3^b - 1$, where f is a small cofactor. Let (P_0, Q_0) be a basis of $E_0[2^a]$ and (R_0, S_0) be a basis of $E_0[3^b]$ where the actions of $i, \frac{i+j}{2}, \frac{1+k}{2}$ are precomputed.

Consider $\mathcal{O}_0$, define $(M_1, \dots, M_4)$ as below:

- denote $\mathcal{O}_0 = \mathbb{Z} + \mathbb{Z}i + \mathbb{Z}\frac{i+j}{2} + \mathbb{Z}\frac{1+k}{2} = \mathbb{Z}b_1 \oplus \mathbb{Z}b_2 \oplus \mathbb{Z}b_3 \oplus \mathbb{Z}b_4$.
- $\mathbf{M}_u$ is the matrix in $M_2(\mathbb{Z}/(2^a \cdot 3^b)\mathbb{Z})$ representing the action of b_u on $E_0[2^a]$ and $E_0[3^b]$ with respect to the basis (P_0, Q_0) and (R_0, S_0) respectively for $u \in \{1, \dots, 4\}$, i.e.

$$b_u(P_0, Q_0) = (P_0, Q_0) \cdot M_u,$$

$$b_u(R_0, S_0) = (R_0, S_0) \cdot M_u.$$

For $\alpha = x_1b_1 + x_2b_2 + x_3b_3 + x_4b_4 \in \mathcal{O}_0$, we can compute the action of α on $E_0[2^a]$ and $E_0[3^b]$ by computing the matrix $\mathbf{M}_\alpha = x_1\mathbf{M}_1 + x_2\mathbf{M}_2 + x_3\mathbf{M}_3 + x_4\mathbf{M}_4$, then

$$\alpha(P_0, Q_0) = (P_0, Q_0) \cdot M_\alpha,$$

$$\alpha(R_0, S_0) = (R_0, S_0) \cdot M_\alpha.$$

We precompute $(\mathcal{O}_0, E_0, (P_0, Q_0), (R_0, S_0))$ and $(\mathbf{M}_1, \dots, \mathbf{M}_4)$, and use them in `SQIsign2D2`.

Converting isogenies to ideals. Translating isogenies to their corresponding ideals is comparatively more straightforward. To find the ideal corresponding to the isogeny generated by a point P of order D is to create a basis $\langle P, \theta(P) \rangle = E[D]$ for some endomorphism θ , and decomposing $\eta(P)$ along this basis, where η is orthogonal to θ , that is, solving

$$[m]P + [n]\theta(P) = \eta(P),$$

which means that $\alpha = m + n\theta - \eta$ sends the point P to the identity, and hence $I = \mathcal{O}\langle\alpha, D\rangle$ is the corresponding ideal.

In `SQIsign2D2`, we only translate points on E_0 of order 3^b to their corresponding ideals, hence we fix the choice $\theta = i$ and $\eta = j$. Further, instead of taking as input a kernel point R , `KernelDecomposedToIdeal`(Algorithm 2.24) takes in the decomposition of R along the fixed precomputed basis (R_0, S_0) , that is, $R = [m]R_0 + [n]S_0$, where $m, n \in \mathbb{Z}/3^b\mathbb{Z}$. By having again precomputed the action of θ and η on this basis, we obtain the required ideal without requiring any discrete log computations.

Algorithm 2.24 `KernelDecomposedToIdeal3b( $c_1, c_2$ )`

Input: $c_1, c_2 \in \mathbb{Z}/3^b\mathbb{Z}$ defining a point $[c_1]R_0 + [c_2]S_0$ on E_0 of order 3^b generating an isogeny φ .

Output: A left $\mathcal{O}_0$ -ideal I_φ .

- 1: $[d_1, d_2]^T \leftarrow \mathbf{M}_{2,i}[c_1, c_2]^T$
 - 2: $\mathbf{M} \leftarrow \begin{pmatrix} c_1 & d_1 \\ c_2 & d_2 \end{pmatrix}$
 - 3: $[m, n]^T \leftarrow \mathbf{M}^{-1}\mathbf{M}_{2,j}[c_1, c_2]^T$
 - 4: $I \leftarrow \mathcal{O}_0 \langle m + ni - j, 3^b \rangle$
 - 5: **return** I
-

2.9. Representation of isogenies between elliptic curves via (2, 2)-isogenies

Kani's lemma [Kan97] provides the method for computing 2-dimensional isogeny from image torsion points. This technique is employed in SQIsign2D² to generate secret key, compute auxiliary isogeny and verify. We first introduce Kani's lemma and the representation of isogenies between elliptic curves via 2-dimensional isogenies.

Let d_1, d_2, e be positive integers such that $\gcd(d_1, d_2) = 1$ and $d_1 + d_2 = 2^e$. Consider the following commutative diagram of isogenies between elliptic curves:

$$\begin{array}{ccc} E_1 & \xrightarrow{\varphi_1} & E_2 \\ \downarrow \varphi_2 & \searrow \psi & \downarrow \varphi'_2 \\ E_3 & \xrightarrow{\varphi'_1} & E_4 \end{array}$$

where $\deg(\varphi_1) = \deg(\varphi'_1) = d_1$ and $\deg(\varphi_2) = \deg(\varphi'_2) = d_2$, $\psi = \varphi'_2 \circ \varphi_1 = \varphi'_1 \circ \varphi_2$. Then the isogeny $\Phi : E_1 \times E_4 \rightarrow E_2 \times E_3$ defined by

$$\Phi = \begin{pmatrix} \varphi_1 & \hat{\varphi}'_2 \\ -\varphi_2 & \hat{\varphi}'_1 \end{pmatrix}$$

is a $(2^e, 2^e)$ -isogeny with kernel $\{([d_1]P, \psi(P)) \mid P \in E_1[2^e]\}$.

Given E_1, E_4, d_1, d_2, e , and the restriction of ψ on $E_1[2^e]$, we can compute an isogeny $\iota \circ \Phi$ for an isomorphism ι between products of elliptic curves. Suppose that $E_2 \neq E_3$, thus $e \geq 2$. Then, for $P \in E_1$, the first component of $\iota \circ \Phi(P, 0_{E_4})$ is $\varphi_1(P)$ or $\varphi_2(P)$ (up to post-composition with an isomorphism). We can determine which one is the case by computing the Weil pairing as follows. Let (P, Q) be a basis of $E_1[2^{e'}]$ for $e' \geq 2$ and $(P_1, P_2) = \iota \circ \Phi(P, 0_{E_4})$, $(Q_1, Q_2) = \iota \circ \Phi(Q, 0_{E_4})$. Then, we have $e_{2^{e'}}(P_1, Q_1) = e_{2^{e'}}(P, Q)^{d_1}$ if $P_1 = \varphi_1(P)$, and $e_{2^{e'}}(P_1, Q_1) = e_{2^{e'}}(P, Q)^{d_2}$ if $P_1 = \varphi_2(P)$. Since $d_1 \equiv -d_2 \pmod{2^e}$ and $e \geq 2$, we have $d_1 \not\equiv d_2 \pmod{2^{e'}}$. Therefore, we can determine which is the case.

The rest of this section describes the two algorithmic employed in SQIsign2D². The first algorithm, named **ImRanIso**(Algorithm 2.25), is used in key generation to construct a random non-smooth d -isogeny starting from the base curve E_0 . The second algorithm, named **GenImRanIso**(Algorithm 2.26), assumes the knowledge of an isogeny $\tau : E_0 \rightarrow E$ and is used to generate a random isogeny of non-smooth degree d starting from E . These two procedures are later invoked in the key-generation and response phases of the protocol, respectively.

2.9.1. Generating a random isogeny from E_0

This subsection describes **ImRanIso**(Algorithm 2.25), which is used in key generation to generate a random non-smooth d -isogeny from the reference curve E_0 .

In our parameter setting $p = 2^a 3^b f - 1$, $a' = \lfloor a/2 \rfloor$, where f is a small cofactor, the secret isogeny has degree approximately $p^{1/4}$. Using a full $(2^a, 2^a)$ -isogeny computation for this purpose is therefore not optimal. **ImRanIso**(Algorithm 2.25) reduces the degree requirements for 2-dimensional isogenies by leveraging $2^{a'} 3^b$ -isogeny in dimension 1. Hence this method reduces the degree that must be handled in dimension two and improves the efficiency of key generation. A sketch of **ImRanIso**(Algorithm 2.25) is illustrated in Figure 2.2.

The procedure begins by constructing an endomorphism $\theta \in \text{End}(E_0)$ of degree $\deg(\theta) = d(2^{a-a'} - d)3^b 2^{a'} > p$, where d is an odd positive integer with $d < p^{1/4}$. Since 3^b and $2^{a'}$ are smooth and the torsion subgroup $E_0[2^{a'} 3^b]$ is contained in $E_0(\mathbb{F}_{p^2})$, the $2^{a'} 3^b$ -isogeny $\varphi : E_0 \rightarrow E'$ corresponding to $I_\varphi = \mathcal{O}_0(\bar{\theta}, 2^{a'} 3^b)$ can be computed and evaluated efficiently.

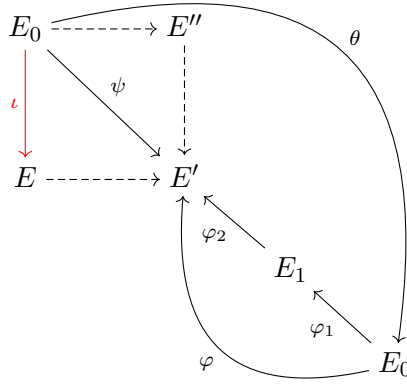


FIGURE 2.2. A sketch of Algorithm 2.25. The isogeny $\varphi = \varphi_2 \circ \varphi_1$, where φ_1 has degree $2^{a'}$ and φ_2 has degree 3^b .

Algorithm 2.25 $\text{ImRanIso}_{\mathcal{O}_0}(d, 3^b, 2^a, 2^{a'}, S)$

Input: An integer d coprime to 2 and 3; a finite set $S \subset E_0$

Output: The image curve E of a random d -isogeny ι starting from E_0 , the finite set $\iota(S) \subset E$, and the ideal I_ι corresponding to ι

- 1: $\theta, n \leftarrow \text{FullRepresentInteger}_{\mathcal{O}_0}(d(2^{a-a'-2} - d)2^{a'}3^b)$
 - 2: $(P, Q) \leftarrow (P_0, Q_0) \cdot \mathbf{M}_{2,\theta}$
 - 3: $e \leftarrow \text{DyadicValuation}(n)$
 - 4: Compute non-zero $(x : y) \in \mathbb{P}^1(\mathbb{Z}/(2^{a'-2e}3^b)\mathbb{Z})$ such that $\mathbf{M}_{2,\bar{\theta}} \cdot (x, y)^T \equiv 0 \pmod{2^{a'-2e}3^b}$
 - 5: $E_1, \{P, Q, R', S'\} \leftarrow \text{TwolsogenyChain}([x2^{a-a'+2e}]P_0 + [y2^{a-a'+2e}]Q_0, E_0, a' - 2e, \{P, Q, R_0, S_0\})$ //
Compute φ_1
 - 6: $E', \{P, Q\} \leftarrow \text{ThreelsogenyChain}([x]R' + [y]S', E_1, b, \{P, Q\})$ // Compute φ_2
 - 7: $r \leftarrow (d \cdot 3^b)^{-1} \pmod{2^{a-a'}}$
 - 8: $(E \times E'', [(\iota(T), \iota'(T)) \mid T \in S]) \leftarrow \text{Isogeny22ChainWithTorsion}([2^{a'}]P_0, [r2^{2e}]P), ([2^{a'}]Q'_0, [r2^{2e}]Q), [(T, 0_{E'}) \mid T \in S])$
 - 9: $I_\iota \leftarrow \mathcal{O}_0\langle \theta, d \rangle$
 - 10: **return** $E, \iota(S), I_\iota$
-

Moreover, the relation $\varphi \circ \theta = [2^{a'}3^b]\psi$ implies

$$\psi([2^{a'}]P_0) = [1/3^b] \circ \varphi \circ \theta(P_0), \quad \psi([2^{a'}]Q_0) = [1/3^b] \circ \varphi \circ \theta(Q_0),$$

where $\{P_0, Q_0\}$ is a basis of $E_0[2^a]$. Since $\langle [2^{a'}]P_0, [2^{a'}]Q_0 \rangle = E_0[2^{a-a'}]$ and $\deg(\psi) = d(2^{a-a'} - d)$ is odd, the target d -isogeny ι can be recovered efficiently by decomposing the corresponding two-dimensional isogeny $\Phi : E_0 \times E' \rightarrow E \times E''$.

The endomorphism θ is obtained by dividing 2, 3 until it is cyclic. If $\deg(\theta) < d(2^{a-a'} - d)$, we can obtain a $d(2^{a-a'} - d)$ -isogeny by supplementing several random 3-isogenies.

2.9.2. Generating a random auxiliary isogeny

This subsection describes GenImRanIso (Algorithm 2.26), which is used in the response phase to generate a random auxiliary isogeny starting from a general supersingular curve E .

In SQIsign2D^2 , the auxiliary isogeny is generated by combining $\text{EichlerModConstraint}$ and $\text{FullStrongApproximation}$ to construct an endomorphism $\theta \in \text{End}(E)$, where E is the codomain of secret key $\tau : E_0 \rightarrow E$. To ensure the solution of $\text{FullStrongApproximation}$ satisfies the condition that $\text{Nrd}(\alpha) > p \deg(\tau)^3 \approx p^{\frac{7}{4}}$, we set $\text{Nrd}(\alpha) = d(2^a - d)3^{2b}$. From the accessible

torsion subgroup $E[3^b]$, we compute $\varphi_1 : E \rightarrow E_1$, $\varphi_2 : E \rightarrow E_2$ by Velu's formula. By leveraging the two 3^b -isogenies of dimension 1, we are able to evaluate the isogeny from E_1 to E_2 of degree $d(D-d)$, and hence one can compute $\Phi : E_1 \times E_2 \rightarrow E' \times E''$ to obtain a d -isogeny $\iota' : E_1 \rightarrow E'$. Finally, pullback the isogeny ι' through φ_1 , and obtain the auxiliary isogeny $\iota = [\varphi_1]^* \iota'$ from E of degree d . A sketch of [GenImRanIso](#)(Algorithm 2.26) is illustrated in Figure 2.3.

Algorithm 2.26 $\text{GenImRanIso}_{\tau, I_\tau}(d, 2^a, 3^b, S)$

Input: An N -isogeny $\tau : E_0 \rightarrow E$ with $\text{End}(E_0) \cong \mathcal{O}_0$ and $\gcd(N, 6) = 1$, $(\frac{3}{N}) = -1$; an ideal I_τ corresponding to τ ; an integer $d \approx \sqrt{p}$ coprime to $2, 3$; and a finite set $S \subset E$ whose points have order coprime to 3

Output: The image curve E_ι of a random d -isogeny ι starting from E , and the finite set $\iota(S) \subset E_\iota$

```

1:  $P \leftarrow \tau(P_0), Q \leftarrow \tau(Q_0)$ 
2:  $(P_\alpha, Q_\alpha) \leftarrow (P_0, Q_0) \cdot \mathbf{M}_{2, \alpha}$ 
3:  $(C_0 : D_0) \leftarrow \text{EichlerModConstraint}(I_\tau, 1, 1)$ 
4: if  $\left(\frac{d(2^a-d)}{N}\right) = 1$  then
5:    $\alpha \leftarrow \text{FullStrongApproximation}_{d(2^a-d)3^{2b}}(N, C_0, D_0)$ 
6:   Compute non-zero  $(x_1 : y_1) \in \mathbb{P}^1(\mathbb{Z}/3^b\mathbb{Z})$  such that  $\mathbf{M}_{2, \alpha} \cdot (x_1, y_1)^T \equiv 0 \pmod{3^b}$ 
7:   Select  $K \in E$  such that  $\langle [x_1]\tau(R_0) + [y_1]\tau(S_0), K \rangle = E[3^b]$ 
8:    $E_1, \{P, Q, S, K\} \leftarrow \text{ThreelsogenyCodomain}([x_1]\tau(R_0) + [y_1]\tau(S_0), E, b, \{P, Q, S, K\})$  // Compute  $\varphi_1$ 
9:   Compute non-zero  $(x_2 : y_2) \in \mathbb{P}^1(\mathbb{Z}/3^b\mathbb{Z})$  such that  $\mathbf{M}_{2, \bar{\alpha}} \cdot (x_2, y_2)^T \equiv 0 \pmod{3^b}$ 
10:   $E_2, \{P_\alpha, Q_\alpha\} \leftarrow \text{ThreelsogenyCodomain}([x_2]\tau(R_0) + [y_2]\tau(S_0), E, b, \{P_\alpha, Q_\alpha\})$  // Compute  $\varphi_2$ 
11: else
12:    $\alpha \leftarrow \text{FullStrongApproximation}_{d(2^a-d)3^{2b-1}}(N, C_0, D_0)$ 
13:   Compute non-zero  $(x_1 : y_1) \in \mathbb{P}^1(\mathbb{Z}/3^b\mathbb{Z})$  such that  $\mathbf{M}_{2, \alpha} \cdot (x_1, y_1)^T \equiv 0 \pmod{3^b}$ 
14:   Select  $K \in E$  such that  $\langle [x_1]\tau(R_0) + [y_1]\tau(S_0), K \rangle = E[3^b]$ 
15:    $E_1, \{P, Q, S, K\} \leftarrow \text{ThreelsogenyCodomain}([x_1]\tau(R_0) + [y_1]\tau(S_0), E, b, \{P, Q, S, K\})$  // Compute  $\varphi_1$ 
16:   Compute non-zero  $(x_2 : y_2) \in \mathbb{P}^1(\mathbb{Z}/3^{b-1}\mathbb{Z})$  such that  $\mathbf{M}_{2, \bar{\alpha}} \cdot (x_2, y_2)^T \equiv 0 \pmod{3^{b-1}}$ 
17:    $E_2, \{P_\alpha, Q_\alpha\} \leftarrow \text{ThreelsogenyCodomain}([3x_2]\tau(R_0) + [3y_2]\tau(S_0), E, b-1, \{P_\alpha, Q_\alpha\})$  // Compute  $\varphi_2$ 
18: end if
19:  $r \leftarrow (d \cdot 3^b)^{-1} \pmod{2^a}$ 
20:  $(E', S, (K, *), \cdot) \leftarrow \text{Isogeny22Chain}((P, [r]P_\alpha)(Q, [r]Q_\alpha), [(T, 0_{E_2}) \mid T \in S], (K, 0_{E_2}))$ 
21:  $(E_\iota, S) \leftarrow \text{ThreelsogenyCodomain}(K, E', b, \{S\})$  // Compute  $\varphi_1$ 
22: return  $E_\iota, [1/3^b]S$ 

```

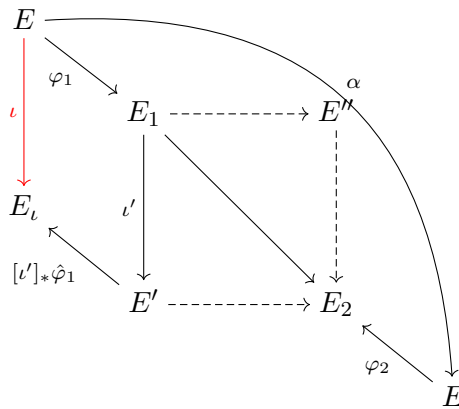


FIGURE 2.3. A schematic view of Algorithm 2.26.

SQIsign2D²

The SQIsign2D variants are attractive due to their compact signatures. SQIsign2D², as a novel digital signature scheme within the SQIsign2D family, is instantiated by applying the Fiat–Shamir transform to a Σ -protocol. The relevant definitions are provided in 3.1.

3.1. Digital signatures

3.1.1. Σ -Protocols

Let $R \subseteq \mathcal{X} \times \mathcal{W}$ be a relation. A Σ -protocol for R is a 3-round interactive protocol between a prover P and a verifier V on common input $x \in \mathcal{X}$, where the prover additionally holds a witness $w \in \mathcal{W}$ such that $(x, w) \in R$. The protocol has the following proceeds:

1. **Commitment:** the prover runs a PPT algorithm P_1 to compute

$$(a, st) \leftarrow P_1(x, w),$$

and sends the commitment a to the verifier.

2. **Challenge:** the verifier samples a challenge

$$c \xleftarrow{\$} \mathcal{C},$$

from a prescribed challenge space $\mathcal{C}$, and sends c to the prover.

3. **Response:** the prover runs a polynomial time algorithm P_2 and computes a response

$$r \leftarrow P_2(x, w, a, c, st),$$

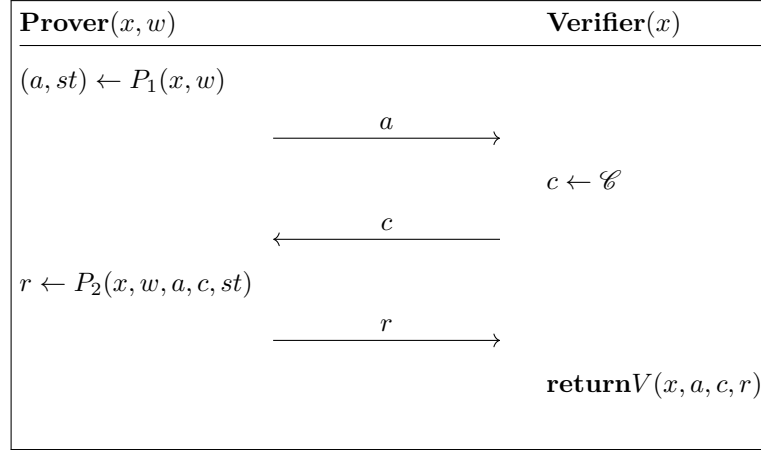
and sends r to the verifier.

Finally, the verifier outputs

$$b \leftarrow V(x, a, c, r) \in \{0, 1\},$$

where $b = 1$ means *accept*.

The transcript of an execution is denoted by $\tau = (a, c, r)$. Moreover, a Σ -protocol must satisfy several properties: correctness; special soundness; honest-verifier zero-knowledge. The detailed definitions of these properties are described in Chapter 9.



The Fiat–Shamir transform converts Σ -protocols into non-interactive proofs by replacing the verifier’s random challenge with a hash output.

3.1.2. The Fiat–Shamir Transform

We recall the Fiat–Shamir transform, which converts three round Σ -protocols into non-interactive ones by replacing the verifier’s random challenge with the output of a hash function. This is the conceptual bridge from identification to digital signatures.

Fiat–Shamir signature from a Σ -protocol Fix a hash function $H : \{0, 1\}^* \rightarrow \mathcal{C}$. Define a signature scheme $\Sigma_{\text{FS}} = (\text{KGen}, \text{Sign}, \text{Verify})$ as:

- **KeyGen**(1^λ): generate the secret key $sk = (x, w)$ and the public key $pk = x$, where $(x, w) \in R$. Output sk and pk .
- **Sign**(sk, m): for secret key pair $(x, w) := sk$, compute $a \leftarrow P_1(x, w)$; set $c := H(a \parallel m)$; compute $r \leftarrow P_2(x, w, a, c)$; output $\sigma := (c, r)$.
- **Verify**(pk, m, σ): for signature $\sigma = (c, r)$; set $c := H(a \parallel m)$; output $V(pk, a, c, r)$.

3.2. Parameter

3.2.1. Scheme parameters

The public parameters and fixed precomputed objects used in SQIsign2D^2 are as follows.

- A security parameter λ .
- A 2λ -bit prime $p = f \cdot 2^a \cdot 3^b - 1$ with $2^a \approx 3^b \approx \sqrt{p}$ and f a small cofactor.
- A power of two $2^{a'} \approx p^{1/4}$. This parameter is used in [ImRanIso](#) to reduce the degree of 2-dimensional isogenies defined over $\mathbb{F}_{p^2}$.
- Two canonical torsion basis $\{P_0, Q_0\} \subset E_0[2^a]$, $\{R_0, S_0\} \subset E_0[3^b]$.
- A hash function HASH onto $[3^b]$.

3.2.2. Algorithmic parameters

Several constants are required in some algorithmic blocks of SQIsign2D². Some of these constants should be chosen carefully as they may influence the failure probability of the algorithms using them. In the following, we choose these constants such that the failure probability is smaller than 2^{-64} . For more explanatory details, see Chapter 8.

- `primality_num_iter` is the number of Miller-Rabin tests used in to test pseudo-primality for the equivalent ideal functionality. It should be such that $4^{-\text{primality_num_iter}} < 2^{-64}$ (this is a common upper bound on the probability of failure of repetitions of Miller Rabin test).
- `QUAT_random_prime_attempts` determines the upper bound of the number of iterations to generate a random prime number. This constant is chosen to be 64.
- `QUAT_repres_num_trial` is the upper bound of the number of iterations in [FullRepresentInteger](#) so that it succeeds with good probability. This constant is computed as

$$\text{QUAT_repres_num_gamma_trial} = \lceil \log_{1-1/(4 \ln p)}(2^{-64}) \rceil.$$

- `QUAT_strongappro_num_trial` is the upper bound of the number of iterations in [FullStrongApproximation](#) so that it succeeds with good probability. This constant is computed as

$$\text{QUAT_strongappro_num_trial} = \lceil \log_{1-1/(3 \ln p)}(2^{-64}) \rceil.$$

The following sections specify the SQIsign2D² identification protocol built from the algorithms introduced in Section 2.9. A compact variant is described later in Section 3.5.

3.3. Key generation

[SQIsign2D².KeyGen](#) generates a public key `pk` and a secret key `sk`. The public key is the curve E_A , which is the codomain of a secret isogeny $\tau : E_0 \rightarrow E_A$ and `hintt` ($t = 2, 3$) is to generate deterministic basis on E_A . The secret key is a representation of this isogeny together with image torsion points data used in later computations.

More precisely, key generation first samples a prime N_τ such that $N_\tau < p^{1/4}$, and then invokes [ImRanIso](#) to construct a corresponding isogeny.

For efficiency, the secret key stores some additional data of the isogeny τ . These extra data will be useful to generate signatures efficiently. Concretely, the secret key consists of:

- the public-key curve E_A ;
- the secret ideal I_τ corresponding to the secret isogeny $\tau : E_0 \rightarrow E_A$;
- the degree N_τ of τ ;
- the images $\tau(P_0), \tau(Q_0), \tau(R_0), \tau(S_0)$ of the fixed basis points P_0, Q_0, R_0, S_0 under τ .

Algorithm 3.1 SQIsign2D².KeyGen()**Output:** Public key pk and secret key

```

1: loop
2:   Randomly select a prime  $N_\tau < p^{1/4}$ 
3:   if  $\left(\frac{3}{N_\tau}\right) = -1$  then
4:     Break
5:   end if
6: end loop
7:  $(E_A, \tau(P_0), \tau(Q_0), \tau(R_0), \tau(S_0), I_\tau) \leftarrow \text{ImRanIso}_{\mathcal{O}_0}(N_\tau, 3^b, 2^a, 2^{\lfloor \frac{a}{2} \rfloor}, \{P_0, Q_0, R_0, S_0\})$ 
8:  $\text{pk} \leftarrow (E_A)$ 
9:  $\text{sk} \leftarrow (E_A, I_\tau, N_\tau, \tau(P_0), \tau(Q_0), \tau(R_0), \tau(S_0))$ 
10: return pk, sk

```

3.4. Signing

As a sigma protocol stated in in section 3.1.1, the signature of SQIsign2D² consist of three phases: commitment phase, challenge phase and response phase in section 1.4 for overview of SQIsign2D². Just like introduced in section 3.1.2. The commitment phase consists of computing a random isogeny $\psi : E_0 \rightarrow E_{\text{com}}$ and the ideal I_ψ . During the challenge phase, the commitment $j(E_{\text{com}})$ and the message msg are hashed to generate a challenge chl . Such a challenge defines a challenge isogeny $\varphi : E_{\text{com}} \rightarrow E_{\text{chl}}$ whose corresponding ideal is denoted by I_φ . Finally, the response phase consists of computing an equivalent ideal $I \sim \bar{I}_\tau I_\psi I_\varphi$ and translating I to its corresponding isogeny $\sigma : E_A \rightarrow E_{\text{chl}}$. The output of the signing procedure is a signature containing the challenge chl and some additional data describing the response isogeny $\sigma : E_A \rightarrow E_{\text{chl}}$. We will give more detail on each phase of the signing procedure.

Algorithm 3.2 SQIsign2D².Sign(sk, msg)**Input:** Secret signing key sk and message msg $\in \{0, 1\}^*$.**Output:** Signature $\mathcal{S}$.

```

1: Parse sk as  $(E_A, I_\tau, N_\tau, \tau(P_0), \tau(Q_0), \tau(R_0), \tau(S_0))$ 
   // Commitment
2: while true do
3:   loop
4:     Randomly select a prime  $N_\psi < \sqrt{p}$ 
5:     if  $\gcd(N_\psi, 6) = 1$  then
6:       Break
7:     end if
8:   end loop
9: end while
10:  $(E_{\text{com}}, \psi(P_0), \psi(Q_0), \psi(R_0), \psi(S_0), I_\psi) \leftarrow \text{ImRanIso}_{\mathcal{O}_0}(N_\psi, 3^b, 2^a, 1, \{P_0, Q_0, R_0, S_0\})$ 
11:  $R_{\text{com}}, S_{\text{com}}, \text{hint}_{3,\text{com}} \leftarrow \text{TorsionBasisToHint}_3(E_{\text{com}}, b)$ 
   // Challenge
12: chl  $\leftarrow \text{HASH}(\text{pk} \parallel j(E_{\text{com}}) \parallel \text{msg})$ 
   // Response
13:  $E_{\text{chl}}, \{\varphi(\psi(P_0)), \varphi(\psi(Q_0))\} \leftarrow \text{ThreelsogenyChain}(R_{\text{com}} + [\text{chl}]S_{\text{com}}, E_{\text{com}}, b, \{\psi(P_0), \psi(Q_0)\})$ 
14:  $P_{\text{chl}}, Q_{\text{chl}}, \text{hint}_{2,\text{chl}} \leftarrow \text{TorsionBasisToHint}_2(E_{\text{chl}}, a)$ 
15:  $M_{\text{com},3} \leftarrow \text{ChangeOfBasis}_{3^b}(E_{\text{com}}, (\psi(R_0), \psi(S_0)), (R_{\text{com}}, S_{\text{com}}))$ 
16:  $[c_0, c_1]^T \leftarrow M_{\text{com},3}[1, \text{chl}]^T$ 
17:  $I'_\varphi \leftarrow \text{KernelDecomposedToIdeal}(c_0, c_1)$ 
18:  $J \leftarrow \overline{I}_\tau(I_\psi \cap I'_\varphi)$ 
19:  $\alpha \leftarrow \text{RandomEquivalentQuaternion}(J)$ 
20:  $I_\sigma \leftarrow J \frac{\bar{\alpha}}{\text{Nrd}(J)}, q \leftarrow \frac{\text{Nrd}(\alpha)}{\text{Nrd}(J)}$ 
21: if  $q$  is even or  $q > 2^a$  then
22:   Go back to Step 19
23: end if
24:  $(\sigma(\tau(P_0)), \sigma(\tau(Q_0))) \leftarrow ([ (3^b N_\psi)^{-1} ] \varphi(\psi(P_0)), [ (3^b N_\psi)^{-1} ] \varphi(\psi(Q_0))) \cdot M_{\bar{\alpha}}$ 
25:  $d \leftarrow 2^a - q$ 
26:  $(E_{\text{aux}}, \{\varphi_{\text{aux}}(\tau(P_0)), \varphi_{\text{aux}}(\tau(Q_0))\}) \leftarrow \text{GenImRanIso}_{\tau, I_\tau}(d, 2^a, 3^b, \{\tau(P_0), \tau(Q_0)\})$ 
27:  $P_{\text{aux}}, Q_{\text{aux}}, \text{hint}_{2,\text{aux}} \leftarrow \text{TorsionBasisToHint}_2(E_{\text{aux}}, a)$ 
28:  $M_{\text{aux},2} \leftarrow \text{ChangeOfBasis}_2(E_{\text{aux}}, (\varphi_{\text{aux}}(\tau(P_0)), \varphi_{\text{aux}}(\tau(Q_0))), (P_{\text{aux}}, Q_{\text{aux}}))$ 
29:  $(U_2, V_2) \leftarrow -(\sigma(\tau(P_0)), \sigma(\tau(Q_0))) \cdot M_{\text{aux},2}$ 
30:  $P_{\text{chl}}, Q_{\text{chl}}, \text{hint}_{2,\text{chl}} \leftarrow \text{TorsionBasisToHint}_2(E_{\text{chl}}, a)$ 
31:  $M_{\text{ver},2} \leftarrow \text{ChangeOfBasis}_2(E_{\text{chl}}, (P_{\text{chl}}, Q_{\text{chl}}), (U_2, V_2))$ 
32:  $\mathcal{S} \leftarrow (E_{\text{aux}}, M_{\text{ver},2}, \text{hint}_{2,\text{aux}}, \text{hint}_{2,\text{chl}})$ 
33: return  $\mathcal{S}$ 

```

3.4.1. Commitment

The prover keeps the associated ideal and the images of the 2^a -torsion basis as secret commitment data. He first randomly generates a prime $N_\psi < \sqrt{p}$. Then he computes the commitment isogeny $\psi : E_0 \rightarrow E_1$ by **ImRanIso**, and obtains the image points of $E_0[2^a], E_0[3^b]$ under ψ . The prover sets $j(E_1)$ as the commitment.

3.4.2. Challenge

The challenge phase consists of hashing the public key pk , the j -invariant of the commitment curve E_{com} , and the message msg to a λ -bit number.

Given the commitment curve E_{com} , the verifier generates the challenge by selecting a cyclic subgroup of $E_{\text{com}}[3^b]$. This is done by fixing the canonical basis of $E_{\text{com}}[3^b]$ and choosing a random linear combination.

3.4.3. Response

The response phase consists of two conceptually distinct parts. First, the prover computes a response isogeny $\sigma : E_A \rightarrow E_{\text{chl}}$ whose degree is denoted by q . This step is carried out in the ideal-theoretic setting by composing the secret ideal, the commitment ideal, and the ideal corresponding to the challenge isogeny.

Second, the prover computes an auxiliary isogeny $\varphi_{\text{aux}} : E_A \rightarrow E_{\text{aux}}$ of degree $d = 2^a - q$. This auxiliary path is generated by [GenImRanIso](#) and is used to encode the response in a form that can later be checked by a single $(2^a, 2^a)$ -isogeny computation.

Before giving the algorithm, we record the admissibility condition imposed on the response degree.

The response ideal is computed as follows. From the challenge subgroup $\langle R_{\text{com}} + [\text{com}]S_{\text{com}} \rangle$, the prover first recovers the ideal I_φ corresponding to the challenge isogeny $\varphi : E_{\text{com}} \rightarrow E_{\text{chl}}$. To be more specific, since $(R_{\text{com}}, S_{\text{com}})$ is the deterministic basis corresponding to $\text{hint}_{3,\text{com}}$, it is computed as follows. First, we look at the pullback of the isogeny with kernel $\langle R_{\text{com}} + [\text{chl}]S_{\text{com}} \rangle$ under the commit isogeny ψ . By computing $\mathbf{M}_{\text{com},3}$ from [ChangeOfBasis₃](#), its kernel is equal to $\langle [c_1]P_0 + [c_2]Q_0 \rangle$, where $[c_1, c_2]^T = \mathbf{M}_{\text{com},3}[1, \text{chl}]^T$. We then generate the ideal I'_φ via the function [KernelDecomposedToIdeal_{3b}](#) and finally compute $I_\varphi = [I_\psi]^* I'_\varphi$.

The composite ideal $J = \overline{I_\tau} I_\psi I_\varphi = \overline{I_\tau} (I_\psi \cap I'_\varphi)$ corresponds to the composed isogeny $\varphi \circ \psi \circ \hat{\tau}$. The prover then uniformly samples an element $\alpha \in J$ such that $q = \frac{\text{Nrd}(\alpha)}{\text{Nrd}(J)}$ is 2^a -adequate. Since the quaternion algebra $B_{p,\infty}$ is a quadratic space whose bilinear form is given by $\langle \alpha, \beta \rangle = \text{Trd}(\alpha\bar{\beta})$. In particular, we have $\langle \alpha, \alpha \rangle = 2\text{Nrd}(\alpha)$. Hence, to sample $\alpha \in J$, it is possible to use [LatticeSampling](#) with bound the quantity 2^a and reject smapling. The procedure is summarized in [RandomEquivalentQuaternion](#). The response ideal is $I_\sigma = J \frac{\bar{\alpha}}{\text{Nrd}(J)}$, which is equivalent to J . By checking $\hat{\sigma} \circ \varphi \circ \psi \circ \hat{\tau} = \alpha$, and evaluating $\hat{\alpha}$, $\hat{\tau}$, ψ , and φ , the prover obtains an efficient representation of the response isogeny σ through its action on the canonical basis of $E_A[2^a]$.

Once the response isogeny has been determined, the prover computes the auxiliary path starting from the public-key curve E_A . This is the dominant part of the response phase. The relevant degree depends on whether the norm prescribed to [FullStrongApproximation](#) satisfies the required quadratic-residue condition modulo N_τ . More precisely, the endomorphism θ produced there satisfies

$$\text{Nrd}(\alpha) = d(2^a - d)3^{2b} \equiv \lambda^2 p(C_0^2 + D_0^2) \pmod{N_\tau}, \quad (3.1)$$

for some integers λ, C_0, D_0 , where $d = 2^a - q$. By the definition of [EichlerModConstraint](#), there exists $m \in \mathbb{Z}$ such that $m + C_0 j + D_0 k \in I_\tau$, which implies

$$m^2 + p(C_0^2 + D_0^2) \equiv 0 \pmod{N_\tau}.$$

Hence, we have

$$\left(\frac{p(C_0^2 + D_0^2)}{N_\tau} \right) = \left(\frac{-1}{N_\tau} \right).$$

If $\left(\frac{d(2^a-d)}{N_\tau} \right) = \left(\frac{-1}{N_\tau} \right)$, then we can solve λ in Equation 3.1; otherwise, since $\left(\frac{3}{N_\tau} \right) = -1$, we choose α has degree $d(2^a - d)3^{2b-1}$, and then $\left(\frac{d(2^a-d)3^{2b-1}}{N_\tau} \right) = \left(\frac{-1}{N_\tau} \right)$. This is exactly the condition required for the existence of the desired endomorphism.

After the auxiliary isogeny has been computed, the prover computes the canonical basis of $E_{\text{aux}}[2^a]$ by $\text{hint}_{2,\text{aux}}$, and obtains $\mathbf{M}_{\text{aux},2}$ from [ChangeOfBasis₂](#), where $(P_{\text{aux}}, Q_{\text{aux}}) = (\varphi_{\text{aux}}(P_A), \varphi_{\text{aux}}(Q_A))\mathbf{M}_{\text{aux},2}$. Then, from

$$(\sigma \circ \hat{\varphi}_{\text{aux}}(P_{\text{aux}}), \sigma \circ \hat{\varphi}_{\text{aux}}(Q_{\text{aux}})) = d(\sigma(P_A), \sigma(Q_A))\mathbf{M}_{\text{aux},2}$$

and $2^a = q + d$, one obtains

$$\begin{aligned} (U_2, V_2) &= -(\sigma(P_A), \sigma(Q_A))\mathbf{M}_{\text{aux},2} \\ &= \left(\left[\frac{1}{q} \right] \sigma \circ \hat{\varphi}_{\text{aux}}(P_{\text{aux}}), \left[\frac{1}{q} \right] \sigma \circ \hat{\varphi}_{\text{aux}}(Q_{\text{aux}}) \right). \end{aligned}$$

After obtaining $\mathbf{M}_{\text{ver},2}$ from [ChangeOfBasis₂](#), where $(U_2, V_2) = (P_{\text{chl}}, Q_{\text{chl}})\mathbf{M}_{\text{ver},2}$, the response transmitted to the verifier is therefore $(E_{\text{aux}}, \mathbf{M}_{\text{ver},2})$.

3.4.4. Verification

The algorithm [SQIsign2D².Verify](#) describes the verification procedure. It checks whether the signature is valid by evaluating the response isogeny $\sigma : E_A \rightarrow E_{\text{chl}}$. On input a message `msg`, a signature $(E_{\text{aux}}, \mathbf{M}_{\text{ver},2}, \text{chl}, \text{hint}_{\text{aux}})$, and a public key $\text{pk} = (E_A, \text{hint}_2, \text{hint}_2)$, the verifier first checks that the curves E_A and E_{aux} are indeed supersingular.

For verifying that a curve E is supersingular, we should check there exists a basis for its 2^a -torsion subgroup in $E(\mathbb{F}_{p^2})$. This is equivalent to show that the bases generated from hint_2 , $\text{hint}_{2,\text{aux}}$ and $\text{hint}_{2,\text{chl}}$ span the full 2^a -torsion. Moreover, these points are used to compute both 1-dimensional and 2-dimensional isogenies. Hence, before isogeny computation, the verifier implicitly checks that the input is a correct basis. Further details are provided in Sections ?? and 7.5.

The formulas to compute the two-dimensional isogenies discussed in 2.5 require that kernels are maximal isotropic subgroups. Hence, to ensure that the algorithms employed during verification behave correctly, it is fundamental to additionally check this condition; we refer to 7.5 for further detail on this step.

The verifier first computes the deterministic basis $(P_{\text{com}}, Q_{\text{com}})$ on E_{com} corresponding to the hint hint_{com} . Such a basis is then used to generate the kernel of the challenge isogeny $\varphi : E_{\text{com}} \rightarrow E_{\text{chl}}$: given the challenge `chl`, the kernel $\ker(\varphi)$ is equal to $\langle P_{\text{com}} + [\text{chl}]Q_{\text{com}} \rangle$.

Using the hint hint_{aux} , the verifier computes the deterministic basis $(P_{\text{aux}}, Q_{\text{aux}})$. Similarly, using the hint hint_{chl} , the basis $(P_{\text{chl}}, Q_{\text{chl}})$ is computed. To reconstruct the correct evaluation under the response isogeny, it is necessary to scale $(P_{\text{chl}}, Q_{\text{chl}})$ by the matrix $\mathbf{M}_{\text{chl}}$, i.e.,

$$(U_2, V_2) = (P_{\text{chl}}, Q_{\text{chl}})\mathbf{M}_{\text{ver},2}.$$

After that, verification is performed by reconstructing a single $(2^a, 2^a)$ -isogeny from the response data. From the subgroup

$$\begin{aligned} K &= \langle (P_{\text{aux}}, U_2), (Q_{\text{aux}}, V_2) \rangle \\ &= \left\langle \left(P_{\text{aux}}, \left[\frac{1}{q} \right] \sigma \circ \hat{\varphi}_{\text{aux}}(P_{\text{aux}}) \right), \left(Q_{\text{aux}}, \left[\frac{1}{q} \right] \sigma \circ \hat{\varphi}_{\text{aux}}(Q_{\text{aux}}) \right) \right\rangle \\ &= \langle ([q]P_{\text{aux}}, \sigma \circ \hat{\varphi}_{\text{aux}}(P_{\text{aux}})), ([q]Q_{\text{aux}}, \sigma \circ \hat{\varphi}_{\text{aux}}(Q_{\text{aux}})) \rangle, \end{aligned}$$

the verifier computes the $(2^a, 2^a)$ -isogeny

$$\Phi : E_{\text{aux}} \times E_{\text{chl}} \rightarrow A.$$

The verifier accepts if the image A is isomorphic to $F_1 \times F_2$ and $F_1 \cong E_A$ or $F_2 \cong E_A$. Otherwise, the verifier rejects.

Algorithm 3.3 SQIsign2D².Verify

Input: The public key $\text{pk} = E_A$, the commitment curve E_{com} , the challenge K_{cha} , and the response $\mathcal{S} = (E_{\text{aux}}, M_{\text{ver},2})$

Output: true or false

```

1:  $R_{\text{com}}, S_{\text{com}}, \text{hint}_{3,\text{com}} \leftarrow \text{TorsionBasisToHint}_3(E_{\text{com}}, b)$ 
2:  $E_{\text{chl}}, T \leftarrow \text{ThreeIsogenyChain}(R_{\text{com}} + [\text{chl}]S_{\text{com}}, E_{\text{com}}, b, 3)$ 
3:  $P_{\text{chl}}, Q_{\text{chl}}, \text{hint}_{2,\text{chl}} \leftarrow \text{TorsionBasisToHint}_2(E_{\text{chl}}, a)$ 
4:  $P_{\text{aux}}, Q_{\text{aux}}, \text{hint}_{2,\text{aux}} \leftarrow \text{TorsionBasisToHint}_2(E_{\text{aux}}, a)$ 
5:  $(U_2, V_2) \leftarrow (P_{\text{chl}}, Q_{\text{chl}})M_{\text{ver},2}$ 
6:  $(F_1 \times F_2, T_1) \leftarrow \text{Isogeny22Chain}((P_{\text{aux}}, U_2), (Q_{\text{aux}}, V_2), (0_{E_{\text{aux}}}, T))$ 
7: if  $E_1 \cong E_A, T_1 = (0_{E_A}, *)$  then
8:   else if  $E_2 \cong E_A, T_1 = (*, 0_{E_A})$  then
9:     return True
10: else
11:   return False
12: end if

```

3.5. Compact SQIsign2D²

Applying the Fiat-Shamir transform to the identification protocol of Section 3.1.2 yields a signature scheme whose uncompressed signature consists of the commitment curve, the auxiliary curve, and the response data derived from the response isogeny. In its basic form, the resulting signature is

$$(j(E_{\text{com}}), j(E_{\text{aux}}), M_{\text{ver},2}).$$

This section describes a more compact representation, denoted *CompactSQIsign2D²*, in which part of the response data is replaced by information that can be reconstructed more efficiently by the verifier.

3.5.1. Compact key generation

Different from SQIsign2D², we will modify the secret key in key generation phase for CompactSQIsign2D². To avoid computing τ in sign phase, the information $(\tau(P_0), \tau(Q_0), \tau(R_0), \tau(S_0))$ is stored in the secret key. We now briefly explain how this is done. First, we generate a deterministic basis of $E_A[2^a]$ from hint_2 . Then, we compute the change-of-basis matrix $\mathbf{M}_{\text{sk},2} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ such that

$$\mathbf{M}_{\text{sk},2} \cdot \begin{pmatrix} \tau(P_0) \\ \tau(Q_0) \end{pmatrix} = \begin{pmatrix} [a]\tau(P_0) + [b]\tau(Q_0) \\ [c]\tau(P_0) + [d]\tau(Q_0) \end{pmatrix} = \begin{pmatrix} P_A \\ Q_A \end{pmatrix},$$

where P_A, Q_A is a deterministic basis on $E_A[2^a]$ from hint_2 . The matrix $\mathbf{M}_{\text{sk},2}$ is part of the secret key. Overall, as we generate the deterministic basis, we save a hint $\text{hint}_2, \text{hint}_3$, which we append to the public key pk . We also compute $\mathbf{M}_{\text{sk},2}, \mathbf{M}_{\text{sk},3}$ as part of secret key. It should be noted that the hint hint_{pk} is then used to recompute the basis of public key in verification phase.

Concretely, the secret key consists of:

- the public-key curve E_A ;
- the secret ideal I_τ corresponding to the secret isogeny $\tau : E_0 \rightarrow E_A$;
- the degree N_τ of τ ;
- the hint hint_2 used to generate a deterministic basis on $E_A[2^a]$;
- the hint hint_3 used to generate a determinsric basis on $E_A[3^b]$;

- the matrix $M_{\text{sk},2}$ for computing $\tau(P_0), \tau(Q_0)$;
- the matrix $M_{\text{sk},3}$ for computing $\tau(R_0), \tau(S_0)$.

Algorithm 3.4 CompactSQIsign2D².KeyGen()

Output: Public key pk and secret key

```

1: loop
2:   Randomly select a prime  $N_\tau < p^{1/4}$ 
3:   if  $\left(\frac{3}{N_\tau}\right) = -1$  then
4:     Break
5:   end if
6: end loop
7:  $(E_A, \tau(P_0), \tau(Q_0), \tau(R_0), \tau(S_0), I_\tau) \leftarrow \text{ImRanIso}_{\mathcal{O}_0}(N_\tau, 3^b, 2^a, 2^{\lfloor \frac{a}{2} \rfloor}, \{P_0, Q_0, R_0, S_0\})$ 
8:  $P_A, Q_A, \text{hint}_2 \leftarrow \text{TorsionBasisToHint}_2(E_A, a)$ 
9:  $R_A, S_A, \text{hint}_3 \leftarrow \text{TorsionBasisToHint}_3(E_A, b)$ 
10:  $M_{\text{sk},2} \leftarrow \text{ChangeOfBasis}_{2^a}(E_A, (\tau(P_0), \tau(Q_0)), (P_A, Q_A))$ 
11:  $M_{\text{sk},3} \leftarrow \text{ChangeOfBasis}_{3^b}(E_A, (\tau(R_0), \tau(S_0)), (R_A, S_A))$ 
12:  $\text{pk} \leftarrow (E_A, \text{hint}_2, \text{hint}_3)$ 
13:  $\text{sk} \leftarrow (E_A, \text{hint}_2, \text{hint}_3, I_\tau, N_\tau, M_{\text{sk},2}, M_{\text{sk},3})$ 
14: return pk, sk

```

3.5.2. Compact signing

For signing, compression step further reduces the signature size by removing the explicit transmission of data attached to E_{aux} and E_{chl} . Instead, the signer sends data attached to E_A and to another elliptic curve E_3 arising from the corresponding $(2^a, 2^a)$ -isogeny computation. Since E_A is already part of the public key, this eliminates the need to transmit any additional information about that factor. To obtain the commitment curve in the signature, the signer also includes data allowing the verifier to reconstruct the dual of the challenge isogeny.

Under this representation, the compressed signature has the form

$$(j(E_3), M_{\text{ver},2} t, g \text{ bin}, \text{chl}),$$

where:

- $j(E_3)$ denotes the elliptic curve E_3 obtained from the $(2^a, 2^a)$ -isogeny computation;
- the matrix $M_{\text{ver},2}$ for computing the data on E_3 ;
- $t \in \mathbb{Z}/3^b\mathbb{Z}$ is a scalar, $g \in \{0, 1\}$ is a bit used to recover the challenge subgroup efficiently;
- $\text{bin} \in \{0, 1\}$ records which factor of the split abelian surface corresponds to E_{chl} ;
- chl is the challenge.

The compact response algorithm begins from the same data as the response algorithm of Section 3.4.3, but outputs the compressed signature instead of the pair $(E_{\text{aux}}, U_2, V_2)$. The signer first derives the challenge subgroup from the message and the commitment curve, then computes the response and auxiliary data exactly as in SQIsign2D².Sign, and finally transfers the response information from $E_{\text{aux}} \times E_{\text{chl}}$ to the split product $E_A \times E_3$.

The values t and g encode the challenge in a form that allows the verifier to reconstruct the dual challenge isogeny without access to the original commitment transcript. The matrix $M_{\text{ver},2}$ is a matrix for computing the compressed image of the response data after moving from the product $E_{\text{aux}} \times E_{\text{chl}}$ to the product $E_A \times E_3$. Finally, the bit bin reveals the elliptic curve corresponds to E_{chl} , and the challenge chl is used in verification.

Algorithm 3.5 CompactSQIsign2D².Sign(sk, msg)**Input:** Secret signing key sk and message msg $\in \{0, 1\}^*$.**Output:** Signature $\mathcal{S}$.

```

1: Parse sk as  $(E_A, \text{hint}_2, \text{hint}_3, I_\tau, N_\tau, M_{\text{sk},2}, M_{\text{sk},3})$ 
2:  $P_A, Q_A, \text{hint}_2 \leftarrow \text{TorsionBasisToHint}_2(E_A, a)$ 
3:  $(\tau(P_0), \tau(Q_0)) \leftarrow (P_A, Q_A)M_{\text{sk},2}$ 
   // Commitment
4: while true do
5:   loop
6:     Randomly select a prime  $N_\psi < \sqrt{p}$ 
7:     if  $\gcd(N_\psi, 6) = 1$  then
8:       Break
9:     end if
10:   end loop
11: end while
12:  $(E_{\text{com}}, \psi(P_0), \psi(Q_0), \psi(R_0), \psi(S_0), I_\psi) \leftarrow \text{ImRanIso}_{\mathcal{O}_0}(N_\psi, 3^b, 2^a, 1, \{P_0, Q_0, R_0, S_0\})$ 
13:  $R_{\text{com}}, S_{\text{com}}, \text{hint}_{3,\text{com}} \leftarrow \text{TorsionBasisToHint}_3(E_{\text{com}}, b)$ 
   // Challenge
14: chl  $\leftarrow \text{HASH}(\text{pk} \parallel j(E_{\text{com}}) \parallel \text{msg})$  // See Section ?? for the binary encoding of elements in  $\mathbb{F}_{p^2}$ 
   // Response
15:  $E_{\text{chl}}, \{\varphi(\psi(P_0)), \varphi(\psi(Q_0))\}, T$  this is the dual of  $\varphi$  delete it  $\leftarrow \text{Threelsogeny-Chain}(R_{\text{com}} + [\text{chl}]S_{\text{com}}, E_{\text{com}}, b, \{\psi(P_0), \psi(Q_0)\})$ 
16:  $R_{\text{chl}}, S_{\text{chl}}, \text{hint}_{3,\text{aux}} \leftarrow \text{TorsionBasisToHint}_3(E_{\text{aux}}, b)$ 
17:  $\zeta_1 \leftarrow t_{3^b}(T, R_{\text{chl}}), \zeta_2 \leftarrow t_{3^b}(T, S_{\text{chl}})$ 
18: if  $\gcd(3, \zeta_1) = 1$  then
19:    $g \leftarrow 0, t \leftarrow \frac{\zeta_2}{\zeta_1}$ 
20: else
21:    $g \leftarrow 1, t \leftarrow \frac{\zeta_1}{\zeta_2}$ 
22: end if
23:  $M_{\text{com},3} \leftarrow \text{ChangeOfBasis}_{3^b}(E_{\text{com}}, (\psi(R_0), \psi(S_0)), (R_{\text{com}}, S_{\text{com}}))$ 
24:  $[c_0, c_1]^T \leftarrow M_{\text{com},3}[1, \text{chl}]^T$ 
25:  $I'_\varphi \leftarrow \text{KernelDecomposedToIdeal}(c_0, c_1)$ 
26:  $J \leftarrow \overline{I}_\tau(I_\psi \cap I'_\varphi)$ 
27:  $\alpha \leftarrow \text{RandomEquivalentQuaternion}(J)$ 
28:  $I_\sigma \leftarrow J \frac{\bar{\alpha}}{\text{Nrd}(J)}, q \leftarrow \frac{\text{Nrd}(\alpha)}{\text{Nrd}(J)}$ 
29: if  $q$  is even or  $q > 2^a$  then
30:   Go back to Step 16
31: end if
32:  $(\sigma(\tau(P_0)), \sigma(\tau(Q_0))) \leftarrow ([ (3^b N_\psi)^{-1} ] \varphi(\psi(P_0)), [ (3^b N_\psi)^{-1} ] \varphi(\psi(Q_0))) \cdot M_{\bar{\alpha}}$ 
33:  $d \leftarrow 2^a - q$ 
34:  $(E_{\text{aux}}, \{\varphi_{\text{aux}}(\tau(P_0)), \varphi_{\text{aux}}(\tau(Q_0))\}) \leftarrow \text{GenImRanIso}_{\tau, I_\tau}(d, 2^a, 3^b, \{\tau(P_0), \tau(Q_0)\})$ 
35:  $P_{\text{aux}}, Q_{\text{aux}}, \text{hint}_{2,\text{aux}} \leftarrow \text{TorsionBasisToHint}_2(E_{\text{aux}}, a)$ 
36:  $M_{\text{aux},2} \leftarrow \text{ChangeOfBasis}_2(E_{\text{aux}}, (\varphi_{\text{aux}}(\tau(P_0)), \varphi_{\text{aux}}(\tau(Q_0))), (P_{\text{aux}}, Q_{\text{aux}}))$ 
37: Compute  $(U_2, V_2) = -(\sigma(\tau(P_0)), \sigma(\tau(Q_0))) \cdot M_{\text{aux},2}$ 
38:  $(E_3, (U_3, *), (V_3, *)) \leftarrow \text{Isogeny22Chain}((P_{\text{aux}}, [q]U_2), (Q_{\text{aux}}, [q]V_2), (0_{E_{\text{aux}}}, [\frac{1}{d}]\sigma(P_A)), (0_{E_{\text{aux}}}, [\frac{1}{d}]\sigma(Q_A)))$ 
39:  $P_3, Q_3, \text{hint}_{2,E_3} \leftarrow \text{TorsionBasisToHint}_2(E_3, a)$ 
40:  $M_{\text{ver},2} \leftarrow \text{ChangeOfBasis}_{2^a}(E_3, (U_3, V_3), (P_3, Q_3))$ 
41: Let  $M, M'$  be the Montgomery coefficient of  $E_{\text{chl}}$  and  $E_{\text{aux}}$ 
42: if  $M \leq M'$  then
43:   bin  $\leftarrow 0$ 
44: else
45:   bin  $\leftarrow 1$ 
46: end if
47:  $\mathcal{S} \leftarrow (E_3, M_{\text{ver},2}, t, g, \text{bin})$ 
48: return  $\mathcal{S}$ 

```

3.5.3. Compact verification

Compact verification checks whether the compressed signature is true. The verifier first computes an isogeny from split abelian surface $E_A \times E_3$, from the image and **bin**, he obtains the curve E_{chl} . Then by computing the dual challenge isogeny in order to compare the transmitted challenge data with the hash output of commitment, public key and message.

Algorithm 3.6 CompactSQIsign2D².Verify

Input: The public key $\text{pk} = E_A$ and the response $\mathcal{S} = (E_3, M_{\text{ver},2}, t, g, \text{bin})$

Output: true or false

```

1:  $P_3, Q_3, \text{hint}_{2,E_3} \leftarrow \text{TorsionBasisToHint}_2(E_3, a)$ 
2:  $(U_3, V_3) \leftarrow (P_3, Q_3)M_{\text{ver},2}$ 
3:  $P_A, Q_A, \text{hint}_2 \leftarrow \text{TorsionBasisToHint}_2(E_A, a)$ 
4:  $F_1 \times F_2 \leftarrow \text{Isogeny22Chain}((P_A, U_3)(Q_A, V_3))$ 
5: Compute the Montgomery coefficient  $M_1, M_2$  of  $F_1, F_2$ 
6: if  $M_1 > M_2$  then
7:    $(M_1, M_2) \leftarrow (M_2, M_1), (F_1, F_2) \leftarrow (F_2, F_1)$ 
8: end if
9: if  $\text{bin} = 0$  then
10:   $E_{\text{chl}} \leftarrow F_1$ 
11: else
12:   $E_{\text{chl}} \leftarrow F_2$ 
13: end if
14:  $R_{\text{chl}}, S_{\text{chl}}, \text{hint}_{3,\text{chl}} \leftarrow \text{TorsionBasisToHint}_3(E_{\text{chl}}, b)$ 
15: if  $g = 0$  then
16:   $E_{\text{com}} \leftarrow \text{ThreelsogenyChain}(R_{\text{chl}} + [t]S_{\text{chl}}, E_{\text{chl}}, b)$ 
17: else
18:   $E_{\text{com}} \leftarrow \text{ThreelsogenyChain}([t]R_{\text{chl}} + S_{\text{chl}}, E_{\text{chl}}, b)$ 
19: end if
20: if  $\text{chl} = \text{HASH}(\text{pk} \parallel j(E_{\text{com}}) \parallel \text{msg})$  then
21:  return True
22: else
23:  return False
24: end if

```

Parameter setting

4.1. Parameter requirements

Each parameter set consists of a prime p (the characteristic of the finite field), and a hash function **HASH** (a hash function used to apply the Fiat–Shamir transform). All other parameters can be deduced from p and the above specification. In SQIsign2D², the prime p is selected under the following considerations:

1. Given a security parameter λ , we require prime sizes of $\log_2(p) \approx 2\lambda$ bits. Then, the best known classical generic attacks have complexity at least $O(\sqrt{p}) = O(2^\lambda)$, and the best known quantum generic attacks have complexity at least $O(\sqrt[4]{p}) = O(2^{\lambda/2})$.
2. We require p to be of the form

$$p = f \cdot 2^a \cdot 3^b - 1,$$

where $2^a \approx 3^b \approx \sqrt{p}$ and f is small prime. Equivalently, we require

$$p + 1 = f \cdot 2^a \cdot 3^b.$$

This special structure is essential for SQIsign2D²: the large 2-power part supports the two-dimensional isogeny computations, while the large 3-power part makes certain one-dimensional isogenies efficiently accessible.

3. Since 2^a is a power of two and 3^b is odd, primes of the form $p = f \cdot 2^a \cdot 3^b - 1$ automatically satisfy $p \equiv 3 \pmod{4}$. This is beneficial for the choice of the base supersingular curve and for efficient arithmetic over $\mathbb{F}_{p^2}$.
4. The hash function **HASH** is used to generate the challenge in the Fiat–Shamir transform. Concretely, the verifier’s challenge is derived from the 3^b -torsion, so the challenge space has size essentially 3^b . In practice, **HASH** should be instantiated with a standard cryptographic hash function providing at least $\lceil \log_2 3^b \rceil$ bits of output. For some parameter sets, the quantity 3^b is slightly smaller than 2^λ , we apply the grinding technique: to compensate for the $\lambda - \lceil \log_2 3^b \rceil$ missing bits, the hash function is constructed by iterating $2^{\lambda - \lceil \log_2 3^b \rceil}$ times a standard cryptographic hash function.
5. When sampling ideals, we need to select a short vector in the corresponding ideal lattice. Therefore, we require $2^a > \frac{2\sqrt{2p}}{\pi}$.

4.2. Parameter sets

In this section, we list parameter sets for all security levels. We consider four parameter categories, denoted by **Level1**, **Level2**, **Level3**, and **Level5**. They are intended to provide classical/quantum security strengths of 128/64, 160/80, 256/128, and 512/256 bits, respectively. Each category contains two concrete parameter sets: an **eff** variant and a **sec** variant. The **eff** variant uses a slightly smaller prime in order to improve efficiency, while the **sec** variant uses a slightly larger prime in order to provide a larger security margin. Accordingly, the eight parameter sets are denoted by **Level1-eff**, **Level1-sec**, **Level2-eff**, **Level2-sec**, **Level3-eff**, **Level3-sec**, **Level5-eff**, and **Level5-sec**. A parameter set consists of a targeted security parameter λ , a prime p and a hash function HASH. All other parameters can be deduced from p and the above specification; for convenience, we also list below the values of f , a and b such that $p = f \cdot 2^a \cdot 3^b - 1$.

Let H be any cryptographic hash function of output length $\lceil \log_2 3^b \rceil$ bits. Then HASH can be derived from H as in Algorithm 4.1. For correctness verification and preliminary performance testing, we instantiate HASH with SM3 by means of Algorithm 4.1. In the following, we write H_t for the first t bits of H , and $H^{\circ n} = H \circ \dots \circ H$ for H iterated n times.

Algorithm 4.1 HASH

Input: A message $msg \in \{0, 1\}^*$

Output: $chl \in [3^b]$

```

1:  $cnt \leftarrow 0$  // encoded as one byte
2: repeat  $chl \leftarrow H(msg || cnt)$ ,  $cnt \leftarrow cnt + 1$ 
3: until  $chl < 3^b$ 
4: return  $chl$ 

```

Level1-eff:

$$\lambda = 128$$

$$p = 2^{131} \cdot 3^{78} - 1$$

$$= 0x62d7f37f9815e5fca7ecc14ec3fa83c7ffffffffffffffffffffffffffff$$

HASH : instantiated with $SM3-512_{124} \circ SM3-512^{\circ 31}$

Level1-sec:

$$\lambda = 128$$

$$p = 2^{137} \cdot 3^{84} - 1$$

$$= 0x465e3d19960596fb1e8c4c9e5305185121ffffffffffffffffffffffffffff$$

HASH : instantiated with $SM3-512_{134}$

Level2-eff:

$$\lambda = 160$$

$$p = 61 \cdot 2^{161} \cdot 3^{96} - 1$$

$$= 0x87f825b5984d21e40b07b3ed88b9b99db034db79ffffffffffffffffffff$$

$$ffffffffffff$$

HASH : instantiated with $SM3-512_{153} \circ SM3-512^{\circ 255}$

Level2-sec:

$$\lambda = 160$$

$$p = 11 \cdot 2^{168} \cdot 3^{101} - 1$$

$$= 0xba310cd174e247321c380e630a924280fb63a8df0ffffffffffffffffffffffffffff$$

HASH : instantiated with SM3-512₁₆₁

Level3-eff:

$$\lambda = 256$$

$$p = 2^{263} \cdot 3^{156} - 1$$

$$= 0x4c54193130bb665f234a5c454b6e0020b8d8a1c1c552969b4c46ef90acec987ffff$$

HASH : instantiated with SM3-512₂₄₈ $\circ$ SM3-512^{o511}

Level3-sec:

$$\lambda = 256$$

$$p = 2^{264} \cdot 3^{163} - 1$$

$$= 0x51824e66e76a1e71d84f88487f6c9762f16455bc4c171194169440131ea92759aff$$

HASH : instantiated with SM3-512₂₅₉

Level5-eff:

$$\lambda = 512$$

$$p = 11 \cdot 2^{514} \cdot 3^{319} - 1$$

$$= 0x85aa094b24b141fc355ad1adc6eb497003fc8db5315de69aea7a411c54b015048e88$$

$$7d93a4d029344919b54f24e1f63be9a1c1509209a83644ea17826cb7963ffffffff$$

$$ff$$

$$ff$$

HASH : instantiated with SM3-1024₅₀₆ $\circ$ SM3-1024^{o127}

Level5-sec:

$$\lambda = 512$$

$$p = 17 \cdot 2^{536} \cdot 3^{334} - 1$$

$$= 0x2c2b13106aca6ca040134d002c46e09b2c26a9480ac240ef78c31b591dff81e70517$$

$$0e1fdcd28180a5e0759c1a94c2a05a2a0a2579388c6c5c2ba227f9e796050cec08ffff$$

$$ff$$

$$ff$$

HASH : instantiated with SM3-1024₅₃₀

Known answer test values

The KAT files for SQISign2D² are generated at runtime by executing the provided KAT program `kat_sig` in the directory of each algorithm instance. By default, the generated file is written to the `output` directory located alongside the executable, under the name `KAT_SIG_[AlgorithmInstance].txt`, where `[AlgorithmInstance]` is one of the eight submitted instances, e.g. `SQISign2DSquare-Level1-eff`. Since each algorithm instance supports both compressed and uncompressed modes through `config.h`, the submitted test-vector files are stored with an additional mode suffix, namely `KAT_SIG_[AlgorithmInstance]_compressed.txt` and `KAT_SIG_[AlgorithmInstance]_uncompressed.txt`. Each KAT file contains values for the secret keys (SK), public keys (PK), signatures (Sn), messages (M), and seeds (Seed).

Performance analysis

The submission package contains a reference implementation written in portable C and an optimized implementation targeting the 64-bit x86 architecture. Both implementations share the same high-level SQIsign2D² code base. The main difference lies in the finite-field arithmetic layer: the reference implementation uses portable C routines, whereas the optimized implementation uses x86-64 GNU assembly for the performance-critical field arithmetic routines.

6.1. The storage and transmission overhead

The storage and transmission overhead of SQIsign2D² is summarized in Table 6.1. We report the sizes of public key, secret key, and signature in bytes for both uncompressed and compressed versions.

TABLE 6.1. Parameter settings, public-key sizes, secret-key sizes, and signature sizes in uncompressed and compressed modes. All sizes are expressed in bytes.

Security Level	Prime	Public key		Secret key		Signature	
		Uncom.	Com.	Uncom.	Com.	Uncom.	Com.
Level1-eff	$p = 2^{131} \cdot 3^{78} - 1$	64	65	488	240	200	168
Level1-sec	$p = 2^{137} \cdot 3^{84} - 1$	68	69	521	257	212	178
Level2-eff	$p = 61 \cdot 2^{161} \cdot 3^{96} - 1$	80	81	610	298	248	208
Level2-sec	$p = 11 \cdot 2^{168} \cdot 3^{101} - 1$	84	85	643	315	260	218
Level3-eff	$p = 2^{263} \cdot 3^{156} - 1$	128	129	976	468	392	326
Level3-sec	$p = 2^{264} \cdot 3^{163} - 1$	132	133	1009	489	404	338
Level5-eff	$p = 11 \cdot 2^{514} \cdot 3^{319} - 1$	256	257	1952	936	776	648
Level5-sec	$p = 17 \cdot 2^{536} \cdot 3^{334} - 1$	268	269	2046	982	812	678

6.2. Reference implementation

The reference implementation is written in portable C. Please refer to the `README.txt` file in the submission package for compilation and test instructions.

6.3. Optimized implementation

The optimized implementation targets the 64-bit x86 architecture. It reuses the high-level code of the reference implementation, while replacing the performance-critical finite-field routines with x86-64 GNU assembly. The current assembly routines use `mulx`, `adcx`, and `adox`; therefore, the optimized implementation requires x86-64 processors supporting the BMI2 and ADX instruction-set extensions. Please refer to the `README.txt` file for compilation and test instructions.

6.4. Performance evaluation

Performance evaluation was performed on a single core of a 64-bit x86 PC platform equipped with an AMD Ryzen 9 9950X3D processor and 30.45 GB of memory. The measured average effective clock frequency was approximately 4.3 GHz. The operating system was Ubuntu 24.04 LTS and the code was built using GNU Make 4.3 and `gcc` 14.2.0. The reference implementation only relies on the basic x86-64 instruction set. The optimized implementation additionally uses BMI2 and ADX instructions, in particular `mulx`, `adcx`, and `adox`, in the handwritten assembly routines. GMP 6.3.0 was used as the external big-integer library. Turbo Boost and HyperThreading were turned off during benchmarking to get consistent timings. The results are shown in Table 6.2.

TABLE 6.2. SQIsign2D² performance of reference and optimized implementations in 10⁶ CPU cycles on an AMD Ryzen 9 9950X3D CPU. Results are averaged over 1,000 benchmark runs.

Version	Security Level	KeyGen		Sign		Verify	
		Uncom.	Com.	Uncom.	Com.	Uncom.	Com.
Reference Implementation	Level1-eff	12.41	21.94	69.09	73.64	14.77	15.42
	Level1-sec	18.17	34.37	102.92	113.17	20.76	23.42
	Level2-eff	21.29	39.76	124.46	134.49	25.70	29.04
	Level2-sec	30.63	59.41	177.87	196.17	36.92	42.69
	Level3-eff	69.71	133.25	416.83	457.45	103.44	106.86
	Level3-sec	101.01	195.11	582.72	636.89	121.71	136.72
	Level5-eff	801.44	1604.31	4825.40	5285.58	1036.21	1180.68
	Level5-sec	1015.75	2008.16	6006.02	6447.71	1285.36	1459.48
Optimized Implementation	Level1-eff	7.63	12.94	39.35	44.94	7.21	8.11
	Level1-sec	10.36	18.53	55.90	62.96	10.37	11.71
	Level2-eff	12.96	22.23	75.53	79.99	14.14	15.85
	Level2-sec	16.45	29.98	94.61	102.01	17.18	19.87
	Level3-eff	40.72	72.96	246.76	267.61	60.40	62.25
	Level3-sec	53.19	97.32	308.11	331.55	56.19	63.12
	Level5-eff	349.72	644.23	2149.16	2311.73	382.07	434.90
	Level5-sec	445.49	786.89	2519.09	2664.77	445.42	507.26

Implementation details

This chapter gives the implementation details of the algorithms used to realize SQIsign2D² in practice.

The submission package includes support for all field arithmetic operations described in Section 2.1. In addition to the reference implementation, we provide an optimized implementation designed for 64-bit x86 architectures. Compared with the reference implementation, these platform-specific versions differ only in the field arithmetic layer. More precisely, pseudo-random number generation is based on SM3-DRNG, and:

- In the reference implementation, arithmetic over $\mathbb{F}_p$ and $\mathbb{F}_{p^2}$ is implemented in plain C.
- In the 64-bit x86 optimized implementation, the most performance-critical routines over $\mathbb{F}_p$ and $\mathbb{F}_{p^2}$, including multiplication and squaring, are implemented using prime-specialized assembly routines; see Section 7.1 for details.

7.1. Finite field arithmetic

Efficient arithmetic over $\mathbb{F}_p$ and $\mathbb{F}_{p^2}$ is one of the main performance-critical components in the implementation of SQIsign2D². The curve arithmetic, isogeny evaluation, challenge reconstruction, and verification procedures all rely heavily on repeated field multiplications and squarings. The finite-field layer is therefore implemented in Montgomery representation, with prime-specific multiplication and reduction routines.

The primes used by SQIsign2D² have the structured form

$$p = f \cdot 2^b \cdot 3^a - 1.$$

In particular, the low bits of p are all equal to one, and for the word radix $r = 2^{64}$ we have

$$p \equiv -1 \pmod{r}.$$

This property is useful for Montgomery reduction. In a standard word-by-word Montgomery multiplication, the reduction coefficient for the current low word is

$$q = t_0 \cdot (-p^{-1} \bmod r) \bmod r.$$

Since $p \equiv -1 \pmod{r}$, we have $-p^{-1} \equiv 1 \pmod{r}$, and hence

$$q = t_0.$$

Thus the reduction coefficient is simply the current low word of the accumulator. No multiplication by a Montgomery constant is needed in the low word. Moreover, because several low

limbs of p are equal to $2^{64} - 1$, the corresponding products by the low limbs of the modulus can be implemented by add/subtract/carry chains. Only the nontrivial higher limbs of p need to appear as explicit constants in the reduction schedule.

Schematically, the multiplication routine follows the word-by-word Montgomery structure shown in Algorithm 7.1. The actual implementation is fully specialized to the fixed modulus and fixed limb count.

Algorithm 7.1 Prime-specialized Montgomery multiplication

Input: $a, b \in [0, p)$ in Montgomery representation; word radix $r = 2^{64}$; Montgomery radix $R = r^n$; modulus p satisfying $p \equiv -1 \pmod{r}$

Output: $c = abR^{-1} \bmod p$

```

1:  $t \leftarrow 0$ 
2: for  $i$  from 0 to  $n - 1$  do
3:    $t \leftarrow t + a_i b$ 
4:    $q \leftarrow t \bmod r$ 
5:    $t \leftarrow (t + qp)/r$ 
6: end for
7: if  $t \geq p$  then
8:    $t \leftarrow t - p$ 
9: end if
10: return  $t$ 

```

The key point is line 4, the special form of the modulus makes the Montgomery coefficient equal to the low accumulator word. In the concrete C and assembly implementations, line 5 is not executed as a generic multiplication by p . Instead, it is expanded into a straight-line reduction schedule using the known limb pattern of the modulus. This gives a compact and efficient implementation while preserving the standard Montgomery-domain interface.

7.1.1. Reference implementation

The reference implementation is written in portable C. It keeps the general Fiat-Crypto style for most base-field routines, including carry and borrow primitives, conditional moves, addition, subtraction, negation, encoding, decoding, and conversion to and from Montgomery representation. The Fiat-Crypto synthesis framework and its correctness methodology are described in [EPG+20], with further implementation details for finite-field backends in [Phi18].

The main performance-critical exception is multiplication. Instead of using a generic generated multiplication routine, the reference implementation uses a hand-written prime-specialized Montgomery multiplication following the structure above. Squaring calls the same multiplication routine with equal operands, so it benefits from the same reduction strategy.

On compilers supporting native 128-bit integer arithmetic, the C backend uses `uint64_t` limbs and `unsigned __int128` temporaries for $64 \times 64 \rightarrow 128$ products and carry propagation. For platforms without `unsigned __int128`, the implementation provides a fully portable fallback that decomposes each 64-bit product into 32-bit partial products and then performs the same word-by-word Montgomery reduction. In this way, the reference implementation remains pure C while still providing efficient arithmetic on common 64-bit compilers.

At the $\mathbb{F}_{p^2}$ level, elements are represented as

$$a = a_0 + ia_1, \quad i^2 = -1.$$

The reference implementation builds $\mathbb{F}_{p^2}$ arithmetic from the underlying $\mathbb{F}_p$ primitives. Multiplication uses the standard Karatsuba formula

$$(a_0 + ia_1)(b_0 + ib_1) = (a_0b_0 - a_1b_1) + i((a_0 + a_1)(b_0 + b_1) - a_0b_0 - a_1b_1),$$

which requires three base-field multiplications. Squaring uses

$$(a_0 + ia_1)^2 = (a_0 + a_1)(a_0 - a_1) + i(2a_0a_1),$$

again reducing the number of base-field multiplications compared with a direct expansion.

7.1.2. x86-64 optimized implementation

The x86-64 optimized implementation keeps the same public finite-field API as the reference implementation, but replaces the most expensive base-field and $\mathbb{F}_{p^2}$ operations with handwritten assembly routines.

The base-field assembly routines implement the same prime-specialized Montgomery multiplication strategy as the C backend. Since the modulus and limb count are fixed at compile time, the multiplication and reduction schedule can be written as straight-line code with fixed constants and fixed carry chains. This avoids the overhead of generic multi-precision loops and exposes more instruction-level parallelism to the processor.

At the instruction level, the optimized backend uses x86-64 integer multiplication and carry-propagation instructions to implement the multiply-accumulate chains. In particular, the multiplication columns and Montgomery reduction steps are arranged so that independent carry chains can be scheduled efficiently. The all-one low limbs of the modulus are handled by carry chains, while only the nontrivial high limbs are multiplied explicitly. The optimized implementation also provides dedicated assembly entry points for $\mathbb{F}_{p^2}$ multiplication and squaring. This avoids repeatedly materializing temporary $\mathbb{F}_p$ values through the generic C composition layer. For $\mathbb{F}_{p^2}$ multiplication, the assembly routine computes the same Karatsuba formula as the reference implementation, but fuses the surrounding additions, subtractions, and base-field multiplications into a lower-overhead routine. For $\mathbb{F}_{p^2}$ squaring, the assembly routine similarly implements the formula

$$(a_0 + ia_1)^2 = (a_0 + a_1)(a_0 - a_1) + i(2a_0a_1)$$

with reduced call overhead and fewer intermediate stores.

This optimization is especially important for verification, where the dominant cost comes from long sequences of $\mathbb{F}_{p^2}$ multiplications and squarings in curve and isogeny computations. By specializing the base-field Montgomery reduction and by dispatching $\mathbb{F}_{p^2}$ multiplication and squaring directly to assembly, the optimized backend accelerates the main finite-field bottleneck while leaving the higher-level algorithm unchanged.

7.2. Elliptic curve arithmetic

We describe the pseudocode for elliptic-curve arithmetic on Montgomery curves using both projective x -only coordinates and full projective coordinates, which are used throughout scalar multiplication and several auxiliary routines.

7.2.1. Projective x -only coordinates

We consider the Montgomery curve over $\mathbb{F}_{p^2}$, where $B(A^2 - 4) \neq 0$:

$$BY^2Z = X^3 + AX^2Z + XZ^2.$$

A point of this curve can be represented by $[X : Y : Z]$, where we identify $[X : Y : Z]$ with $[\lambda X : \lambda Y : \lambda Z]$ for all $\lambda \in \mathbb{F}_{p^2}^*$. When $Z \neq 0$, the projective point $[X : Y : Z]$ corresponds to an affine point $(x, y) = (\frac{X}{Z}, \frac{Y}{Z})$. And $Z = 0$ corresponds to the infy point $0_{E_{A,B}}$. So we can also use the affine form introduced in Section 2.2.1:

$$By^2 = x^3 + Ax^2 + x.$$

For convenience, we write $E_{A,B}$ for the Montgomery curve with coefficients A and B , whether represented in affine or projective form. For fixed X, Z , there are at most two points on $E_{A,B}$ with these coordinates, namely $[X : Y : Z]$ and $[X : -Y : Z]$. Therefore, it is sufficient in many routines to represent a point by its x -only projective coordinates $[X : Z]$.

The coefficient B affects only the y -coordinate, so it is omitted from the x -only arithmetic. In the implementation, the Montgomery coefficient A is represented projectively by the pair $(A_{\text{Proj}} : C_{\text{Proj}})$ with $A = A_{\text{Proj}}/C_{\text{Proj}}$.

For differential doubling and tripling formulas, it is convenient to additionally use the representation $(A_{24} : C_{24}) = (A_{\text{Proj}} + 2C_{\text{Proj}} : 4C_{\text{Proj}})$ and $(A_{24}^+ : A_{24}^-) = (A_{\text{Proj}} + 2C_{\text{Proj}} : A_{\text{Proj}} - 2C_{\text{Proj}})$. If the curve has already been normalized to $C_{\text{proj}} = 1$, this becomes $(A_{24} : C_{24}) = (A + 2 : 4)$ and $(A_{24}^+ : A_{24}^-) = (A + 2 : A - 2)$. The implementation uses the following basic operations in x -only coordinates. Given a point $P = (X_P : Z_P)$, the doubling routine computes the projective representation of $x([2]P)$. Given points P, Q , and a projective representative of the difference point $P - Q$, the differential addition routine returns a projective representative of $x(P + Q)$. Combining both operations yields the standard double-and-add step used in Montgomery ladder routines. These formulas are specified in [xDBL](#)(Algorithms 7.2)–[LadderBisclar](#)(Algorithm 7.8).

7.2.2. Curve arithmetic

We now list the basic arithmetic routines on Montgomery curves in projective x -only coordinates. These routines form the core of scalar multiplication, differential addition chains, and several auxiliary procedures used elsewhere in the implementation.

- [xDBL](#)(Algorithm 7.2) takes as input a projective point $P = (X_P : Z_P)$ together with the projective constants $(A_{24} : C_{24})$, and returns a projective representative of $x([2]P)$.
- [xTPL](#)(Algorithm 7.3) takes as input a projective point $P = (X_P : Z_P)$ together with the projective constants $(A_{24}^+ : A_{24}^-)$, and returns a projective representative of $x([3]P)$.
- [xADD](#)(Algorithm 7.4) takes as input the projective representatives of P, Q , and $P - Q$, and outputs a projective representative of $x(P + Q)$. Here, as throughout this section, the third x -only input is the *difference* point $P - Q$.
- [xDBLADD](#)(Algorithm 7.5) combines one doubling and one differential addition into a single routine. Take as input the projective x -coordinates of the points $P, Q, P - Q$ and $(A_{24} : C_{24})$, returning projective representatives of both $x([2]P)$ and $x(P + Q)$.
- [Ladder](#)(Algorithm 7.6) computes scalar multiplication from the binary expansion of the scalar by iterating the combined double-and-add step.
- [Ladder3pt](#)(Algorithm 7.7)(LADDER3PT) is the usual three-point Montgomery ladder: given projective representatives of P, Q and $P - Q$, together with an integer m , it returns a projective representative of $x(P + [m]Q)$.
- [LadderBisclar](#)(Algorithm 7.8) takes as input the projective x -coordinates of the points $P, Q, P - Q$, $(A_{24} : C_{24})$ and two positive integers m, n , and returns the projective representative of $x([m]P + [n]Q)$.
- [CurveStandardization](#)(Algorithm 7.9) takes as input $(A : C)$ and a list of points **pts**, and computes a standard montgomery curve in the isomorphic class of the curve represented by input $(A : C)$, and transforms **pts** by the isomorphism.

Remark 7.1. When $(A/C)^2$ equals 0 or 3 or $9/2$, [CurveStandardization](#) fails. Under honest input generation, the procedure may fail with negligible probability.

Algorithm 7.2 xDBL($P, (A_{24} : C_{24})$)

Input: A projective point $P = (X_P : Z_P)$ and the Montgomery coefficient $(A_{24} : C_{24})$ of the curve E .**Output:** The projective point $[2]P = (X_{2P} : Z_{2P})$.

```

1:  $t_0 \leftarrow X_P + Z_P$ 
2:  $t_0 \leftarrow t_0^2$ 
3:  $t_1 \leftarrow X_P - Z_P$ 
4:  $t_1 \leftarrow t_1^2$ 
5:  $t_2 \leftarrow t_0 - t_1$ 
6:  $t_1 \leftarrow t_1 \cdot C_{24}$ 
7:  $X_{2P} \leftarrow t_0 \cdot t_1$ 
8:  $t_0 \leftarrow t_2 \cdot A_{24}$ 
9:  $t_0 \leftarrow t_0 + t_1$ 
10:  $Z_{2P} \leftarrow t_0 \cdot t_2$ 
11: return  $[2]P = (X_{2P} : Z_{2P})$  // Cost: 2S + 4M + 4a

```

Algorithm 7.3 xTPL($P, (A_{24}^+ : A_{24}^-)$)

Input: A projective point $P = (X_P : Z_P)$ and the Montgomery coefficient $(A_{24}^+ : A_{24}^-)$ of the curve E .**Output:** The projective point $[3]P = (X_{[3]P} : Z_{[3]P})$.

```

1:  $t_0 \leftarrow X_P - Z_P$ 
2:  $t_2 \leftarrow t_0^2$ 
3:  $t_1 \leftarrow X_P + Z_P$ 
4:  $t_3 \leftarrow t_1^2$ 
5:  $t_4 \leftarrow t_1 + t_0$ 
6:  $t_0 \leftarrow t_1 - t_0$ 
7:  $t_1 \leftarrow t_4^2$ 
8:  $t_1 \leftarrow t_1 - t_3$ 
9:  $t_1 \leftarrow t_1 - t_2$ 
10:  $t_5 \leftarrow t_3 \cdot A_{24}^+$ 
11:  $t_3 \leftarrow t_3 \cdot t_5$ 
12:  $t_6 \leftarrow t_2 \cdot A_{24}^-$ 
13:  $t_2 \leftarrow t_2 \cdot t_6$ 
14:  $t_3 \leftarrow t_2 - t_3$ 
15:  $t_2 \leftarrow t_5 - t_6$ 
16:  $t_1 \leftarrow t_1 \cdot t_2$ 
17:  $t_2 \leftarrow t_3 + t_1$ 
18:  $t_2 \leftarrow t_2^2$ 
19:  $X_{[3]P} \leftarrow t_2 \cdot t_4$ 
20:  $t_1 \leftarrow t_3 - t_1$ 
21:  $t_1 \leftarrow t_1^2$ 
22:  $Z_{[3]P} \leftarrow t_1 \cdot t_0$ 
23: return  $[3]P = (X_{[3]P} : Z_{[3]P})$  // Cost: 5S + 7M + 10a

```

Algorithm 7.4 $\text{xADD}(P, Q, P - Q)$

Input: Projective points $P = (X_P : Z_P), Q = (X_Q : Z_Q), P - Q = (X_{P-Q} : Z_{P-Q})$.**Output:** The projective sum $P + Q = (X_{P+Q} : Z_{P+Q})$.

```

1:  $t_0 \leftarrow X_P + Z_P$ 
2:  $t_1 \leftarrow X_P - Z_P$ 
3:  $t_2 \leftarrow X_Q + Z_Q$ 
4:  $t_3 \leftarrow X_Q - Z_Q$ 
5:  $t_0 \leftarrow t_0 \cdot t_3$ 
6:  $t_1 \leftarrow t_1 \cdot t_2$ 
7:  $t_2 \leftarrow t_0 + t_1$ 
8:  $t_3 \leftarrow t_0 - t_1$ 
9:  $t_2 \leftarrow t_2^2$ 
10:  $t_3 \leftarrow t_3^2$ 
11:  $X_{P+Q} \leftarrow Z_{P-Q} \cdot t_2$ 
12:  $Z_{P+Q} \leftarrow X_{P-Q} \cdot t_3$ 
13: return  $P + Q = (X_{P+Q} : Z_{P+Q})$ 

```

// Cost: 2S + 4M + 6a

Algorithm 7.5 $\text{xDBLADD}(P, Q, P - Q, (A_{24} : C_{24}))$

Input: Projective points $P = (X_P : Z_P), Q = (X_Q : Z_Q), P - Q = (X_{P-Q} : Z_{P-Q})$, and the Montgomery coefficient $(A_{24} : C_{24})$ of the curve E .**Output:** Projective points $[2]P = (X_{2P} : Z_{2P})$ and $P + Q = (X_{P+Q} : Z_{P+Q})$.

```

1:  $t_0 \leftarrow X_P + Z_P$ 
2:  $t_1 \leftarrow X_P - Z_P$ 
3:  $X_{2P} \leftarrow t_0^2$ 
4:  $t_2 \leftarrow X_Q - Z_Q$ 
5:  $X_{P+Q} \leftarrow X_Q + Z_Q$ 
6:  $t_0 \leftarrow t_0 \cdot t_2$ 
7:  $Z_{2P} \leftarrow t_1^2$ 
8:  $t_1 \leftarrow t_1 \cdot X_{P+Q}$ 
9:  $t_2 \leftarrow X_{2P} - Z_{2P}$ 
10:  $Z_{2P} \leftarrow Z_{2P} \cdot C_{24}$ 
11:  $X_{2P} \leftarrow X_{2P} \cdot Z_{2P}$ 
12:  $X_{P+Q} \leftarrow A_{24} \cdot t_2$ 
13:  $Z_{P+Q} \leftarrow t_0 - t_1$ 
14:  $Z_{2P} \leftarrow Z_{2P} + X_{P+Q}$ 
15:  $X_{P+Q} \leftarrow t_0 + t_1$ 
16:  $Z_{2P} \leftarrow Z_{2P} \cdot t_2$ 
17:  $Z_{P+Q} \leftarrow Z_{P+Q}^2$ 
18:  $X_{P+Q} \leftarrow X_{P+Q}^2$ 
19:  $Z_{P+Q} \leftarrow Z_{P+Q} \cdot X_{P-Q}$ 
20:  $X_{P+Q} \leftarrow X_{P+Q} \cdot Z_{P-Q}$ 
21: return  $([2]P, P + Q) = ((X_{2P} : Z_{2P}), (X_{P+Q} : Z_{P+Q}))$ 

```

4S + 8M + 8a

Algorithm 7.6 Ladder(P, E, m)

Input: A projective point $P = (X_P : Z_P)$, the Montgomery coefficient $(A_{24} : C_{24})$ of the curve E , and a positive scalar m with binary representation $m = (m_{k-1}, \dots, m_0)_2$.

Output: The projective point $[m]P = (X_{[m]P} : Z_{[m]P})$.

```

1:  $((X_0 : Z_0), (X_1 : Z_1)) \leftarrow ((1 : 0), (X_P : Z_P))$ 
2: for  $i$  from  $k - 1$  down to 0 do
3:   if  $m_i = 1$  then
4:      $((X_1, Z_1), (X_0, Z_0)) \leftarrow \text{xDBLADD}((X_1 : Z_1), (X_0 : Z_0), (X_P : Z_P), (A_{24} : C_{24}))$ 
5:   else
6:      $((X_0 : Z_0), (X_1 : Z_1)) \leftarrow \text{xDBLADD}((X_0 : Z_0), (X_1 : Z_1), (X_P : Z_P), (A_{24} : C_{24}))$ 
7:   end if
8: end for
9:  $(X_{[m]P} : Z_{[m]P}) \leftarrow (X_0 : Z_0)$ 
10: return  $[m]P = (X_{[m]P} : Z_{[m]P})$ 

```

Algorithm 7.7 Ladder3pt($P, Q, P - Q, (A_{24} : C_{24}), m$)

Input: Projective points $P = (X_P : Z_P)$, $Q = (X_Q : Z_Q)$, $P - Q = (X_{P-Q} : Z_{P-Q})$, the Montgomery coefficient $(A_{24} : C_{24})$ of the curve E , and a positive scalar m with binary representation $m = (m_{k-1}, \dots, m_0)_2$.

Output: The projective point $P + [m]Q = (X_{P+[m]Q} : Z_{P+[m]Q})$.

```

1:  $((X_0 : Z_0), (X_1 : Z_1), (X_2 : Z_2)) \leftarrow ((X_P : Z_P), (X_Q : Z_Q), (X_{P-Q} : Z_{P-Q}))$ 
2: for  $i$  from 0 up to  $k - 1$  do
3:   if  $m_i = 1$  then
4:      $((X_0, Z_0), (X_1, Z_1)) \leftarrow \text{xDBLADD}((X_0 : Z_0), (X_1 : Z_1), (X_2 : Z_2), (A_{24} : C_{24}))$ 
5:   else
6:      $((X_0 : Z_0), (X_2 : Z_2)) \leftarrow \text{xDBLADD}((X_0 : Z_0), (X_2 : Z_2), (X_1 : Z_1), (A_{24} : C_{24}))$ 
7:   end if
8: end for
9:  $(X_{P+[m]Q} : Z_{P+[m]Q}) \leftarrow (X_1 : Z_1)$ 
10: return  $P + [m]Q = (X_{P+[m]Q} : Z_{P+[m]Q})$ 

```

Algorithm 7.8 LadderBisclar($P, Q, P - Q, (A_{24} : C_{24}), m, n$)

Input: Projective points $P = (X_P : Z_P)$, $Q = (X_Q : Z_Q)$, $P - Q = (X_{P-Q} : Z_{P-Q})$, the Montgomery coefficient $(A_{24} : C_{24})$ of the curve E , and positive scalars m, n with binary representations $m = (m_{k-1}, \dots, m_0)_2$ and $n = (n_{k-1}, \dots, n_0)_2$, resp.

Output: The projective point $[m]P + [n]Q = (X_{[m]P+[n]Q} : Z_{[m]P+[n]Q})$.

```

1: Recoding stage:
2: if  $m$  is even and  $n$  is odd then
3:    $\sigma = (\sigma_0, \sigma_1) \leftarrow (1, 0)$ 
4: else
5:    $\sigma = (\sigma_0, \sigma_1) \leftarrow (0, 1)$ 
6: end if
7:  $m' \leftarrow m, n' \leftarrow n$ 
8: if  $m$  is even then
9:    $m' \leftarrow m' - 1$ 
10: end if
11: if  $n$  is even then
12:    $n' \leftarrow n' - 1$ 
13: end if
14: Set  $b = (b_0, b_1)$ , where  $b_0 \leftarrow (0, m'_{k-1}, \dots, m'_0)$  and  $b_1 \leftarrow (0, n'_{k-1}, \dots, n'_0)$ 
15: for  $i$  from 0 up to  $k - 1$  do
16:    $r_{2i} \leftarrow b_{\sigma_0, i} \oplus b_{\sigma_0, i+1}$ 
17:    $r_{2i+1} \leftarrow b_{\sigma_1, i} \oplus b_{\sigma_1, i+1}$ 
18:   if  $r_{2i+1} = 1$  then
19:      $t \leftarrow \sigma_0$ 
20:      $\sigma_0 \leftarrow \sigma_1$ 
21:      $\sigma_1 \leftarrow t$ 
22:   end if
23: end for
24: Evaluation stage:
25:  $R_0 \leftarrow (1 : 0)$ 
26:  $T = (T_0, T_1) \leftarrow (P, Q)$ 
27:  $R_1 \leftarrow T_{\sigma_0}$ 
28:  $R_2 \leftarrow T_{\sigma_0+1 \pmod{2}}$ 
29:  $D_1 \leftarrow R_1$ 
30:  $D_2 \leftarrow R_2$ 
31:  $R_2 \leftarrow \text{xADD}(R_1, R_2, P - Q)$ 
32:  $F_1 \leftarrow R_2$ 
33:  $F_2 \leftarrow P - Q$ 
34: for  $i$  from  $k - 1$  down to 0 do
35:    $h \leftarrow r_{2i} + r_{2i+1}$ 
36:    $T_0 \leftarrow R_h \pmod{2}$ 
37:    $T = (T_0, T_1) \leftarrow (T_0, R_2)$ 
38:    $T_0 \leftarrow \text{xDBL}(T_{\lfloor h/2 \rfloor}, (A_{24} : C_{24}))$ 
39:    $T_1 \leftarrow R_{r_{2i+1}}$ 
40:    $T_2 \leftarrow R_{r_{2i+1}+1}$ 
41:   if  $r_{2i+1} = 1$  then
42:      $\text{TMP} \leftarrow D_1$ 
43:      $D_1 \leftarrow D_2$ 
44:      $D_2 \leftarrow \text{TMP}$ 
45:   end if
46:    $T_1 \leftarrow \text{xADD}(T_1, T_2, D_1)$ 
47:    $T_2 \leftarrow \text{xADD}(R_0, R_2, F_1)$ 
48:   if  $h \pmod{2} = 1$  then
49:      $\text{TMP} \leftarrow F_1$ 
50:      $F_1 \leftarrow F_2$ 
51:      $F_2 \leftarrow \text{TMP}$ 
52:   end if
53:    $R_0 \leftarrow T_0$ 
54:    $R_1 \leftarrow T_1$ 
55:    $R_2 \leftarrow T_2$ 
56: end for
57:  $(X_{[m]P+[n]Q} : Z_{[m]P+[n]Q}) \leftarrow R_{(m \pmod{2} \oplus 1) + (n \pmod{2} \oplus 1)}$ 
58: return  $[m]P + [n]Q = (X_{[m]P+[n]Q} : Z_{[m]P+[n]Q})$ 

```

Algorithm 7.9 CurveStandardization(E, pts)**Input:** Montgomery coefficient ($A : C$) and a list of points to be evaluated pts .**Output:** A standard montgomery curve ($A' : C'$) isomorphic to E , and the image of points in pts under the isomorphism.

```

1:  $u \leftarrow -(A^2 - 3C^2)/(2A^3 - 9AC^2)$ 
2:  $x_0 \leftarrow u \cdot 2A$ 
3:  $x_1 \leftarrow u \cdot (-A + 3\sqrt{A^2 - 4C^2})/6$ 
4:  $x_2 \leftarrow u \cdot (-A - 3\sqrt{A^2 - 4C^2})/6$ 
5: if  $x_0 > x_1$  and  $x_0 > x_2$  then
6:    $A' \leftarrow A, C' \leftarrow C$ 
7:    $N_x \leftarrow 1, N_z \leftarrow 0, D \leftarrow 1$ 
8: else if  $x_1 > x_0$  and  $x_1 > x_2$  then
9:    $A' \leftarrow -A + 3\sqrt{A^2 - 4C^2}, C' \leftarrow \sqrt{2(A^2 - 4C^2 - A\sqrt{A^2 - 4C^2})}$ 
10:   $N_x \leftarrow 2C, N_z \leftarrow -A + \sqrt{A^2 - 4C^2}, D \leftarrow \sqrt{2(A^2 - 4C^2 - A\sqrt{A^2 - 4C^2})}$ 
11: else
12:    $A' \leftarrow -A - 3\sqrt{A^2 - 4C^2}, C' \leftarrow \sqrt{2(A^2 - 4C^2 + A\sqrt{A^2 - 4C^2})}$ 
13:    $N_x \leftarrow 2C, N_z \leftarrow -A - \sqrt{A^2 - 4C^2}, D \leftarrow \sqrt{2(A^2 - 4C^2 + A\sqrt{A^2 - 4C^2})}$ 
14: end if
15: return ( $A' : C'$ ) and  $[(N_x X - N_z Z : DZ) \mid (X : Z) \in \text{pts}]$ 

```

7.2.3. Projective point difference

When computing the difference of two points using x -only coordinates, we need to make an arbitrary choice of sign due to Equation 2.8. To obtain the same basis during signing and verification, it is necessary to choose one deterministically. Inevitably, we must consider how to represent the square root with a projective coordinate during computation. Thus, we can define

$$\sqrt{(x : z)} := (\sqrt{xz^3\bar{z}^4} : z^2\bar{z}^2).$$

Obviously, this definition is projectively invariant due to the fact that our choice of square root over $\mathbb{F}_{p^2}$ satisfies the special property that $\sqrt{\lambda^4 x} = \lambda^2 \sqrt{x}$ for all $x \in \mathbb{F}_{p^2}$, $\lambda \in \mathbb{F}_p$.

ProjectiveDifference (Algorithm 7.10) implements Equation 2.8 for x_{P+Q} projectively using this principle.

Algorithm 7.10 ProjectiveDifference($P, Q, (A : C)$)**Input:** Projective points $P = (X_P : Z_P)$ and $Q = (X_Q : Z_Q)$ and the Montgomery coefficient ($A : C$).**Output:** A deterministic x -coordinate x_{PQ} , either x_{P-Q} or x_{P+Q} .

```

1:  $B_{XX} \leftarrow C \cdot (X_P X_Q - Z_P Z_Q)^2$ 
2:  $B_{XZ} \leftarrow C \cdot (X_P X_Q + Z_P Z_Q)(X_P Z_Q + Z_P X_Q) + 2A X_P X_Q Z_P Z_Q$ 
3:  $B_{ZZ} \leftarrow C \cdot (X_P Z_Q - Z_P X_Q)^2$ 
4:  $\gamma \leftarrow C \cdot (\bar{C} \cdot \bar{Z}_P \cdot \bar{Z}_Q)^2$ 
5:  $B_{XX} \leftarrow \gamma \cdot B_{XX}, B_{XZ} \leftarrow \gamma \cdot B_{XZ}, B_{ZZ} \leftarrow \gamma \cdot B_{ZZ}$ 
6:  $\delta \leftarrow \text{SquareRoot}(B_{XZ}^2 - B_{XX} B_{ZZ})$ 
7:  $X_{PQ} \leftarrow \delta + B_{XZ}, Z_{PQ} \leftarrow B_{ZZ}$ 
8: return  $x_{PQ} = (X_{PQ} : Z_{PQ})$ 

```

7.2.4. Jacobian coordinates

In our implementation, we sometimes need full projective coordinate arithmetic rather than just x -only, as in Section 7.5.5. In such cases, we define our arithmetic using Jacobian coordinates for efficiency reasons, in which a point P over the curve E is given by coordinates $(X : Y : Z)$

satisfying

$$BY^2 = X^3 + AX^2Z^2 + XZ^4$$

with the point $(0 : 1 : 0)$ at infinity. Instead of the standard projective equivalence $(X : Y : Z) \sim (\lambda X : \lambda Y : \lambda Z)$ for any $\lambda \in \mathbb{F}_q^*$, the Jacobian equivalence is defined as $(X : Y : Z) \sim (\lambda^2 X : \lambda^3 Y : \lambda Z)$ for any $\lambda \in \mathbb{F}_q^*$. The transformation between Montgomery and Jacobian coordinates is defined as follows:

- A point P in Jacobian coordinates $(X_J : Y_J : Z_J)$ corresponds to Montgomery coordinates

$$(X_M : Y_M : Z_M) = (X_J : Y_J/Z_J : Z_J^2).$$

- Conversely, a point P in Montgomery coordinates $(X_M : Y_M : Z_M)$ has Jacobian representation

$$(X_J : Y_J : Z_J) = (X_M - AZ_M^2/3, Y_M, Z_M).$$

The basic operations for arithmetic in Jacobian coordinates are as follows:

- **DBL** (Algorithm 7.11) takes as input the Jacobian coordinates of a point P and the projective curve parameters $(A_{24} : C_{24})$, and outputs the Jacobian coordinates of the point $[2]P$.
- **ADD** (Algorithm 7.12) takes as input the Jacobian coordinates of two points P, Q and the projective curve parameters $(A_{24} : C_{24})$, and outputs the Jacobian coordinates of the point $P + Q$.
- **ADDComponents** (Algorithm 7.13) takes as input the Jacobian coordinates of distinct points P, Q and the projective curve parameters $(A_{24} : C_{24})$, and outputs u, v, w with $P + Q = (u - v : w)$ and $P - Q = (u + v : w)$.

Algorithm 7.11 $\text{DBL}(P, (A_{24} : C_{24}))$

Input: Jacobian point $P = (X_P : Y_P : Z_P)$ and the Montgomery coefficient $(A_{24} : C_{24})$ of the curve E .

Output: The Jacobian point $[2]P = (X_{[2]P} : Y_{[2]P} : Z_{[2]P})$.

```

1:  $t_0 \leftarrow X_P^2$ 
2:  $t_1 \leftarrow t_0 + t_0$ 
3:  $t_0 \leftarrow t_0 + t_1$ 
4:  $t_1 \leftarrow Z_P^2$ 
5:  $t_2 \leftarrow X_P \cdot A_{24}$ 
6:  $t_2 \leftarrow t_2 + t_2$ 
7:  $t_2 \leftarrow t_1 + t_2$ 
8:  $t_2 \leftarrow t_1 \cdot t_2$ 
9:  $t_2 \leftarrow t_0 + t_2$ 
10:  $Z_{[2]P} \leftarrow Y_P \cdot Z_P$ 
11:  $Z_{[2]P} \leftarrow Z_{[2]P} + Z_{[2]P}$ 
12:  $t_0 \leftarrow Z_{[2]P}^2$ 
13:  $t_0 \leftarrow t_0 \cdot A_{24}$ 
14:  $t_1 \leftarrow Y_P^2$ 
15:  $t_1 \leftarrow t_1 + t_1$ 
16:  $t_3 \leftarrow x_P \cdot x_P$ 
17:  $t_3 \leftarrow t_1 \cdot t_3$ 
18:  $X_{[2]P} \leftarrow t_2^2$ 
19:  $X_{[2]P} \leftarrow X_{[2]P} - t_0$ 
20:  $X_{[2]P} \leftarrow X_{[2]P} - t_3$ 
21:  $X_{[2]P} \leftarrow X_{[2]P} - t_3$ 
22:  $Y_{[2]P} \leftarrow t_3 - X_{[2]P}$ 
23:  $Y_{[2]P} \leftarrow Y_{[2]P} \cdot t_2$ 
24:  $t_1 \leftarrow t_1^2$ 
25:  $Y_{[2]P} \leftarrow Y_{[2]P} - t_1$ 
26:  $Y_{[2]P} \leftarrow Y_{[2]P} - t_1$ 
27: return  $[2]P = (X_{[2]P} : Y_{[2]P} : Z_{[2]P})$ 

```

// Cost: 6S + 6M + 14a

Algorithm 7.12 $\text{ADD}(P, Q, (A_{24} : C_{24}))$

Input: Two Jacobian points $P = (X_P : Y_P : Z_P)$, $Q = (X_Q : Y_Q : Z_Q)$, and the Montgomery coefficient $(A_{24} : C_{24})$ of the curve E .

Output: The Jacobian point $P + Q = (X_{P+Q} : Y_{P+Q} : Z_{P+Q})$.

```

1:  $\text{ctl}_1 \leftarrow (P = 0)$ 
2:  $\text{ctl}_2 \leftarrow (Q = 0)$ 
3:  $t_0 \leftarrow Z_P^2$ 
4:  $t_1 \leftarrow Z_Q^2$ 
5:  $v_1 \leftarrow t_1 \cdot Z_Q$ 
6:  $t_2 \leftarrow t_0 \cdot Z_P$ 
7:  $v_1 \leftarrow v_1 \cdot Y_P$ 
8:  $t_2 \leftarrow t_2 \cdot Y_Q$ 
9:  $dy \leftarrow t_2 - v_1$ 
10:  $u_2 \leftarrow t_0 \cdot X_Q$ 
11:  $u_1 \leftarrow t_1 \cdot X_P$ 
12:  $dx \leftarrow u_2 - u_1$ 
13:  $t_1 \leftarrow Y_P + Y_Q$ 
14:  $t_2 \leftarrow A_{24} \cdot X_P$ 
15:  $t_2 \leftarrow t_2 + t_2$ 
16:  $t_2 \leftarrow t_2 + t_0$ 
17:  $t_2 \leftarrow t_2 \cdot t_0$ 
18:  $t_0 \leftarrow X_P^2$ 
19:  $t_2 \leftarrow t_2 + t_0$ 
20:  $t_2 \leftarrow t_2 + t_0$ 
21:  $t_2 \leftarrow t_2 + t_0$ 
22:  $t_2 \leftarrow t_2 \cdot Z_Q$ 
23:  $\text{ctl} \leftarrow (dx = 0) \wedge (dy = 0)$ 
24: if  $\text{ctl}$  then
25:   swap  $(dx, t_1)$  and  $(dy, t_2)$ 
26: end if
27:  $t_0 \leftarrow Z_P \cdot Z_Q$ 
28:  $t_1 \leftarrow t_0^2$ 
29:  $t_2 \leftarrow dx^2$ 
30:  $t_3 \leftarrow dy^2$ 
31:  $X_{P+Q} \leftarrow A_{24} \cdot t_1$ 
32:  $X_{P+Q} \leftarrow X_{P+Q} + u_1$ 
33:  $X_{P+Q} \leftarrow X_{P+Q} + u_2$ 
34:  $X_{P+Q} \leftarrow X_{P+Q} \cdot t_2$ 
35:  $X_{P+Q} \leftarrow t_3 - X_{P+Q}$ 
36:  $Y_{P+Q} \leftarrow u_1 \cdot t_2$ 
37:  $Y_{P+Q} \leftarrow Y_{P+Q} - X_{P+Q}$ 
38:  $Y_{P+Q} \leftarrow dy \cdot Y_{P+Q}$ 
39:  $t_3 \leftarrow dx \cdot t_2$ 
40:  $t_3 \leftarrow t_3 \cdot v_1$ 
41:  $Y_{P+Q} \leftarrow Y_{P+Q} - t_3$ 
42:  $Z_{P+Q} \leftarrow dx \cdot t_0$ 
43: if  $\text{ctl}_1$  then
44:    $(X_{P+Q}, Y_{P+Q}, Z_{P+Q}) \leftarrow (X_Q, Y_Q, Z_Q)$ 
45: end if
46: if  $\text{ctl}_2$  then
47:    $(X_{P+Q}, Y_{P+Q}, Z_{P+Q}) \leftarrow (X_P, Y_P, Z_P)$ 
48: end if
49: return  $(X_{P+Q} : Y_{P+Q} : Z_{P+Q})$ 

```

// Cost: 6S + 17M + 13a

Algorithm 7.13 ADDComponents($P, Q, (A_{24} : C_{24})$)

Input: Distinct Jacobian points $P = (X_P : Y_P : Z_P)$, $Q = (X_Q : Y_Q : Z_Q)$ and the Montgomery coefficient $(A_{24} : C_{24})$ of the curve E .

Output: (u, v, w) such that the x -only coordinates of $P + Q$ and $P - Q$ are $P + Q = (u - v : w)$ and $P - Q = (u + v : w)$.

```

1:  $t_0 \leftarrow Z_P^2$ 
2:  $t_1 \leftarrow Z_Q^2$ 
3:  $t_2 \leftarrow X_P \cdot t_1$ 
4:  $t_3 \leftarrow t_0 \cdot X_Q$ 
5:  $t_4 \leftarrow Y_P \cdot Z_Q$ 
6:  $t_4 \leftarrow t_4 \cdot t_1$ 
7:  $t_5 \leftarrow Z_P \cdot Y_Q$ 
8:  $t_5 \leftarrow t_5 \cdot t_0$ 
9:  $t_0 \leftarrow t_0 \cdot t_1$ 
10:  $t_6 \leftarrow t_4 \cdot t_5$ 
11:  $t_4 \leftarrow t_4^2$ 
12:  $t_5 \leftarrow t_5^2$ 
13:  $t_4 \leftarrow t_4 + t_5$ 
14:  $t_5 \leftarrow t_2 + t_3$ 
15:  $t_7 \leftarrow t_3 + t_3$ 
16:  $t_7 \leftarrow t_5 - t_7$ 
17:  $t_7 \leftarrow t_7^2$ 
18:  $t_1 \leftarrow A_{24} \cdot t_0$ 
19:  $t_1 \leftarrow t_5 + t_1$ 
20:  $t_1 \leftarrow t_1 \cdot t_7$ 
21:  $u \leftarrow t_4 - t_1$ 
22:  $v \leftarrow t_6 + t_6$ 
23:  $w \leftarrow t_6 \cdot t_0$ 
24: return  $(u, v, w)$ 
```

// Cost: 5S + 11M + 7a

7.3. Computing pairings

This section explains how pairings are computed in the SQIsign2D² reference implementation (usage is described in Section 2.3.3). Given P, Q and n , there are multiple methods to compute $T_n(P, Q)$ including Miller's loop approach [Mil04], cubical arithmetic [Rob24] and elliptic nets [Sta07]. In the remainder of this section, we outline the first two approaches.

A morphism $f : E \rightarrow \mathbb{P}^1$ with divisor $\text{div}(f) = n(P) - ([n]P) - (n-1)(0_E)$ (i.e., it has a zero of multiplicity n at P , a simple pole at $[n]P$, and a pole of multiplicity $n-1$ at (0_E)) is called a Miller function, denoted by $f_{n,P}$. In particular, $\text{div}(f_{n,P}) = n(P) - n(0_E)$ when $P \in E[n]$. Defining divisors $D_P = (P) - (0_E)$ and $D_Q = (Q) - (0_E)$, the (unreduced) Tate pairing for $P, Q \in E[n]$ can be effectively computed as $T_n(P, Q) = f_{n,P}(D_Q)$. With the exception of minor technical details discussed in [BKL+02], we can simply compute $f_{n,P}(D_Q) = f_{n,P}(Q)$. In this section, for any two points P, Q , the equation $L_{P,Q} = 0$ represents the line passing through P and Q (when $P = Q$, it represents the tangent line at P).

7.3.1. The Miller loop approach

This method has been extensively investigated in previous works (see [FST10] for a pairing survey, and [Rei23] for applications in isogeny-based cryptography). Briefly, T_n is computed via a Miller loop that evaluates the Miller function $f_{n,P}$, which satisfies $\text{div}(f_{n,P}) = n(P) - n(0_E)$ [Mil04]. The loop employs a double-and-add algorithm, requiring $\lceil \log_2(n) \rceil$ iterations. More concretely, we compute the Miller function using the Miller formula ([Mil04], Lemma 2):

$$f_{(n+m),P} = f_{n,P} \cdot f_{m,P} \cdot \frac{L_{[n]P, [m]P}}{L_{[n+m]P, [-m-n]P}}$$

When $n = m$, we refer to this as doubling $[n]P$. SQIsign2D² requires pairings of degree $n = 2^r$ and $n = 3^s$. Therefore, the Miller loop must perform exactly r point doublings for $n = 2^r$, and s doublings together with s point additions for $n = 3^s$. Having computed $f_{n,P}(Q)$, we raise it to the power

$$(p-1) \cdot \frac{p+1}{n},$$

to compute the unreduced Tate pairing t_n . The Frobenius operator $\pi : x \rightarrow x^p$ acts on $\mathbb{F}_{p^2}$ as $\pi(a+bi) = a-bi$. Using this map, together with the fact that division in the subgroup μ_n is nearly cost-free, the value $f_{n,P}(Q)$ can be efficiently raised to the exponent $p-1$. Moreover, because $\frac{p+1}{n}$ is small, the final exponentiation is relatively inexpensive. This completes the evaluation of the Tate pairing via the Miller algorithm described in [Mil04].

7.3.2. The cubical arithmetic approach

In [Sta07], the author explains that $T_n(P, Q)$ can be reinterpreted as the monodromy $g_{P,Q}^{*,n} = T_n(P, Q)$, viewed as a constant function on the curve, in the biextension associated to the principal polarization (0_E) on E , where $g_{P,Q}$ is a rational biextension element above (P, Q) . This idea is further developed in [Rob24], where it is shown that the biextension arithmetic and the associated monodromy can be refined by means of cubical arithmetic.

We note that we will be working with the cubical arithmetic of level 2, associated to the divisor $2(0_E)$, on an elliptic curve in Montgomery form $E : y^2 = x^3 + Ax^2 + 1$, using the cubical formulas from [Rob24]. We have $\Gamma(2(0_E)) = (X, Z)$, where $x = X/Z$ represent the x -Weierstrass coordinate of E . The x coordinate $x(P)$ of a point $P = (x(P), y(P)) \in E$ allows to recover P up to a sign. We can also use the projective coordinates $(X(P) : Z(P)) = (x(P) : 1)$. A (level 2) cubical point $\tilde{P}$ above P is specified by a choice $\tilde{P} = (X(P), Z(P))$ of affine coordinates such that $x(P) = X(P)/Z(P)$.

Using cubical arithmetic, computing $T_n(P, Q)$ requires us to compute two monodromies λ_P and $\lambda_{P,Q}$, where $\lambda_P \in \mathbb{F}_{p^2}$ such that $[n]\tilde{P} = \lambda_P \cdot \tilde{0}$ and $\lambda_{P,Q} \in \mathbb{F}_{p^2}$ such that $[n]\tilde{P} + \tilde{Q} = \lambda_{P,Q} \cdot \tilde{Q}$, with $\tilde{0} = (0, 1)$, $\tilde{Q} = (x_Q, 1)$, where $[n]\tilde{P}$ and $[n]\tilde{P} + \tilde{Q}$ are the outputs of [GeneralCubicalLadder](#), given P, Q and n . We then find $T_n(P, Q) = \lambda_{P,Q}/\lambda_P$. This is the result of the Monodromy algorithm.

When $n \mid p+1$, the final reduction to obtain $t_n(P, Q)$ requires us to raise $T_n(P, Q)$ to $(p^2 - 1)/n = (p - 1) \cdot \frac{p+1}{n}$. Raising $\zeta \in \mathbb{F}_{p^2}$ to the power $(p - 1)$ is again simply $\pi(\zeta)/\zeta$. Hence, the final reduction can be computed as

$$t_n(P, Q) = (\pi(\lambda_{P,Q})/\pi(\lambda_P) \cdot \lambda_P/\lambda_{P,Q})^{\frac{p+1}{n}}. \quad (10)$$

The [Tate](#) algorithms (Algorithm 7.20 and Algorithm 7.21) computes this value $t_n(P, Q)$ given P and Q for $n = 2^e$ and $n = 3^e$.

To summarize, Algorithm 7.20 (resp. Algorithm 7.21) computes the Tate algorithm by first calculating a monodromy λ followed by the final exponentiation to the power $\frac{p^2-1}{2^e}$ (resp. $\frac{p^2-1}{3^e}$). The monodromy is computed by executing the three-point cubical arithmetic ladder in Algorithm 7.16 for $n = 2^e$ and in Algorithm 7.17 for $n = 3^e$, which uses the cubical double and cubical differential additions from Algorithm 7.14 and Algorithm 7.15 as auxiliary blocks. Notice also that the monodromy computation requires performing a couple of cubical translations from Algorithm 7.18 and ratios from Algorithm 7.19.

We warn the reader that our implementation of [Tate](#) (Algorithm 7.20 and Algorithm 7.21) is slightly different from the simplified description given above, as we use x -only arithmetic. Thus, we work with cubical points of level 2, which encode elements in the biextension associated to the divisor $2(0_E)$ rather than (0_E) . The former biextension monodromy yields the square of the Tate pairing $T_n(P, Q)$. For case $n = 2^e$, this loses one bit of information. Instead, in Algorithm 7.20, we use the action of the theta group $G(2(0_E))$, via [CubicalTranslate](#), to get the correct monodromy factor, i.e., from the biextension associated to (0_E) .

A final important caveat is that [CubicalDiffAdd](#) as described is off by a factor 4 from the correct cubical arithmetic: in the algorithm we should return $(X_2/4, Z_2/4)$. However, the current version still gives the correct Weil pairing $e_n(P, Q) = T_n(P, Q)/T_n(Q, P)$, and, thanks to the final exponentiation, the reduced Tate-Lichtenbaum pairing $t_n(P, Q)$ over $\mathbb{F}_{p^2}$. But one should be careful that the current version will not give the correct reduced Tate-Lichtenbaum pairing if computed over $\mathbb{F}_p$.

Algorithm 7.14 CubicalDbl($E, \tilde{P}$)

Input: An elliptic curve $E : y^2 = x^3 + Ax^2 + x$ in Montgomery model and a cubical point $\tilde{P} = (X(P), Z(P))$.

Output: The cubical double $2\tilde{P}$.

- 1: $a \leftarrow (X(P) + Z(P))^2$
 - 2: $b \leftarrow (X(P) - Z(P))^2$
 - 3: $c \leftarrow a - b$
 - 4: $X_2 \leftarrow a \cdot b$
 - 5: $Z_2 \leftarrow c \cdot \left(b + \frac{A+2}{4} \cdot c\right)$
 - 6: **return** (X_2, Z_2)
-

Algorithm 7.15 CubicalDiffAdd($E, \tilde{P}, \tilde{Q}, x(P - Q)$)

Input: Two cubical points in cubical Montgomery coordinates $\tilde{P} = (X(P), Z(P))$, $\tilde{Q} = (X(Q), Z(Q))$ and the x -coordinate of their difference $x(P - Q)$.

Output: The cubical differential addition $\widetilde{P + Q}$.

- 1: $a \leftarrow X(P) + Z(P)$
 - 2: $b \leftarrow X(P) - Z(P)$
 - 3: $c \leftarrow X(Q) + Z(Q)$
 - 4: $d \leftarrow X(Q) - Z(Q)$
 - 5: $X_2 \leftarrow (a \cdot d + b \cdot c)^2$
 - 6: $Z_2 \leftarrow (a \cdot d - b \cdot c)^2$
 - 7: $X_2 \leftarrow X_2 / x(P - Q)$
 - 8: **return** (X_2, Z_2)
-

Algorithm 7.16 CubicalLadder($E, e, \widetilde{P + Q}, \tilde{P}, x(Q)$)

Input: An elliptic curve $E : y^2 = x^3 + Ax^2 + x$ in Montgomery model, an integer e , two cubical points $\widetilde{P + Q} = (X(P + Q), Z(P + Q))$, $\tilde{P} = (X(P), Z(P))$ and the x -coordinate $x(Q)$ of Q .

Output: The cubical points $2^e \tilde{P}$ and $2^e \tilde{P} + \tilde{Q}$.

- 1: $nPQ \leftarrow \widetilde{P + Q}$
 - 2: $nP \leftarrow \tilde{P}$
 - 3: **for** k from 1 up to e **do**
 - 4: $nPQ \leftarrow \text{CubicalDiffAdd}(E, nPQ, nP, x(Q))$
 - 5: $nP \leftarrow \text{CubicalDbl}(E, nP)$
 - 6: **end for**
 - 7: **return** (nP, nPQ)
-

Algorithm 7.17 GeneralCubicalLadder($E, n, x(P + Q), x(P), x(Q)$)

Input: An elliptic curve $E : y^2 = x^3 + Ax^2 + x$ in Montgomery model, an integer n , and the x -coordinate $x(P + Q)$ of $P + Q$, $x(P)$ of P and $x(Q)$ of Q .

Output: The cubical points $n\tilde{P}$ and $n\tilde{P} + \tilde{Q}$.

- 1: $nPQ \leftarrow (x(P + Q), 1)$
 - 2: $nP \leftarrow (x(P), 1)$
 - 3: **for** each bit b of $n - 1$ (from the most significant to the least significant) **do**
 - 4: **if** $b = 1$ **then**
 - 5: $nPQ \leftarrow \text{CubicalDiffAdd}(E, nPQ, nP, x(Q))$
 - 6: $nP \leftarrow \text{CubicalDbl}(E, nP)$
 - 7: **if** b is the most significant bit **then**
 - 8: $nm1P \leftarrow nP$
 - 9: **else**
 - 10: $nm1P \leftarrow \text{CubicalDiffAdd}(E, nP, nm1P, x(P))$
 - 11: **end if**
 - 12: **else**
 - 13: $nPQ \leftarrow \text{CubicalDiffAdd}(E, nPQ, nm1P, x(P + Q))$
 - 14: $nm1P \leftarrow \text{CubicalDbl}(E, nm1P)$
 - 15: $nP \leftarrow \text{CubicalDiffAdd}(E, nP, nm1P, x(P))$
 - 16: **end if**
 - 17: **end for**
 - 18: **return** (nP, nPQ)
-

Algorithm 7.18 CubicalTranslate($\widetilde{P}, \widetilde{T}$)

Input: Cubical Montgomery coordinates $\widetilde{P} = (X(P), Z(P))$ and $\widetilde{T} = (X(T), Z(T))$ where $T = (X(T) : Z(T))$ is a 2-torsion point.

Output: The cubical translation $\widetilde{P} + \widetilde{T} = (X(P + T), Z(P + T))$.

```

1:  $X \leftarrow X(T)X(P) - Z(T)Z(P)$ 
2:  $Z \leftarrow Z(T)X(P) - X(T)Z(P)$ 
3: if  $Z(T) = 0$  then
4:    $Z \leftarrow -Z$ 
5: end if
6: if  $X(T) = 0$  then
7:    $X \leftarrow -X$ 
8: end if
9: return  $(X, Z)$ 
```

Algorithm 7.19 CubicalRatio($\widetilde{P}_1, \widetilde{P}_2$)

Input: Cubical Montgomery coordinates $\widetilde{P}_1 = (X(P_1), Z(P_1))$ and $\widetilde{P}_2 = (X(P_2), Z(P_2))$ such that $P_1 = (X(P_1) : Z(P_1)) = P_2 = (X(P_2) : Z(P_2))$.

Output: The ratio λ such that $X(P_2) = \lambda X(P_1)$ and $Z(P_2) = \lambda Z(P_1)$.

```

1: if  $X(P_1) = 0$  then
2:   return  $Z(P_2)/Z(P_1)$ 
3: else
4:   return  $X(P_2)/X(P_1)$ 
5: end if
```

Algorithm 7.20 Tate₂($E, e, x(P), x(Q), x(P + Q)$)

Input: A supersingular elliptic curve $E/\mathbb{F}_{p^2} : y^2 = x^3 + Ax^2 + x$ in Montgomery model, an integer e , the x -coordinates of the points $P, Q, P + Q \in E(\mathbb{F}_{p^2})$ where $2^e P = 0_E$.

Output: The reduced Tate pairing $t_{2^e}(P, Q) \in \mu_{2^e}$.

```

1:  $nP, nPQ \leftarrow \text{CubicalLadder}(E, e - 1, (x(P + Q), 1), (x(P), 1), x(Q))$ 
2:  $O \leftarrow \text{CubicalTranslate}(nP, nP)$ 
3:  $Q' \leftarrow \text{CubicalTranslate}(nPQ, nP)$ 
4:  $\lambda \leftarrow \text{CubicalRatio}(Q, Q') / \text{CubicalRatio}((1, 0), O)$ 
5: return  $\lambda^{(p^2-1)/2^e}$ 
```

// Using Eq. (10)

Algorithm 7.21 Tate₃($E, e, x(P), x(Q), x(P + Q)$)

Input: A supersingular elliptic curve $E/\mathbb{F}_{p^2} : y^2 = x^3 + Ax^2 + x$ in Montgomery model, an integer e , the x -coordinates of the points $P, Q, P + Q \in E(\mathbb{F}_{p^2})$ where $3^e P = 0_E$.

Output: The reduced Tate pairing $t_{3^e}(P, Q) \in \mu_{3^e}$.

```

1:  $nP, nPQ \leftarrow \text{GeneralCubicalLadder}(E, 3^e, (x(P + Q), 1), (x(P), 1), x(Q))$ 
2:  $\lambda \leftarrow \text{CubicalRatio}(Q, nPQ) / \text{CubicalRatio}((1, 0), nP)$ 
3: return  $\lambda^{(p^2-1)/3^e}$ 
```

// Using Eq. (10)

7.4. Isogenies

SQIsign2D² requires isogenies of degree 2^a and 3^b , which we compute through chains of 2-isogenies, 4-isogenies and 3-isogenies using the following algorithms:

- **TwoIsogenyCodomain** (Algorithm 7.22) takes as input the projective x -coordinate of a kernel generator $P \in E$ of order 2 with $P \neq (0 : 1)$, and outputs the codomain curve E' of the 2-isogeny of kernel $\langle P \rangle$ represented by the projective Montgomery coefficient $(A'_{24} : C'_{24})$.
- **TwoIsogenyEval** (Algorithm 7.23) takes as input the projective x -coordinates of a kernel generator $P \in E$ of order 2 with $P \neq (0 : 1)$ and of a point $Q \in E$, and outputs the projective x -coordinate of the image $Q' \in E'$ of Q under the isogeny computed by **TwoIsogenyCodomain**.
- **TwoIsogenyCodomainSingular** (Algorithm 7.24) and **TwoIsogenyEvalSingular** (Algorithm 7.25) are equivalent to **TwoIsogenyCodomain** and **TwoIsogenyEval** for the case that $P = (0 : 1)$.
- **FourIsogenyCodomain** (Algorithm 7.26) takes as input the projective x -coordinate of a kernel generator $P \in E$ of order 4 with $[2]P \neq (0 : 1)$, and outputs the codomain curve E' of the 4-isogeny of kernel $\langle P \rangle$ represented by the projective Montgomery coefficient $(A'_{24} : C'_{24})$ and constants **consts** needed for **FourIsogenyEval**.
- **FourIsogenyEval** (Algorithm 7.27) takes as input the projective x -coordinate of a point $Q \in E$, and the constants **consts** returned by **FourIsogenyCodomain**. It outputs the projective x -coordinate of the image $Q' \in E'$ of Q under the isogeny computed by **FourIsogenyCodomain**.
- **TwoIsogenyChain** (Algorithm 7.28) takes as input the projective x -coordinate of a kernel generator $P \in E$ of order 2^e , the Montgomery coefficient $(A'_{24} : C'_{24})$ of E , the positive integer e , and a list of points **pts** to be evaluated. It outputs the codomain curve E' of the 2^e -isogeny of kernel $\langle P \rangle$ represented by the projective Montgomery coefficient $(A'_{24} : C'_{24})$, and the list of evaluated points **pts'**. The algorithm computes a chain of $\lfloor e/2 \rfloor$ isogenies of degree 4, followed by a final 2-isogeny if e is odd, using balanced strategies.
- **ThreeIsogenyCodomain** (Algorithm 7.30) takes as input the projective x -coordinate of a kernel generator $P \in E$ of order 3, and outputs the codomain curve E' of the 3-isogeny of kernel $\langle P \rangle$ represented by the projective Montgomery coefficient $(A'_{24} : A'_{24})$ and constants **consts** needed for **ThreeIsogenyEval**.
- **ThreeIsogenyEval** (Algorithm 7.31) takes as input the projective x -coordinate of a point $Q \in E$, and the constants **consts** returned by **ThreeIsogenyCodomain**. It outputs the projective x -coordinate of the image $Q' \in E'$ of Q under the isogeny computed by **ThreeIsogenyCodomain**.
- **ThreeIsogenyChain** (Algorithm 7.32) takes as input the projective x -coordinate of a point $P \in E$ of order 3^e , the Montgomery coefficient $(A'_{24} : C'_{24})$ of E , the positive integer e , and a list of points **pts** to be evaluated. It outputs the codomain curve E' of the 3^e -isogeny of kernel $\langle P \rangle$ represented by the projective Montgomery coefficient $(A'_{24} : C'_{24})$, and the list of evaluated points **pts'**.

When computing a chain of 2^e -isogenies during **TwoIsogenyChain**, it is necessary to verify the order of P . This is achieved in the first step of the chain by checking for a point of order 4 (if $e > 1$) or of order 2 (if $e = 1$). For honestly generated inputs, our implementation ensures that the special cases $P = (0 : 1)$ and $[2]P = (0 : 1)$ cannot arise. Thus, any signature leads to these cases is rejected by the verification step.

We further compute small chains of 2-isogenies that can include the special case of a kernel generator $P = (0 : 1)$ only during signing, as described in **TwoIsogenyChainSmall** (Algorithm 7.29).

Algorithm 7.22 TwoIsogenyCodomain(P)

Input: Projective point $P = (X_P : Z_P) \in E$ of order 2 with $P \neq (0 : 1)$.**Output:** The coefficient $(A'_{24} : C'_{24})$ corresponding to the image curve $E' := E/\langle P \rangle$ under the 2-isogeny.

- 1: $A'_{24} \leftarrow X_P^2$
 - 2: $C'_{24} \leftarrow Z_P^2$
 - 3: $A'_{24} \leftarrow C'_{24} - A'_{24}$
 - 4: **return** $(A'_{24} : C'_{24})$
-

Algorithm 7.23 TwoIsogenyEval(P, Q)

Input: Projective point $P = (X_P : Z_P)$ of order 2 with $P \neq (0 : 1)$, and projective point $Q = (X_Q : Z_Q)$ where $P, Q \in E$.**Output:** Projective point $Q' = (X_{Q'} : Z_{Q'}) \in E'$ corresponding to the image of Q under the 2-isogeny defined by P .

- 1: $t_0 \leftarrow X_P + Z_P$
 - 2: $t_1 \leftarrow X_P - Z_P$
 - 3: $t_2 \leftarrow X_Q + Z_Q$
 - 4: $t_3 \leftarrow X_Q - Z_Q$
 - 5: $t_0 \leftarrow t_0 \cdot t_3$
 - 6: $t_1 \leftarrow t_1 \cdot t_2$
 - 7: $t_2 \leftarrow t_0 + t_1$
 - 8: $t_3 \leftarrow t_0 - t_1$
 - 9: $X_{Q'} \leftarrow X_Q \cdot t_2$
 - 10: $Z_{Q'} \leftarrow Z_Q \cdot t_3$
 - 11: **return** $(X_{Q'} : Z_{Q'})$
-

Algorithm 7.24 TwoIsogenyCodomainSingular($(A_{24} : C_{24})$)

Input: Montgomery coefficient $(A_{24} : C_{24})$ of the curve E .**Output:** The coefficients $(A'_{24} : C'_{24})$ corresponding to the image curve $E' := E/\langle P \rangle$ under the 2-isogeny with $P = (0 : 1)$, and constants `consts` needed for TwoIsogenyEvalSingular.

- 1: $t_0 \leftarrow A_{24} + A_{24}$
 - 2: $t_0 \leftarrow t_0 - C_{24}$
 - 3: $t_0 \leftarrow t_0 + t_0$
 - 4: $t_1 \leftarrow C_{24}^{-1}$
 - 5: $t_0 \leftarrow t_0 \cdot t_1$
 - 6: $c_1 \leftarrow t_0$
 - 7: $A'_{24} \leftarrow t_0 + t_0$
 - 8: $t_0 \leftarrow t_0^2$
 - 9: $t_0 \leftarrow t_0 - 4$
 - 10: $t_0 \leftarrow \sqrt{t_0}$
 - 11: $c_1 \leftarrow -t_0$
 - 12: $C'_{24} \leftarrow t_0 + t_0$
 - 13: $A'_{24} \leftarrow A'_{24} + C'_{24}$
 - 14: $C'_{24} \leftarrow C'_{24} + C'_{24}$
 - 15: **consts** $\leftarrow (c_0, c_1)$
 - 16: **return** $(A'_{24} : C'_{24}), \text{consts}$
-

Algorithm 7.25 TwoIsogenyEvalSingular(Q, consts)

Input: Projective point $Q = (X_Q : Z_Q)$ where $Q \in E$, constants consts returned by TwoIsogenyCodomainSingular.

Output: Projective point $Q' = (X_{Q'} : Z_{Q'}) \in E'$ corresponding to the image of Q under the 2-isogeny defined by $P = (0 : 1)$.

- 1: $c_0, c_1 \leftarrow \text{consts}$
 - 2: $t_0 \leftarrow X_Q \cdot Z_Q$
 - 3: $t_1 \leftarrow c_0 \cdot Z_Q$
 - 4: $t_1 \leftarrow t_1 + X_Q$
 - 5: $t_1 \leftarrow t_1 \cdot X_Q$
 - 6: $X_{Q'} \leftarrow Z_Q^2$
 - 7: $X_{Q'} \leftarrow X_{Q'} + t_1$
 - 8: $Z_{Q'} \leftarrow t_0 \cdot c_1$
 - 9: **return** $(X_{Q'} : Z_{Q'})$
-

Algorithm 7.26 FourIsogenyCodomain(P)

Input: Projective point $P = (X_P : Z_P) \in E$ of order 4 such that $[2]P \neq (0 : 1)$.

Output: The coefficient $(A'_{24} : C'_{24})$ corresponding to the image curve $E' := E/\langle P \rangle$ under the 4-isogeny, and constants consts needed for FourIsogenyEval.

- 1: $c_0 \leftarrow Z_P^2$
 - 2: $c_1 \leftarrow X_P - Z_P$
 - 3: $c_2 \leftarrow X_P + Z_P$
 - 4: $t_0 \leftarrow X_P^2$
 - 5: $t_3 \leftarrow c_0 + t_0$
 - 6: $t_4 \leftarrow c_0 - t_0$
 - 7: $A'_{24} \leftarrow t_3 \cdot t_4$
 - 8: $C'_{24} \leftarrow c_0^2$
 - 9: $c_0 \leftarrow c_0 + c_0$
 - 10: $c_0 \leftarrow c_0 + c_0$
 - 11: $\text{consts} \leftarrow (c_0, c_1, c_2)$
 - 12: **return** $(A'_{24} : C'_{24}), \text{consts}$
-

Algorithm 7.27 FourIsogenyEval(Q, consts)

Input: Projective point $Q = (X_Q : Z_Q)$ where $Q \in E$, and constants consts returned by FourIsogenyCodomain.

Output: Projective point $Q' = (X_{Q'} : Z_{Q'}) \in E'$ corresponding to the image of Q under the 4-isogeny defined by P .

- 1: $c_0, c_1, c_2 \leftarrow \text{consts}$
 - 2: $t_0 \leftarrow X_Q + Z_Q$
 - 3: $t_1 \leftarrow X_Q - Z_Q$
 - 4: $X_{Q'} \leftarrow t_0 \cdot c_1$
 - 5: $Z_{Q'} \leftarrow t_1 \cdot c_2$
 - 6: $t_0 \leftarrow t_0 \cdot t_1$
 - 7: $t_0 \leftarrow t_0 \cdot c_0$
 - 8: $t_1 \leftarrow X_{Q'} + Z_{Q'}$
 - 9: $Z_{Q'} \leftarrow X_{Q'} - Z_{Q'}$
 - 10: $t_1 \leftarrow t_1^2$
 - 11: $Z_{Q'} \leftarrow Z_{Q'}^2$
 - 12: $X_{Q'} \leftarrow t_0 + t_1$
 - 13: $t_0 \leftarrow t_0 - Z_{Q'}$
 - 14: $X_{Q'} \leftarrow X_{Q'} \cdot t_1$
 - 15: $Z_{Q'} \leftarrow Z_{Q'} \cdot t_0$
 - 16: **return** $(X_{Q'} : Z_{Q'})$
-

Algorithm 7.28 TwoIsogenyChain($P, (A_{24} : C_{24}), e, \text{pts}$)

Input: Projective point $P = (X_P : Z_P) \in E$ of order 2^e , Montgomery coefficient $(A_{24} : C_{24})$ of the curve E , a positive integer e , a list of points pts to be evaluated.

Output: The coefficient $(A'_{24} : C'_{24})$ corresponding to the image curve $E' := E/\langle P \rangle$ under the 2^e -isogeny, list of evaluated points pts' .

```

1:  $(A'_{24} : C'_{24}) \leftarrow (A_{24} : C_{24})$ 
2: Initialize a list  $\text{strat\_pts} \leftarrow [P]$ 
3: Initialize a list  $\text{orders} \leftarrow [e]$ 
4:  $k \leftarrow 0$ 
5: for  $j$  from 0 up to  $\lfloor e/2 \rfloor - 1$  do                                     // Chain of 4-isogenies
6:   while  $\text{orders}[k] \neq 2$  do
7:      $k \leftarrow k + 1$ 
8:      $n \leftarrow \lfloor \text{orders}[k-1]/4 \rfloor \cdot 2 + (\text{orders}[k-1] \bmod 2)$ 
9:      $\text{orders}[k] \leftarrow \text{orders}[k-1] - n$ 
10:     $\text{strat\_pts}[k] \leftarrow \text{strat\_pts}[k-1]$ 
11:    for  $i$  from 0 up to  $n-1$  do
12:       $\text{strat\_pts}[k] \leftarrow \text{xDBL}(\text{strat\_pts}[k], (A'_{24} : C'_{24}))$ 
13:    end for
14:  end while
15:   $K \leftarrow \text{strat\_pts}[k]$ 
16:  if  $j = 0$  then                                                         // Input point validation
17:    if  $[2]K = 0_E$  or  $[4]K \neq 0_E$  then
18:      raise Exception: ("TwolsogenyChain failed: wrong point order")
19:    end if
20:    if  $[2]K = (0 : 1)$  then
21:      raise Exception: ("TwolsogenyChain failed: unexpected singular 4-isogeny")
22:    end if
23:  end if
24:   $((A'_{24} : C'_{24}), \text{consts}) \leftarrow \text{FourslogenyCodomain}(K)$ 
25:   $\text{strat\_pts} \leftarrow [\text{FourslogenyEval}(Q, \text{consts}) \mid Q \in \text{strat\_pts}]$ 
26:   $\text{orders} \leftarrow [m-2 \mid m \in \text{orders}]$ 
27:   $k \leftarrow k-1$ 
28:   $\text{pts} \leftarrow [\text{FourslogenyEval}(Q, \text{consts}) \mid Q \in \text{pts}]$ 
29: end for
30: if  $e \bmod 2 = 1$  then                                                 // Final 2-isogeny if  $e$  is odd
31:    $K \leftarrow \text{strat\_pts}[0]$ 
32:   if  $e = 1$  then                                                         // Input point validation.
33:     if  $K = 0_E$  or  $[2]K \neq 0_E$  then
34:       raise Exception: ("TwolsogenyChain failed: wrong point order")
35:     end if
36:     if  $K = (0 : 1)$  then
37:       raise Exception: ("TwolsogenyChain failed: unexpected singular 2-isogeny")
38:     end if
39:   end if
40:    $(A'_{24} : C'_{24}) \leftarrow \text{TwolsogenyCodomain}(K)$ 
41:    $\text{pts} \leftarrow [\text{TwolsogenyEval}(Q, K) \mid Q \in \text{pts}]$ 
42: end if
43: return  $(A'_{24} : C'_{24}), \text{pts}$ 

```

Algorithm 7.29 TwoIsogenyChainSmall($P, (A_{24} : C_{24}), e, \text{pts}, \text{cond}$)

Input: Projective point $P = (X_P : Z_P) \in E$ of order 2^e , Montgomery coefficient $(A_{24} : C_{24})$ of the curve E , a positive integer e , a list of points pts to be evaluated, a boolean cond signifying if the function is called during signing or verifying.

Output: The coefficient $(A'_{24} : C'_{24})$ corresponding to the image curve $E' := E/\langle P \rangle$ under the 2^e -isogeny, list of evaluated points pts' .

```

1:  $(A'_{24} : C'_{24}) \leftarrow (A_{24} : C_{24})$ 
2: for  $i$  from 0 up to  $e - 1$  do
3:    $K \leftarrow P$ 
4:   for  $j$  from 0 up to  $e - i - 2$  do
5:      $K \leftarrow \text{xDBL}(P, (A_{24} : C_{24}))$ 
6:   end for
7:   if  $i = 0$  then
8:     if  $[2]K \neq 0_E$  then
9:       raise Exception: ("TwoIsogenyChainSmall failed: wrong order") // Can only occur during
verification
10:    end if
11:    if  $K = (0 : 1)$  then
12:      if  $\text{cond} = \text{true}$  then // Function called during signing
13:         $((A'_{24} : C'_{24}), \text{consts}) \leftarrow \text{TwoIsogenyCodomainSingular}((A'_{24} : C'_{24}))$ 
14:         $P \leftarrow \text{TwoIsogenyEvalSingular}(P, \text{consts})$ 
15:         $\text{pts} \leftarrow [\text{TwoIsogenyEvalSingular}(Q, \text{consts}) \mid Q \in \text{pts}]$ 
16:      else // Function called during verification.
17:        raise Exception: ("TwoIsogenyChainSmall failed: unexpected singular 2-isogeny")
18:      end if
19:    end if
20:  else
21:     $(A'_{24} : C'_{24}) \leftarrow \text{TwoIsogenyCodomain}(K)$ 
22:     $P \leftarrow \text{TwoIsogenyEval}(K, P)$ 
23:     $\text{pts} \leftarrow [\text{TwoIsogenyEval}(K, Q) \mid Q \in \text{pts}]$ 
24:  end if
25: end for
26: return  $(A'_{24} : C'_{24}), \text{pts}$ 

```

Algorithm 7.30 ThreeIsogenyCodomain(P)

Input: Projective point $P = (X_P : Z_P) \in E$ of order 3.**Output:** The coefficient $(A_{24}^+ : A_{24}^-)$ corresponding to the image curve $E' := E/\langle P \rangle$ under the 3-isogeny, and constants `consts` needed for ThreeIsogenyEval.

```

1:  $c_1 \leftarrow X_P - Z_P$ 
2:  $t_0 \leftarrow c_1^2$ 
3:  $c_2 \leftarrow X_P + Z_P$ 
4:  $t_1 \leftarrow c_2^2$ 
5:  $t_2 \leftarrow t_0 + t_1$ 
6:  $t_3 \leftarrow c_1 + c_2$ 
7:  $t_3 \leftarrow t_3^2$ 
8:  $t_3 \leftarrow t_3 - t_2$ 
9:  $t_2 \leftarrow t_1 + t_3$ 
10:  $t_3 \leftarrow t_3 + t_0$ 
11:  $t_4 \leftarrow t_3 + t_0$ 
12:  $t_4 \leftarrow t_4 + t_4$ 
13:  $t_4 \leftarrow t_1 + t_4$ 
14:  $A_{24}^- \leftarrow t_2 \cdot t_4$ 
15:  $t_4 \leftarrow t_1 + t_2$ 
16:  $t_4 \leftarrow t_4 + t_4$ 
17:  $t_4 \leftarrow t_0 + t_4$ 
18:  $A_{24}^+ \leftarrow t_3 \cdot t_4$ 
19: consts  $\leftarrow (c_1, c_2)$ 
20: return  $(A_{24}^+ : A_{24}^-)$ , consts

```

Algorithm 7.31 ThreeIsogenyEval(Q , `consts`)

Input: Projective point $Q = (X_Q : Z_Q)$ where $Q \in E$, and constants `consts` returned by ThreeIsogenyCodomain.**Output:** Projective point $Q' = (X_{Q'} : Z_{Q'}) \in E'$ corresponding to the image of Q under the 3-isogeny defined by P .

```

1:  $c_1, c_2 \leftarrow \text{consts}$ 
2:  $t_0 \leftarrow X_Q + Z_Q$ 
3:  $t_1 \leftarrow X_Q - Z_Q$ 
4:  $t_0 \leftarrow t_0 \cdot c_1$ 
5:  $t_1 \leftarrow t_1 \cdot c_2$ 
6:  $t_2 \leftarrow t_0 + t_1$ 
7:  $t_0 \leftarrow t_1 - t_0$ 
8:  $t_2 \leftarrow t_2^2$ 
9:  $t_0 \leftarrow t_0^2$ 
10:  $X_{Q'} \leftarrow X_Q \cdot t_2$ 
11:  $Z_{Q'} \leftarrow Z_Q \cdot t_0$ 
12: return  $(X_{Q'} : Z_{Q'})$ 

```

Algorithm 7.32 ThreeIsogenyChain($P, (A_{24}^+ : A_{24}^-), e, \text{pts}$)

Input: Projective point $P = (X_P : Z_P) \in E$ of order 3^e , Montgomery coefficient $(A_{24}^+ : A_{24}^-)$ of the curve E , a positive integer e , a list of points pts to be evaluated.

Output: The coefficient $(A_{24}^+ : A_{24}^-)$ corresponding to the image curve $E' := E/\langle P \rangle$ under the 3^e -isogeny, list of evaluated points pts' .

```

1: Initialize a list strat_pts  $\leftarrow [P]$ 
2: Initialize a list orders  $\leftarrow [e]$ 
3:  $k \leftarrow 0$ 
4: for  $j$  from 0 up to  $e - 1$  do                                     // Chain of 3-isogenies
5:   while orders[ $k$ ]  $\neq 1$  do
6:      $k \leftarrow k + 1$ 
7:      $n \leftarrow \lfloor \text{orders}[k - 1] / 2 \rfloor$ 
8:     orders[ $k$ ]  $\leftarrow \text{orders}[k - 1] - n$ 
9:     strat_pts[ $k$ ]  $\leftarrow \text{strat\_pts}[k - 1]$ 
10:    for  $i$  from 0 up to  $n - 1$  do
11:      strat_pts[ $k$ ]  $\leftarrow \text{xTPL}(\text{strat\_pts}[k], (A_{24}^+ : A_{24}^-))$ 
12:    end for
13:  end while
14:   $K \leftarrow \text{strat\_pts}[k]$ 
15:  if  $K = 0_E$  or  $[3]K \neq 0_E$  then
16:    raise Exception: ("ThreeIsogenyChain failed: wrong point order")
17:  end if
18:   $((A_{24}^+ : A_{24}^-), \text{consts}) \leftarrow \text{ThreelsogenyCodomain}(K)$ 
19:  strat_pts  $\leftarrow [\text{ThreelsogenyEval}(Q, \text{consts}) \mid Q \in \text{strat\_pts}]$ 
20:  orders  $\leftarrow [m - 1 \mid m \in \text{orders}]$ 
21:   $k \leftarrow k - 1$ 
22:  pts  $\leftarrow [\text{ThreelsogenyEval}(Q, \text{consts}) \mid Q \in \text{pts}]$ 
23: end for
24: return  $(A_{24}^+ : A_{24}^-), \text{pts}$ 

```

7.5. Theta coordinates

We now describe the concrete implementation of $(2, 2)$ -isogenies. This implementation is grounded in the mathematical framework presented in [DMP+24]. For every algorithm, we report its computational cost over the base field $\mathbb{F}_{p^2}$ in terms of squarings (**S**), multiplications (**M**), inversions (**I**), and additions (**a**).

7.5.1. Theta coordinates of level 2

For efficiency reasons, we use *theta coordinates* which can be seen as a generalization of x -only arithmetic instead of the Jacobian model for arithmetic operations on principally polarized abelian surface. In this section, we define the theta coordinates on Montgomery curves and on principally polarized abelian surface.

7.5.1.1 On Montgomery curves

Let $E_{A,B}$ be a Montgomery curve defined over $\mathbb{F}_q$. A system of theta coordinates (of level 2) on E , denoted by $(\theta_0 : \theta_1)$, is determined by the *theta null point* $(a : b)$ whose coordinates are called the *theta constants*.

Given a basis (T'_1, T'_2) of $E[4]$, where $T'_1 = (-1 : 1)$ and $T'_2 = (r : s)$ in $(X : Z)$ -coordinates, we have $(a : b) = (r + s : r - s)$. Given the theta null point $(a : b)$, we obtain change of coordinates formulas between Montgomery $(X : Z)$ -coordinates and theta coordinates:

$$\begin{aligned} (X : Z) &\longrightarrow (\theta_0 : \theta_1) = (a(X - Z) : b(X + Z)), \\ (X : Z) &= (a\theta_1 + b\theta_0 : a\theta_1 - b\theta_0) \longleftarrow (\theta_0 : \theta_1), \end{aligned}$$

using the convention that $0_E = (1 : 0)$ in $(X : Z)$ -coordinates. Obviously, the different choice of the basis of $E[4]$ will lead to different theta coordinates. For the theta null point $(a : b)$, we can compute the change of coordinates using [MontgomeryToTheta](#)(Algorithm 7.33).

Algorithm 7.33 [MontgomeryToTheta](#)($P, 0$)

Input: A point $P := (X : Z)$ in Montgomery coordinates, and theta null point $0 := (a : b)$.

Output: Point P in theta coordinates $(\theta_0 : \theta_1)$.

- 1: $\theta_0 \leftarrow X - Z$
- 2: $\theta_0 \leftarrow a \cdot \theta_0$
- 3: $\theta_1 \leftarrow X + Z$
- 4: $\theta_1 \leftarrow b \cdot \theta_1$
- 5: **return** $(\theta_0 : \theta_1)$

// Total cost: $2\mathbf{M} + 2\mathbf{a}$

7.5.1.2 On principally polarized abelian surfaces

Let A be a principally polarized abelian surface defined over $\mathbb{F}_q$. A system of theta coordinates (of level 2) on A consists of projective coordinates $(x : y : z : w)$ (possibly defined over an extension of $\mathbb{F}_q$). Indeed, theta coordinates of level 2 are canonical coordinate functions on an abelian variety determined by a chosen theta structure. They are obtained from a basis of the global sections of the line bundle associated with 2Θ , where Θ is a fixed divisor, as detailed in [DMP+24].

Just like the elliptic curve case, a system of theta coordinates (of level 2) on principally polarized abelian surface A , along with A itself, is fully determined by the theta null point $(a : b : c : d) := (x(0) : y(0) : z(0) : w(0))$. Similar to the elliptic curve case, we can represent points on A with part of projective coordinates. If A is a product of elliptic curves, the coordinate $(x : y : z : w)$ determines two elliptic points up to sign on each component, and thus represents

up to four points $(\pm P, \pm Q)$ of the product surface. As for the Jacobian case, the coordinates $(x : y : z : w)$ determine points up to sign.

However, for a given principally polarized abelian surface, there are multiple systems of theta coordinates, and these systems are related by coordinate changes [Dar25, Theorem 12]. In the implementation, we fix a choice of theta coordinate system following [DMP+24].

7.5.1.3 Product theta coordinates

Let E_1 (resp. E_2) be a Montgomery elliptic curve equipped with level-2 theta coordinates as in Section 7.5.1.1, denoted by (θ_0, θ_1) (resp. (θ'_0, θ'_1)). Let $A = E_1 \times E_2$ be the principally polarized abelian surface endowed with the theta coordinates induced by the theta structures on the elliptic curves. These are called the *product theta coordinates* on A , and are given by

$$(x : y : z : w) := (\theta_0\theta'_0 : \theta_1\theta'_0 : \theta_0\theta'_1 : \theta_1\theta'_1).$$

Not every system of theta coordinates on A is a product theta coordinate system. The conversion between product and non-product theta coordinates therefore plays an important role.

- **ProductToTheta** (Algorithm 7.43) takes as input a change-of-basis matrix of an isogeny Φ together with a list of points on $E_1 \times E_2$ given in product theta coordinates. It outputs the coordinates of these points in a corresponding non-product theta coordinate system.
- **SplittingIsomorphism** (Algorithm 7.46) takes as input the origin 0_A on $A \cong E_1 \times E_2$, given in a theta coordinate system that is not a product one. It outputs the product theta coordinates of this point, thereby determining a product theta coordinate system on A .

When computing a gluing isogeny $\Phi_1 : E_1 \times E_2 \rightarrow A_1$, we may have to change coordinates from a product to a non-product system of theta coordinates (see **ProductToTheta**). Similarly, when we compute a splitting isogeny $\Phi_e : A_{e-1} \rightarrow E_3 \times E_4$, the system of theta coordinates on the codomain $E_3 \times E_4$ may not be of product form, and so we may have to compute a change of coordinates to express image points component-wise in $E_3 \times E_4$ (see **SplittingIsomorphism**).

7.5.2. Doubling formulas using theta coordinates

Let A be a principally polarized abelian surface with a system of theta coordinates of level 2 induced by the theta null point $(a : b : c : d)$. Following the expression in [DMP+24], we introduce the *Hadamard* operators acting on theta-coordinates which cost **8a**:

$$\mathcal{H}(x, y, z, w) := (x + y + z + w, x - y + z - w, x + y - z - w, x - y - z + w),$$

and the *squaring* operators acting on theta-coordinates which cost **4S**:

$$\mathcal{S}(x, y, z, w) := (x^2, y^2, z^2, w^2),$$

using recursive formulae for the Hadamard operator.

The Hadamard operator maps the theta null point (a, b, c, d) to $(a', b', c', d') := \mathcal{H}(a, b, c, d)$, called the *dual theta null point*. Accordingly, this operator induces a system of theta coordinates called the *dual theta coordinates*.

A level-2 theta null point determines a canonical 2-isogeny $A \rightarrow B$. The dual of the theta null point of B , with coordinates denoted by $(\alpha : \beta : \gamma : \delta)$, is called the *dual isogenous theta null point*. These coordinates satisfy $(\alpha^2 : \beta^2 : \gamma^2 : \delta^2) = \mathcal{H}(a^2 : b^2 : c^2 : d^2)$.

ThetaDBL(Algorithm 7.35) takes as input the theta coordinates of a point P and auxiliary constants which depend only on the theta null point. The precomputation of these constants is done by **ThetaPrecomp**(Algorithm 7.34). It outputs the theta coordinates of the point $[2]P$.

To streamline the exposition in the remainder of this section, we write $[2]P$ for the output of [ThetaDBL](#) on input P . The constants `consts` required by this operation are assumed to be available, for example from [ThetaPrecomp](#) or from the output of the isogeny algorithms.

Algorithm 7.34 $\text{ThetaPrecomp}(0_A)$

Input: Theta null point $0_A := (a : b : c : d)$, and optionally the dual isogeneous theta null point $0'_A := (\alpha : \beta : \gamma : \delta)$.

Output: Auxiliary constants `consts` := $\{abc, abd, acd, bcd, \alpha^2\beta^2\gamma^2, \alpha^2\beta^2\delta^2, \alpha^2\gamma^2\delta^2, \beta^2\gamma^2\delta^2\}$.

```

1: if  $0'_A$  then
2:    $(\alpha^2, \beta^2, \gamma^2, \delta^2) \leftarrow \mathcal{S}(\alpha, \beta, \gamma, \delta)$ 
3: else
4:    $(\alpha^2, \beta^2, \gamma^2, \delta^2) \leftarrow \mathcal{H} \circ \mathcal{S}(a, b, c, d)$ 
5: end if
6:  $t_1 \leftarrow \alpha^2 \cdot \beta^2$ 
7:  $t_2 \leftarrow \gamma^2 \cdot \delta^2$ 
8:  $\alpha^2\beta^2\gamma^2 \leftarrow t_1 \cdot \gamma^2$ 
9:  $\alpha^2\beta^2\delta^2 \leftarrow t_1 \cdot \delta^2$ 
10:  $\alpha^2\gamma^2\delta^2 \leftarrow t_2 \cdot \alpha^2$ 
11:  $\beta^2\gamma^2\delta^2 \leftarrow t_2 \cdot \beta^2$ 
12:  $t_1 \leftarrow a \cdot b$ 
13:  $t_2 \leftarrow c \cdot d$ 
14:  $abc \leftarrow t_1 \cdot c$ 
15:  $abd \leftarrow t_1 \cdot d$ 
16:  $acd \leftarrow t_2 \cdot a$ 
17:  $bcd \leftarrow t_2 \cdot b$ 
18: consts  $\leftarrow \{abc, abd, acd, bcd, \alpha^2\beta^2\gamma^2, \alpha^2\beta^2\delta^2, \alpha^2\gamma^2\delta^2, \beta^2\gamma^2\delta^2\}$ 
19: return consts

```

// Total cost: 4S + 12M(+8a)

Algorithm 7.35 $\text{ThetaDBL}(P, \text{consts})$

Input: The theta coordinates of P on A with theta null point $0_A := (a : b : c : d)$, and the auxiliary constants `consts` := $\text{ThetaPrecomp}(0_A)$.

Output: The theta coordinates of the point $[2]P$.

```

1:  $x_P, y_P, z_P, w_P \leftarrow P$ 
2:  $c_1, c_2, c_3, c_4, c_5, c_6, c_7, c_8 \leftarrow \text{consts}$ 
3:  $X_{2P}, Y_{2P}, Z_{2P}, W_{2P} \leftarrow \mathcal{S} \circ \mathcal{H} \circ \mathcal{S}(x_P, y_P, z_P, w_P)$ 
4:  $X_{2P} \leftarrow X_{2P} \cdot c_8$ 
5:  $Y_{2P} \leftarrow Y_{2P} \cdot c_7$ 
6:  $Z_{2P} \leftarrow Z_{2P} \cdot c_6$ 
7:  $W_{2P} \leftarrow W_{2P} \cdot c_5$ 
8:  $X_{2P}, Y_{2P}, Z_{2P}, W_{2P} \leftarrow \mathcal{H}(X_{2P}, Y_{2P}, Z_{2P}, W_{2P})$ 
9:  $X_{2P} \leftarrow X_{2P} \cdot c_4$ 
10:  $Y_{2P} \leftarrow Y_{2P} \cdot c_3$ 
11:  $Z_{2P} \leftarrow Z_{2P} \cdot c_2$ 
12:  $W_{2P} \leftarrow W_{2P} \cdot c_1$ 
13: return  $(X_{2P} : Y_{2P} : Z_{2P} : W_{2P})$ 

```

// Total cost: 8S + 8M + 16a

7.5.3. Generic $(2, 2)$ -isogeny computation

In this section, We first describe how to compute a generic $(2, 2)$ -isogeny $\Phi : A \rightarrow B$ given torsion points lying above $\ker(\Phi)$ and then introduce the algorithm for calculating image points of a given point. Two points $T_1, T_2 \in A[2^k]$ (for some $k > 1$) are said to lie above the kernel of Φ if $\ker(\Phi) = \langle [2^{k-1}]T_1 \rangle \oplus \langle [2^{k-1}]T_2 \rangle$. For the subsequent formulas to be valid, we require that $\ker(\Phi)$ be compatible with the theta coordinate system on A . In a chain of $(2, 2)$ -isogeny, it is

enough to ensure this compatibility at the first step, as it will then be maintained throughout the chain. More details on this condition can be found in [DMP+24, Sections 2.3 and 4].

7.5.3.1 With isotropic 8-torsion lying above the kernel

[GenericCodomainWith8Torsion](#) (Algorithm 7.36) computes the codomain of a gluing $(2, 2)$ -isogeny in the generic case where all coordinates of the dual isogeneous theta null point are non-zero. It takes as input the theta coordinates of two 8-torsion points T_1'', T_2'' such that $\ker(\Phi) = \langle [4]T_1'' \rangle \oplus \langle [4]T_2'' \rangle$ and outputs:

- the dual isogeneous theta null point $(\alpha : \beta : \gamma : \delta)$,
- its projective inverse $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$, and
- the theta null point 0_B on the codomain B .

After applying the composite transformation $\mathcal{H} \circ \mathcal{S}$ to both input points, the algorithm computes the required theta constants through a sequence of multiplications. The total computational cost, excluding verification steps, is $8\mathbf{S} + 9\mathbf{M} + 24\mathbf{a}$.

Two types of failure can occur when the algorithm is used repeatedly in a $(2, 2)$ -isogeny chain:

1. **Unexpected splitting:** if any coordinate of $(\alpha : \beta : \gamma : \delta)$ becomes zero (checked in Line 5), the isogeny splits in the middle, i.e., the codomain is a product of elliptic curves before the end of the chain.
2. **Non-isotropic or incompatible points:** the input 4-torsion points $[2]T_1'', [2]T_2''$ must be isotropic and compatible with the theta structure. This is verified by checking that the images of T_1'', T_2'' under the isogeny Φ have the specific forms $(x : x : y : y)$ and $(z : w : z : w)$, respectively, which is done by calling [CheckIsotropic](#). This ensures that the points have order 4, lie above the correct 2-torsion points on the codomain, and are compatible with the theta structure of the codomain.

To bootstrap the chain, the initial gluing kernel is checked to be non-diagonal and isotropic (Line 1 of [ThetaChangeOfBasis](#)). Moreover, [ThetaChangeOfBasis](#) guarantees that the 4-torsion points are compatible with the initial theta structure.

Because the first failure occurs with heuristic probability $O(1/p)$, the checks are skipped during signing. Consequently, the related algorithms [GenericCodomainWith4Torsion](#) and [GenericCodomain](#) (used only in signing) do not include these checks. Also, we can ignore the isotropy checks for signing, because we control the kernels in the signing procedure. However, they must be performed during verification to ensure that the signature has not been malformed.

7.5.3.2 With isotropic 4-torsion lying above the kernel

[GenericCodomainWith4Torsion](#) (Algorithm 7.38) describe how to compute Φ with only isotropic 4-torsion T_1', T_2' lying above the kernel. It takes input as theta constants of T_1' of order 4 such that $[2]T_1' \in \ker(\Phi)$ and the theta null point 0_A of A . However, it is much more costly than [GenericCodomainWith8Torsion](#) as it involves two square roots.

7.5.3.3 With kernel generators only

[GenericCodomain](#) (Algorithm 7.39) describe how to compute Φ with only generators T_1, T_2 of $\ker(\Phi)$. It takes input as theta constants 0_A of A without need of the points T_1 and T_2 .

7.5.4. Generic (2,2)-isogeny evaluation

If we have the theta null point 0_B of B and the dual isogeneous theta null point $I = (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$ (these information can be obtained by [GenericCodomainWith8Torsion](#), [GenericCodomainWith4Torsion](#) or [GenericCodomain](#)), We can compute the theta coordinates of the image point $\Phi(P)$ of a point $P \in A$ using [GenericEval](#) (Algorithm 7.40).

7.5.5. Gluing (2,2)-isogeny computation

We now describe how to compute a gluing (2,2)-isogeny $\Phi : E_1 \times E_2 \rightarrow A$, where E_1, E_2 are elliptic curves and A is a principally polarized abelian surface.

Algorithm 7.36 [GenericCodomainWith8Torsion](#)(T_1'', T_2'')

Input: Theta coordinates of T_1'' and T_2'' , such that $\ker(\Phi) = \langle [4]T_1'' \rangle \oplus \langle [4]T_2'' \rangle$.

Output: Dual isogeneous theta null point $(\alpha : \beta : \gamma : \delta)$, the inverse of the dual projective isogeneous theta null point $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$, and the theta null point 0_B on B . // Case $\alpha \cdot \beta \cdot \gamma \cdot \delta \neq 0$.

```

1:  $x_{T_1''}, y_{T_1''}, z_{T_1''}, w_{T_1''} \leftarrow T_1''$ 
2:  $x_{T_2''}, y_{T_2''}, z_{T_2''}, w_{T_2''} \leftarrow T_2''$ 
3:  $(x\alpha, x\beta, y\gamma, y\delta) \leftarrow \mathcal{H} \circ \mathcal{S}(x_{T_1''}, y_{T_1''}, z_{T_1''}, w_{T_1''})$ 
4:  $(z\alpha, w\beta, z\gamma, w\delta) \leftarrow \mathcal{H} \circ \mathcal{S}(x_{T_2''}, y_{T_2''}, z_{T_2''}, w_{T_2''})$ 
5: if  $0 \in \{x\alpha, x\beta, z\alpha, w\beta, z\gamma, w\delta\}$  then
6:   raise Exception: ("GenericCodomainWith8Torsion failed: unexpected splitting") // Check only done in
   verification.
7: end if
8:  $x\alpha w\beta \leftarrow x\alpha \cdot w\beta$ 
9:  $z\alpha x\beta \leftarrow z\alpha \cdot x\beta$ 
10:  $\alpha \leftarrow z\alpha \cdot x\alpha w\beta$ 
11:  $\beta \leftarrow w\beta \cdot z\alpha x\beta$ 
12:  $\gamma \leftarrow z\gamma \cdot x\alpha w\beta$ 
13:  $\delta \leftarrow w\delta \cdot z\alpha x\beta$ 
14:  $z\gamma w\delta \leftarrow z\gamma \cdot w\delta$ 
15:  $\alpha^{-1} \leftarrow x\beta \cdot z\gamma w\delta$ 
16:  $\beta^{-1} \leftarrow x\alpha \cdot z\gamma w\delta$ 
17:  $\gamma^{-1} \leftarrow \delta$ 
18:  $\delta^{-1} \leftarrow \gamma$ 
19:  $P \leftarrow (x\alpha : x\beta : y\gamma : y\delta)$ 
20:  $Q \leftarrow (z\alpha : w\beta : z\gamma : w\delta)$ 
21:  $I \leftarrow (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$ 
22: if  $\neg \text{CheckIsotropic}(P, Q, I)$  then
23:   raise Exception: ("GenericCodomainWith8Torsion failed:  $P, Q$  not isotropic") // Check only done in
   verification.
24: end if
25:  $(a_2, b_2, c_2, d_2) \leftarrow \mathcal{H}(\alpha, \beta, \gamma, \delta)$ 
26:  $0_B \leftarrow (a_2 : b_2 : c_2 : d_2)$ 
27: return  $(\alpha : \beta : \gamma : \delta), (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1}), 0_B$  // Total cost (without checks): 8S + 9M + 24a

```

The algorithm [GluingCodomain](#) takes as input two 8-torsion points T_1'', T_2'' on $E_1 \times E_2$ and computes the codomain A of the gluing (2,2)-isogeny $\Phi : E_1 \times E_2 \rightarrow A$, along with associated theta data. It first doubles the input points, then transforms them into theta coordinates via a change-of-basis matrix using [ProductToTheta](#). After applying the transformation $\mathcal{H} \circ \mathcal{S}$, the algorithm verifies the coordinate conditions (e.g., the last coordinate being zero, non-zero projective factors) and checks the isotropy of the input points. Upon passing these checks, it computes the dual isogeneous theta null point, its inverse, the theta null point on A , and returns these together with the change-of-basis matrix. Due to the way we set up our theta structure in Algorithm 7.42, we will always have $\delta = 0$ and this condition together with the verification steps in the algorithm guarantees the correctness of the algorithm.

7.5.6. Gluing (2,2)-isogeny evaluation

With the outputs of [GluingCodomain](#), we can compute the image point $\Phi(P)$ of a point $P \in E_1 \times E_2$ using [GluingEval](#) (Algorithm 7.51). Moreover, when P is of the form $P = (P_1, 0)$ or $(0, P_2)$, we use [GluingEvalSpecial](#) (see Algorithm 7.52) for efficiency.

Algorithm 7.37 [CheckIsotropic](#)(P, Q, I)

Input: Theta coordinates of $P := \mathcal{H} \circ \mathcal{S}(T_1'')$ and $Q := \mathcal{H} \circ \mathcal{S}(T_2'')$, where points T_1'', T_2'' are as in [GenericCodomainWith8Torsion](#), and theta coordinates of $I := (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$.

Output: A boolean `bool` indicating whether the 4-torsion points $\Phi(T_1''), \Phi(T_2'')$ are isotropic (i.e., $e_4(\Phi(T_1''), \Phi(T_2'')) = 1$).

```

1:  $x_P, y_P, z_P, w_P \leftarrow P$ 
2:  $x_Q, y_Q, z_Q, w_Q \leftarrow Q$ 
3:  $\alpha^{-1}, \beta^{-1}, \gamma^{-1}, \delta^{-1} \leftarrow I$ 
4:  $t_1 \leftarrow x_P \cdot \alpha^{-1}$ 
5:  $t_2 \leftarrow y_P \cdot \beta^{-1}$ 
6: if  $t_1 \neq t_2$  then
7:   return false
8: end if
9:  $t_1 \leftarrow z_P \cdot \gamma^{-1}$ 
10:  $t_2 \leftarrow t_P \cdot \delta^{-1}$ 
11: if  $t_1 \neq t_2$  then
12:   return false
13: end if
14:  $t_1 \leftarrow x_Q \cdot \alpha^{-1}$ 
15:  $t_2 \leftarrow z_Q \cdot \gamma^{-1}$ 
16: if  $t_1 \neq t_2$  then
17:   return false
18: end if
19:  $t_1 \leftarrow y_Q \cdot \beta^{-1}$ 
20:  $t_2 \leftarrow t_Q \cdot \delta^{-1}$ 
21: if  $t_1 \neq t_2$  then
22:   return false
23: end if
24: return true

```

// Total cost: at most 8M

[GluingEval](#) takes as input the point $P \in E_1 \times E_2$, the 8-torsion point T_1'' such that $[4]T_1'' \in \ker(\Phi)$, the point $J = (x : x : y : y)$, the dual of the theta point $\Phi(T_1'')$, and the change-of-basis matrix computed during [GluingCodomain](#), and outputs the theta coordinates of $\Phi(P)$.

[GluingEvalSpecial](#) takes as input a point $P \in E_1 \times E_2$ of the form $(P_1, 0)$ or $(0, P_2)$, the point $I = (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : 0)$, the “inverse” of the dual isogeneous theta null point on A and outputs the theta point $\Phi(P)$.

7.5.7. Splitting change of coordinates

Let E_1, E_2 be elliptic curves and A be a principally polarized abelian surface. To compute a splitting (2,2)-isogeny $\Phi : A \rightarrow E_1 \times E_2$, we first compute a (2,2)-isogeny from A to B where $B \cong E_1 \times E_2$ using [GenericCodomainWith8Torsion](#) (Algorithm 7.36), [GenericCodomainWith4Torsion](#) (Algorithm 7.38) or [GenericCodomain](#) (Algorithm 7.39). Then, we use [SplittingIsomorphism](#) (Algorithm 7.46) to compute the isomorphism whose action on 0_B gives back the product theta structure $E_1 \times E_2$. Algorithm [GetIndexSplitting](#) (Algorithm 7.45) determines the index (i, j) such that $U_{i,j}(0) = 0$, where $U_{i,j}$ are level 4 theta coordinates. The function χ^1 used

¹We use the notation χ as it is a character.

in it is defined as follows:

$$\chi(i, j) := \begin{cases} 1, & \text{if } (i, j) \in \{(0, 0), (0, 1), (0, 2), (0, 3), (1, 0), (1, 2), (2, 0), (2, 1), (3, 0), (3, 3)\}, \\ -1, & \text{if } (i, j) \in \{(1, 1), (1, 3), (2, 2), (2, 3), (3, 1), (3, 2)\} \end{cases}.$$

After computing a splitting $(2, 2)$ -isogeny $\Phi : A \rightarrow B$ such that $B \cong E_1 \times E_2$, we recover the Montgomery coefficients of E_1, E_2 from the theta null point 0_B of B using [ThetaToProduct](#)(Algorithm 7.48). We convert our points from theta product coordinates to Montgomery coordinates using [ThetaProductPointToMontgomery](#)(Algorithm 7.49).

To eliminate potential information leakage, the output matrix of [SplittingIsomorphism](#) is randomized. This randomization is based on the property of Montgomery curves: given a curve E and its corresponding theta null point, the representations of the other five isomorphic curves can be obtained by a change of variables. For randomization, we simply multiply the output matrix by one of six precomputed change-of-variables matrices, chosen at random. This step (see [RandomizedProduct](#)) ensures that the coordinates obtained after the complete $(2, 2)$ -isogeny chain do not reveal information about the underlying one-dimensional isogenies.

Algorithm 7.38 [GenericCodomainWith4Torsion](#)($T'_1, 0_A$)

Input: Theta coordinates of T'_1 of order 4 such that $[2]T'_1 \in \ker(\Phi)$ and the theta null point $0_A := (a : b : c : d)$ of A .

Output: Dual isogeneous theta null point $(\alpha : \beta : \gamma : \delta)$, the inverse of the dual projective isogeneous theta null point $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$, and the theta null point 0_B on B . // Case $\alpha \cdot \beta \cdot \gamma \cdot \delta \neq 0$.

```

1:  $(x\alpha\beta, x\gamma\delta, \_) \leftarrow \mathcal{H} \circ \mathcal{S}(x_{T'_1}, y_{T'_1}, z_{T'_1}, w_{T'_1})$ 
2:  $(\alpha^2, \beta^2, \gamma^2, \delta^2) \leftarrow \mathcal{H} \circ \mathcal{S}(a, b, c, d)$ 
3:  $\alpha\beta \leftarrow \text{SquareRoot}(\alpha^2 \cdot \beta^2)$ 
4:  $\alpha\gamma \leftarrow \text{SquareRoot}(\alpha^2 \cdot \gamma^2)$ 
5:  $\beta \leftarrow \alpha\beta \cdot \alpha\gamma$ 
6:  $\delta^{-1} \leftarrow \beta \cdot x\gamma\delta$ 
7:  $\beta \leftarrow \beta \cdot x\alpha\beta$ 
8:  $\delta \leftarrow x\gamma\delta \cdot \alpha\beta \cdot \alpha^2$ 
9:  $\alpha \leftarrow x\alpha\beta \cdot \alpha^2$ 
10:  $\gamma \leftarrow \alpha \cdot \gamma^2$ 
11:  $\alpha \leftarrow \alpha \cdot \alpha\gamma$ 
12:  $\alpha^{-1} \leftarrow x\alpha\beta \cdot \delta^2$ 
13:  $\gamma^{-1} \leftarrow \alpha^{-1} \cdot \beta^2$ 
14:  $\alpha^{-1} \leftarrow \alpha^{-1} \cdot \gamma^2$ 
15:  $\beta^{-1} \leftarrow \alpha^{-1} \cdot \alpha\beta$ 
16:  $\alpha^{-1} \leftarrow \alpha^{-1} \cdot \beta^2$ 
17:  $\gamma^{-1} \leftarrow \gamma^{-1} \cdot \alpha\gamma$ 
18:  $\delta^{-1} \leftarrow \delta^{-1} \cdot \beta^2$ 
19:  $(a_2, b_2, c_2, d_2) \leftarrow \mathcal{H}(\alpha, \beta, \gamma, \delta)$ 
20:  $0_B \leftarrow (a_2 : b_2 : c_2 : d_2)$ 
21: return  $(\alpha, \beta, \gamma, \delta), (\alpha^{-1}, \beta^{-1}, \gamma^{-1}, \delta^{-1}), 0_B$ 

```

// Total cost: 8S + 17M + 24a + 2Sqrt

7.5.8. Computing a $(2, 2)$ -isogeny chain between products of elliptic curves

- [Isogeny22Chain](#) (Algorithm 7.50) takes input as points $P, Q \in E_1 \times E_2$ of order 2^e , a positive integer e , and an array `pts` containing points on $E_1 \times E_2$ with at least one point zero, outputs the codomain $E_3 \times E_4$ of the $(2, 2)$ -isogeny chain $\Phi = \Phi_e \circ \dots \circ \Phi_1$ such that $\ker(\Phi) = \langle P, Q \rangle$ with $\deg(\Phi_k) = 2$ for all $k = 1, \dots, e$, and the evaluated points $[\Phi(P) \mid P \in \text{pts}]$, using a balanced strategy.
- [Isogeny22ChainWithTorsion](#)(Algorithm 7.53) takes input as points $P, Q \in E_1 \times E_2$ of order 2^{e+2} , Montgomery coefficients A_1, A_2 of E_1, E_2 respectively, a positive even integer e , and an

Algorithm 7.39 GenericCodomain(0_A)**Input:** The theta constants $0_A := (a : b : c : d)$ of A .**Output:** Dual isogeneous theta null point $(\alpha : \beta : \gamma : \delta)$, the inverse of the dual projective isogeneous theta null point $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$, and the theta null point 0_B on B . // Case $\alpha \cdot \beta \cdot \gamma \cdot \delta \neq 0$.

```

1:  $(\alpha^2, \beta^2, \gamma^2, \delta^2) \leftarrow \mathcal{H} \circ \mathcal{S}(a, b, c, d)$ 
2:  $\alpha \leftarrow \alpha^2$ 
3:  $\beta \leftarrow \alpha^2 \cdot \beta^2$ 
4:  $\gamma \leftarrow \alpha^2 \cdot \gamma^2$ 
5:  $\delta \leftarrow \alpha^2 \cdot \delta^2$ 
6:  $\beta \leftarrow \text{SquareRoot}(\beta)$ 
7:  $\gamma \leftarrow \text{SquareRoot}(\gamma)$ 
8:  $\delta \leftarrow \text{SquareRoot}(\delta)$ 
9:  $\alpha^{-1} \leftarrow \gamma^2 \cdot \delta^2$ 
10:  $\beta^{-1} \leftarrow \alpha^{-1} \cdot \beta^2$ 
11:  $\alpha^{-1} \leftarrow \alpha^{-1} \cdot \beta^2$ 
12:  $\gamma^{-1} \leftarrow \delta^2 \cdot \beta^2 \cdot \gamma$ 
13:  $\delta^{-1} \leftarrow \gamma^2 \cdot \beta^2 \cdot \delta$ 
14:  $(a_2, b_2, c_2, d_2) \leftarrow \mathcal{H}(\alpha, \beta, \gamma, \delta)$ 
15:  $0_B \leftarrow (a_2 : b_2 : c_2 : d_2)$ 
16: return  $(\alpha, \beta, \gamma, \delta), (\alpha^{-1}, \beta^{-1}, \gamma^{-1}, \delta^{-1}), 0_B$  // Total cost:  $4S + 10M + 24a + 3Sqrt$ 

```

Algorithm 7.40 GenericEval(P, I)**Input:** Theta coordinates of P and the inverse of the dual isogeneous theta null point $I := (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : \delta^{-1})$ on B .**Output:** Theta coordinates of $P' := \Phi(P)$. // Case $\beta \cdot \gamma \cdot \delta \neq 0$

```

1:  $x_P, y_P, z_P, w_P \leftarrow P$ 
2:  $\alpha^{-1}, \beta^{-1}, \gamma^{-1}, \delta^{-1} \leftarrow I$ 
3:  $(X_P, Y_P, Z_P, W_P) \leftarrow \mathcal{H} \circ \mathcal{S}(x_P, y_P, z_P, w_P)$ 
4:  $X' \leftarrow \alpha^{-1} \cdot X_P$ 
5:  $Y' \leftarrow \beta^{-1} \cdot Y_P$ 
6:  $Z' \leftarrow \gamma^{-1} \cdot Z_P$ 
7:  $W' \leftarrow \delta^{-1} \cdot W_P$ 
8:  $(x_{P'}, y_{P'}, z_{P'}, w_{P'}) \leftarrow \mathcal{H}(X', Y', Z', W')$ 
9:  $P' \leftarrow (x_{P'} : y_{P'} : z_{P'} : w_{P'})$ 
10: return  $P'$  // Total cost:  $4S + 4M + 16a$ 

```

Algorithm 7.41 ActionByTranslation(P, Q)

Input: Four-torsion points $P = (P_1, P_2)$ and $Q = (Q_1, Q_2)$ on $E_1 \times E_2$.**Output:** Array mats containing 2×2 matrices giving the action-of-translation by $P_1, Q_1 \in E_1$ and $P_2, Q_2 \in E_2$.

```

1:  $P' := (P'_1, P'_2) \leftarrow [2]P$  // Cost: 4S + 8M + 8a using xDBL
2:  $Q' := (Q'_1, Q'_2) \leftarrow [2]Q$  // Cost: 4S + 8M + 8a using xDBL
3: Write  $P'_i = (X_i^{(P)} : Z_i^{(P)})$  and  $Q'_i = (X_i^{(Q)} : Z_i^{(Q)})$  for  $i = 1, 2$ 
4: Write  $P_i = (U_i^{(P)} : W_i^{(P)})$  and  $Q_i = (U_i^{(Q)} : W_i^{(Q)})$  for  $i = 1, 2$ 
5: for  $i$  from 1 up to 2 do
6:    $WX_i^{(P)}, WZ_i^{(P)}, UX_i^{(P)}, UZ_i^{(P)} \leftarrow W_i^{(P)} \cdot Z_i^{(P)}, W_i^{(P)} \cdot Z_i^{(P)}, U_i^{(P)} \cdot X_i^{(P)}, U_i^{(P)} \cdot Z_i^{(P)}$ 
7:    $\delta_i^{(Q)} \leftarrow WX_i^{(Q)} - UZ_i^{(Q)}$ 
8:    $WX_i^{(Q)}, WZ_i^{(Q)}, UX_i^{(Q)}, UZ_i^{(Q)} \leftarrow W_i^{(Q)} \cdot Z_i^{(Q)}, W_i^{(Q)} \cdot Z_i^{(Q)}, U_i^{(Q)} \cdot X_i^{(Q)}, U_i^{(Q)} \cdot Z_i^{(Q)}$ 
9:    $\delta_i^{(Q)} \leftarrow WX_i^{(Q)} - UZ_i^{(Q)}$ 
10: end for
11: Compute the inverses of  $\delta_1^{(P)}, \delta_2^{(P)}, \delta_1^{(Q)}, \delta_2^{(Q)}, Z_1^{(P)}, Z_2^{(P)}, Z_1^{(Q)}, Z_2^{(Q)}$  using batched inversions // Cost:
    21M + 1I
12: Initialize array mats to store matrices
13:  $\text{pts} \leftarrow [P, Q]$ 
14: for  $i$  from 1 up to 2 do
15:   for  $j$  from 1 up to 2 do
16:      $R \leftarrow \text{pts}[j]$ 
17:      $M_{0,0} \leftarrow -U_i^{(R)} Z_i^{(R)} \cdot (\delta_i^{(R)})^{-1}$ 
18:      $M_{0,1} \leftarrow -W_i^{(R)} Z_i^{(R)} \cdot (\delta_i^{(R)})^{-1}$ 
19:      $M_{1,0} \leftarrow U_i^{(R)} X_i^{(R)} \cdot (\delta_i^{(R)})^{-1} - X_i^{(R)} \cdot (Z_i^{(R)})^{-1}$ 
20:      $M_{1,1} \leftarrow -M_{0,0}$ 
21:      $\mathbf{M} \leftarrow (M_{i,j})_{0 \leq i,j \leq 1}$ 
22:     Append  $\mathbf{M}$  to mats
23:   end for
24: end for
25: return mats // Total cost: 8S + 69M + 1I + 24a

```

Algorithm 7.42 ThetaChangeOfBasis(P, Q)

Input: The points $P = (P_1, P_2)$ and $Q = (Q_1, Q_2)$ in the four torsion of $E_1 \times E_2$ such that the kernel for the gluing isogeny is $\langle [2]P, [2]Q \rangle$.

Output: The 4×4 change of basis matrix $\mathbf{N}$.

```

1: if  $P_1, P_2, Q_1, Q_2$  do not have order 4 or  $[2]P_1 = [2]P_2$  or  $[2]Q_1 = [2]Q_2$  then
2:   raise ("ThetaChangeOfBasis failed: gluing kernel is diagonal or not isotropic.")
3: end if
4:  $\mathbf{G}, \mathbf{G}', \mathbf{H}, \mathbf{H}' \leftarrow \text{ActionByTranslation}(P, Q)$  // Cost: 4S + 61M + 1I + 16a
5:  $t_1 \leftarrow \mathbf{G}_{0,0} \cdot \mathbf{H}_{0,0} + \mathbf{G}_{0,1} \cdot \mathbf{H}_{1,0}$ 
6:  $t_2 \leftarrow \mathbf{G}_{1,0} \cdot \mathbf{H}_{0,0} + \mathbf{G}_{1,1} \cdot \mathbf{H}_{1,0}$ 
7:  $t_3 \leftarrow \mathbf{G}'_{0,0} \cdot \mathbf{H}'_{0,0} + \mathbf{G}'_{0,1} \cdot \mathbf{H}'_{1,0}$ 
8:  $t_4 \leftarrow \mathbf{G}'_{1,0} \cdot \mathbf{H}'_{0,0} + \mathbf{G}'_{1,1} \cdot \mathbf{H}'_{1,0}$ 
9:  $N_{0,0} \leftarrow \mathbf{G}_{0,0} \cdot \mathbf{G}'_{0,0} + \mathbf{H}_{0,0} \cdot \mathbf{H}'_{0,0} + t_1 \cdot t_3 + 1$ 
10:  $N_{0,1} \leftarrow \mathbf{G}_{0,0} \cdot \mathbf{G}'_{1,0} + \mathbf{H}_{0,0} \cdot \mathbf{H}'_{1,0} + t_1 \cdot t_4$ 
11:  $N_{0,2} \leftarrow \mathbf{G}_{1,0} \cdot \mathbf{G}'_{0,0} + \mathbf{H}_{1,0} \cdot \mathbf{H}'_{0,0} + t_2 \cdot t_3$ 
12:  $N_{0,3} \leftarrow \mathbf{G}_{1,0} \cdot \mathbf{G}'_{1,0} + \mathbf{H}_{1,0} \cdot \mathbf{H}'_{1,0} + t_2 \cdot t_4$ 
13:  $N_{1,0} \leftarrow \mathbf{H}'_{0,0} \cdot N_{0,0} + \mathbf{H}'_{0,1} \cdot N_{0,1}$ 
14:  $N_{1,1} \leftarrow \mathbf{H}'_{1,0} \cdot N_{0,0} + \mathbf{H}'_{1,1} \cdot N_{0,1}$ 
15:  $N_{1,2} \leftarrow \mathbf{H}'_{0,0} \cdot N_{0,2} + \mathbf{H}'_{0,1} \cdot N_{0,3}$ 
16:  $N_{1,3} \leftarrow \mathbf{H}'_{1,0} \cdot N_{0,2} + \mathbf{H}'_{1,1} \cdot N_{0,3}$ 
17:  $N_{2,0} \leftarrow \mathbf{G}_{0,0} \cdot N_{0,0} + \mathbf{G}_{0,1} \cdot N_{0,2}$ 
18:  $N_{2,1} \leftarrow \mathbf{G}_{0,0} \cdot N_{0,1} + \mathbf{G}_{0,1} \cdot N_{0,3}$ 
19:  $N_{2,2} \leftarrow \mathbf{G}_{1,0} \cdot N_{0,0} + \mathbf{G}_{1,1} \cdot N_{0,2}$ 
20:  $N_{2,3} \leftarrow \mathbf{G}_{1,0} \cdot N_{0,1} + \mathbf{G}_{1,1} \cdot N_{0,3}$ 
21:  $N_{3,0} \leftarrow \mathbf{G}_{0,0} \cdot N_{1,0} + \mathbf{G}_{0,1} \cdot N_{1,2}$ 
22:  $N_{3,1} \leftarrow \mathbf{G}_{0,0} \cdot N_{1,1} + \mathbf{G}_{0,1} \cdot N_{1,3}$ 
23:  $N_{3,2} \leftarrow \mathbf{G}_{1,0} \cdot N_{1,0} + \mathbf{G}_{1,1} \cdot N_{1,2}$ 
24:  $N_{3,3} \leftarrow \mathbf{G}_{1,0} \cdot N_{1,1} + \mathbf{G}_{1,1} \cdot N_{1,3}$ 
25:  $\mathbf{N} \leftarrow (N_{i,j})_{0 \leq i,j \leq 3}$ 
26: return  $\mathbf{N}$  // Total cost: 8S + 113M + 1I + 49a

```

Algorithm 7.43 ProductToTheta($pts, \mathbf{N}$)

Input: List of points pts , where for $P \in pts$ we have $P := (P_1, P_2) \in E_1 \times E_2$, and $\mathbf{N}$ the change-of-basis matrix computed using ThetaChangeOfBasis.

Output: The corresponding point P' in theta coordinates on $A \cong E_1 \times E_2$.

```

1:  $\text{eval\_pts} \leftarrow []$ 
2:  $L \leftarrow \#pts$ 
3: for  $j$  from 1 up to  $L$  do
4:    $P = (P_1, P_2) \leftarrow pts[j]$ 
5:   Write  $P_i = (\theta_{0,i} : \theta_{1,i})$  for  $i = 1, 2$ .
6:    $x \leftarrow \theta_{0,1} \cdot \theta_{0,2}$ 
7:    $y \leftarrow \theta_{0,1} \cdot \theta_{1,2}$ 
8:    $z \leftarrow \theta_{1,1} \cdot \theta_{0,2}$ 
9:    $w \leftarrow \theta_{1,1} \cdot \theta_{1,2}$ 
10:   $P' \leftarrow \mathbf{N} \cdot (x : y : z : w)$  // Cost: at most 20M + 12a
11:  Append  $P'$  to  $\text{eval\_pts}$ 
12: end for
13: return  $\text{eval\_pts}$  // Total cost: +20kM + 12ka with  $k = \#pts$ 

```

array **pts** containing points on $E_1 \times E_2$ with at least one point zero, outputs the codomain $E_3 \times E_4$ of the $(2, 2)$ -isogeny chain $\Phi = \Phi_e \circ \dots \circ \Phi_1$ such that $\ker(\Phi) = \langle P, Q \rangle$ with $\deg(\Phi_k) = 2$ for all $k = 1, \dots, e$, and the evaluated points $[\Phi(P) \mid P \in \text{pts}]$. Thus, the $(2, 2)$ -isogeny chain can be computed without square roots.

Algorithm 7.44 GluingCodomain(T_1'', T_2'')

Input: 8-torsion points $T_1'', T_2'' \in E_1 \times E_2$, and $\ker(\Phi) = \langle [4]T_1'' \rangle \oplus \langle [4]T_2'' \rangle$.

Output: Dual isogeneous theta null point $(\alpha : \beta : \gamma : 0)$, the “inverse” of the dual isogeneous theta null point $(\alpha^{-1} : \beta^{-1} : \gamma^{-1} : 0)$, the theta null point 0_A on A , the dual isogeneous theta point $\Phi(T_1'')$ on A , and the change-of-basis matrix $\mathbf{N}$ (re-used for point evaluation). // Case $\delta = 0$

```

1:  $T_1' \leftarrow [2](T_1'')$  // Cost: 2S + 4M + 4a using xDBL
2:  $T_2' \leftarrow [2](T_2'')$  // Cost: 2S + 4M + 4a using xDBL
3:  $\mathbf{N} \leftarrow \text{ThetaChangeOfBasis}(T_1', T_2')$  // Total cost: 8S + 113M + 1I + 49a
4:  $[P_1, P_2] \leftarrow \text{ProductToTheta}([T_1'', T_2''], \mathbf{N})$  // Total cost: 40M + 24a
5:  $x_1, y_1, z_1, t_1 \leftarrow P_1$ 
6:  $x_2, y_2, z_2, t_2 \leftarrow P_2$ 
7:  $(X_1, Y_1, Z_1, T_1) \leftarrow \mathcal{H} \circ \mathcal{S}(x_1, y_1, z_1, t_1)$ 
8:  $(X_2, Y_2, Z_2, T_2) \leftarrow \mathcal{H} \circ \mathcal{S}(x_2, y_2, z_2, t_2)$ 
9: if  $T_1 \neq 0$  or  $T_2 \neq 0$  then
10:   raise (“GluingCodomain failed: last coordinate is non-zero”)
11: end if
12: if  $X_1, X_2, Y_1, Z_2 = 0$  then
13:   raise (“GluingCodomain failed: projective factors are zero”)
14: end if
15:  $\alpha \leftarrow X_1 \cdot X_2$ 
16:  $\beta \leftarrow Y_1 \cdot X_2$ 
17:  $\gamma \leftarrow X_1 \cdot Z_2$ 
18:  $\alpha^{-1} \leftarrow Y_1 \cdot Z_2$ 
19:  $\beta^{-1} \leftarrow \gamma$ 
20:  $\gamma^{-1} \leftarrow \beta$ 
21:  $x \leftarrow X_1 \cdot \alpha^{-1}$ 
22:  $y \leftarrow Z_1 \cdot \gamma^{-1}$ 
23:  $J \leftarrow (x : x : y : y)$ 
24: if  $(Y_1 \cdot \beta^{-1} \neq x)$  or  $(X_2 \cdot \alpha^{-1} \neq Y_2 \cdot \beta^{-1})$  then
25:   raise Exception: (“GluingCodomain failed:  $[2]T_1'', [2]T_2''$  not isotropic”) // Check only done in verification.
26: end if
27:  $(a_2, b_2, c_2, d_2) \leftarrow \mathcal{H}(\alpha, \beta, \gamma, 0)$ 
28:  $0_A \leftarrow (a_2 : b_2 : c_2 : d_2)$ 
29: return  $(\alpha : \beta : \gamma : 0), (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : 0), 0_A, J, \mathbf{N}$  // Total cost (without checks): 12S + 159M + 1I + 105a

```

Algorithm 7.45 GetIndexSplitting(0_A)

Input: Theta null point 0_A of theta structure $A \cong E_1 \times E_2$.

Output: Index (i, j) such that $U_{i,j}(0) = 0$.

```

1: inds  $\leftarrow \{(0, 0), (0, 1), (0, 2), (0, 3), (1, 0), (1, 2), (2, 0), (2, 1), (3, 0), (3, 3)\}$ 
2: for  $k$  from 1 up to #inds do
3:    $(i, j) \leftarrow \text{inds}[k]$ 
4:    $U \leftarrow \sum_{k=0}^3 \chi(i, k) \cdot 0_A[k] \cdot 0_A[j \oplus k]$  // Here,  $\oplus$  denotes bitwise XOR.
5:   if  $U = 0$  then
6:     return  $(i, j)$ 
7:   end if
8: end for

```

Algorithm 7.46 SplittingIsomorphism(0_A)

Input: Theta null point 0_A of $A \cong E_1 \times E_2$.

Output: Isomorphism given by matrix M , whose action on 0_A gives back the theta null point associated with the product theta structure.

```

1:  $(i, j) \leftarrow \text{GetIndexSplitting}(0_A)$ 
2: if  $(i, j) = (00, 00)$  then
3:    $M \leftarrow \begin{pmatrix} 1 & \sqrt{-1} & 1 & \sqrt{-1} \\ 1 & -\sqrt{-1} & -1 & \sqrt{-1} \\ 1 & \sqrt{-1} & -1 & -\sqrt{-1} \\ -1 & \sqrt{-1} & -1 & \sqrt{-1} \end{pmatrix}$ 
4: else if  $(i, j) = (01, 00)$  then
5:    $M \leftarrow \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 \\ -1 & 1 & -1 & 1 \end{pmatrix}$ 
6: else if  $(i, j) = (10, 00)$  then
7:    $M \leftarrow \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \\ -1 & -1 & 1 & 1 \end{pmatrix}$ 
8: else if  $(i, j) = (11, 00)$  then
9:    $M \leftarrow \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ -1 & 1 & 1 & -1 \end{pmatrix}$ 
10: else if  $(i, j) = (00, 01)$  then
11:    $M \leftarrow \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \end{pmatrix}$ 
12: else if  $(i, j) = (10, 01)$  then
13:    $M \leftarrow \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 \end{pmatrix}$ 
14: else if  $(i, j) = (00, 10)$  then
15:    $M \leftarrow \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -1 & 0 \end{pmatrix}$ 
16: else if  $(i, j) = (01, 10)$  then
17:    $M \leftarrow \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$ 
18: else if  $(i, j) = (00, 11)$  then
19:    $M \leftarrow \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$ 
20: else if  $(i, j) = (11, 11)$  then
21:    $M \leftarrow \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ 
22: end if
23:  $M \leftarrow \text{RandomizedProduct}(M)$  // Only done in signing.
24: return  $M$ 

```

Algorithm 7.47 RandomizedProduct(**M**)**Input:** The splitting matrix **M**.**Output:** The randomized matrix **N**.

```

1:  $\mathcal{M} \leftarrow \left\{ \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}, \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}, \begin{pmatrix} 1 & -1 & 1 & 1 \\ -1 & 1 & -1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & 1 \end{pmatrix}, \right.$ 
 $\left. \begin{pmatrix} -1 & \sqrt{-1} & \sqrt{-1} & 1 \\ \sqrt{-1} & -1 & 1 & \sqrt{-1} \\ \sqrt{-1} & 1 & -1 & \sqrt{-1} \\ 1 & \sqrt{-1} & \sqrt{-1} & -1 \end{pmatrix}, \begin{pmatrix} 1 & \sqrt{-1} & \sqrt{-1} & -1 \\ \sqrt{-1} & -1 & 1 & \sqrt{-1} \\ \sqrt{-1} & 1 & -1 & \sqrt{-1} \\ -1 & \sqrt{-1} & \sqrt{-1} & 1 \end{pmatrix} \right\}$ 
2: Sample a random  $\mathbf{N} \in \mathcal{M}$ 
3:  $\mathbf{N} \leftarrow \mathbf{N} \cdot \mathbf{M}$ 
4: return  $\mathbf{N}$ 

```

Algorithm 7.48 ThetaToProduct(0_A)**Input:** The theta null point $0_A := (a : b : c : d)$ of PPAS A with theta product structure.**Output:** The Montgomery coefficients of E_1, E_2 , namely $(A_{24}^{(1)} : C_{24}^{(1)})$ and $(A_{24}^{(2)} : C_{24}^{(2)})$, such that $A \cong E_1 \times E_2$.

```

1:  $(a, b, c, d) \leftarrow 0_A$ 
2:  $t_1 \leftarrow ad$ 
3:  $t_2 \leftarrow bc$ 
4: if  $t_1 \neq t_2$  then
5:   raise ("ThetaToProduct failed:  $0_A$  does not come from a product theta structure")
6: end if
7:  $x \leftarrow a^4$ 
8:  $y \leftarrow b^4$ 
9:  $A_2 \leftarrow x + y$ 
10:  $C_2 \leftarrow x - y$ 
11:  $A_2 \leftarrow -2A_2$ 
12:  $z \leftarrow c^4$ 
13:  $A_1 \leftarrow x + z$ 
14:  $C_1 \leftarrow x - z$ 
15:  $A_1 \leftarrow -2A_1$ 
16: if  $C_1 = 0$  or  $C_2 = 0$  then
17:   raise ("ThetaToProduct failed")
18: end if
19: return  $(A_1 : C_1), (A_2 : C_2)$ 

```

Algorithm 7.49 ThetaProductPointToMontgomery($P, 0_A$)**Input:** A theta point $P := (x : y : z : t)$ and the theta null point $0_A := (a : b : c : d)$ of PPAS A with theta product structure.**Output:** The Montgomery coordinates $(X(P_1) : Z(P_1))$ and $(X(P_2) : Z(P_2))$ of $P = (P_1, P_2) \in E_1 \times E_2$.

```

1:  $(a, b, c, d) \leftarrow 0_A$ 
2:  $(x, y, z, t) \leftarrow P$ 
3:  $X_1 \leftarrow a \cdot z + c \cdot x$ 
4:  $Z_1 \leftarrow a \cdot z - c \cdot x$ 
5:  $X_2 \leftarrow a \cdot y + b \cdot x$ 
6:  $Z_2 \leftarrow a \cdot y - b \cdot x$ 
7: return  $(X_1 : Z_1), (X_2 : Z_2)$ 

```

Algorithm 7.50 Isogeny22Chain(P, Q, pts)

Input: Points $P, Q \in E_1 \times E_2$ of order 2^e , a positive integer e , and an array pts containing points on $E_1 \times E_2$ with at least one point zero.

Output: The codomain $E_3 \times E_4$ of the $(2, 2)$ -isogeny chain $\Phi = \Phi_e \circ \dots \circ \Phi_1$ such that $\ker(\Phi) = \langle P, Q \rangle$ with $\deg(\Phi_k) = 2$ for all $k = 1, \dots, e$, and the evaluated points $[\Phi(P) \mid P \in \text{pts}]$.

```

1: Initialize a list  $\text{strat\_pts} \leftarrow [(P, Q)]$ 
2: Initialize a list  $\text{orders} \leftarrow [e - 2]$ 
3:  $k \leftarrow 0$ 
4: while  $\text{orders}[k] \neq 1$  do
5:    $k \leftarrow k + 1$ 
6:   if  $\text{orders}[k - 1] \geq 16$  then
7:      $n \leftarrow \lfloor \text{orders}[k - 1] / 2 \rfloor$ 
8:   else
9:      $n \leftarrow \text{orders}[k - 1] - 1$ 
10:  end if
11:   $(R, S) \leftarrow \text{strat\_pts}[k - 1]$ 
12:  for  $i$  from 0 up to  $n - 1$  do
13:     $R \leftarrow \text{DBL}(R, A_1)$ 
14:     $S \leftarrow \text{DBL}(S, A_2)$ 
15:  end for
16:   $\text{strat\_pts}[k] \leftarrow (R, S)$ 
17:   $\text{orders}[k] \leftarrow \text{orders}[k - 1] - n$ 
18: end while
19:  $\_, I, 0_A, J, N \leftarrow \text{GluingCodomain}(\text{strat\_pts}[k])$  // Computing  $\Phi_1 : E_1 \times E_2 \rightarrow A_1$ 
20:  $\text{pts} \leftarrow [\text{GluingEvalSpecial}(P, I, N) \mid P \in \text{pts}]$ 
21: for  $i$  from 0 up to  $k - 1$  do
22:    $\text{strat\_pts}[i] \leftarrow \text{GluingEval}(\text{strat\_pts}[i], I, J, N)$ 
23:    $\text{orders}[i] \leftarrow \text{orders}[i] - 1$ 
24: end for
25:  $k \leftarrow k - 1$ 
26:  $\text{consts} \leftarrow \text{ThetaPrecomp}(0_A)$ 
27:  $j \leftarrow 0$ 
28: while  $k \geq 0$  and  $\text{orders}[k] \neq 0$  do
29:    $j \leftarrow j + 1$ 
30:   while  $\text{orders}[k] \neq 1$  do
31:      $k \leftarrow k + 1$ 
32:      $n \leftarrow \lfloor \text{orders}[k - 1] / 2 \rfloor$ 
33:     for  $i$  from 0 up to  $n - 1$  do
34:        $\text{strat\_pts}[k] \leftarrow \text{ThetaDBL}(\text{strat\_pts}[k - 1], \text{consts})$ 
35:        $\text{orders}[k] \leftarrow \text{orders}[k - 1] - n$ 
36:     end for
37:   end while
38:    $0'_A, I, 0_A \leftarrow \text{GenericCodomainWith8Torsion}(\text{strat\_pts}[k])$  // Computing  $\Phi_k : A_{k-1} \rightarrow A_k$ 
39:    $\text{pts} \leftarrow [\text{GenericEval}(P, I) \mid P \in \text{pts}]$ 
40:    $\text{consts} \leftarrow \text{ThetaPrecomp}(0_A, 0'_A)$ 
41:   for  $i$  from 0 up to  $k - 1$  do
42:      $\text{strat\_pts}[i] \leftarrow \text{GenericEval}(\text{strat\_pts}[i], I)$ 
43:      $\text{orders}[j] \leftarrow \text{orders}[j] - 1$ 
44:   end for
45:    $k \leftarrow k - 1$ 
46: end while
47: if  $e \geq 3$  then
48:    $\text{strat\_pts}[0] \leftarrow \text{GenericEval}(\text{strat\_pts}[0], I)$ 
49: end if
50:  $\_, I, 0_A \leftarrow \text{GenericCodomainWith4Torsion}(\text{strat\_pts}[0], 0_A)$  // Computing  $\Phi_{e-1} : A_{e-2} \rightarrow A_{e-1}$ 
51:  $\text{pts} \leftarrow [\text{GenericEval}(P, I) \mid P \in \text{pts}]$ 
52:  $\_, I, 0_A \leftarrow \text{GenericCodomain}(0_A)$  // Computing  $\Phi_e : A_{e-1} \rightarrow A_e$ 

```

```

53: pts ← [GenericEval(P, I) | P ∈ pts]
54: M ← SplittingIsomorphism(0A)
55: 0A ← M · 0A
56: pts ← [M · P | P ∈ eval_pts]
57: (A3 : C3), (A4 : C4) ← ThetaToProduct(0A)
58: pts ← [ThetaProductPointToMontgomery(P, 0A) | P ∈ pts]
59: Let E3, E4 be the elliptic curves defined by Montgomery coefficients (A3 : C3), (A4 : C4)
60: return E3 × E4, pts

```

Algorithm 7.51 GluingEval($P, T_1'', J, \mathbf{N}$)

Input: The point $P \in E_1 \times E_2$ and T_1'' a 8-torsion point such that $[4]T_1'' \in \ker(\Phi)$ both in Jacobian coordinates and the point $J = (x : x : y : y)$, the dual of the theta point $\Phi(T_1'')$ on A , and the change-of-basis matrix $\mathbf{N}$ computed during GluingCodomain.

Output: Theta coordinates of $\Phi(P)$.

```

1: P1, P2 ← P
2: T1, T2 ← T1''
3: x, y ← J
4: u1, v1, z1 ← ADDComponents(P1, T1, E1) // Total cost: 11M + 5S + 7a
5: u2, v2, z2 ← ADDComponents(P2, T2, E2) // Total cost: 11M + 5S + 7a
6: U ← (u1 · u2 + v1 · v2, u1 · w1, w1 · u2, w1 · w2)
7: V ← (v1 · u2 + u1 · v2, w1 · w1, w1 · v2, w1 · v2, 0)
8: U ← N · U
9: V ← N · V
10: U ← S(U)
11: V ← S(V)
12: X±, Y±, Z±, W± ← H(U - V)
13: XΦ(P), YΦ(P), ZΦ(P), WΦ(P) ← (X± · y, Y± · y, Z± · x, W± · x)
14: (xΦ(P), yΦ(P), zΦ(P), wΦ(P)) ← H(XΦ(P), YΦ(P), ZΦ(P), WΦ(P))
15: return (xΦ(P) : yΦ(P) : zΦ(P) : wΦ(P)) // Total cost: 62M + 18S + 50a

```

Algorithm 7.52 GluingEvalSpecial($P, I, \mathbf{N}$)

Input: The point $P \in E_1 \times E_2$ of the form $(P_1, 0)$ or $(0, P_2)$, the “inverse” of the dual isogeneous theta null point $I := (\alpha^{-1} : \beta^{-1} : \gamma^{-1} : 0)$ on A and the change-of-basis matrix $\mathbf{N}$ computed during GluingCodomain.

Output: Theta coordinates of $\Phi(P)$.

```

1: P ← ProductToTheta(P, N) // 20M + 12a
2: xP, yP, zP, tP ← P
3: α-1, β-1, γ-1, _ ← I
4: (XP, YP, ZP, 0) ← H ∘ S(xP, yP, zP, wP)
5: (XΦ(P), YΦ(P), ZΦ(P)) ← XP · α-1, YP · β-1, ZP · γ-1
6: (xΦ(P), yΦ(P), zΦ(P), wΦ(P)) ← H(XΦ(P), YΦ(P), ZΦ(P), 0)
7: return (xΦ(P) : yΦ(P) : zΦ(P) : wΦ(P)) // Total cost: 4S + 23M

```

Algorithm 7.53 Isogeny22ChainWithTorsion(P, Q, pts)

Input: Jacobian points $P, Q \in E_1 \times E_2$ of order 2^{e+2} , Montgomery coefficients A_1, A_2 of E_1, E_2 respectively, a positive even integer e , and an array pts containing points on $E_1 \times E_2$ with at least one point zero.

Output: The codomain $E_3 \times E_4$ of the $(2, 2)$ -isogeny chain $\Phi = \Phi_e \circ \dots \circ \Phi_1$ such that $\ker(\Phi) = \langle P, Q \rangle$ with $\deg(\Phi_k) = 2$ for all $k = 1, \dots, e$, and the evaluated points $[\Phi(P) \mid P \in \text{pts}]$.

```

1: Initialize a list  $\text{strat\_pts} \leftarrow [(P, Q)]$ 
2: Initialize a list  $\text{orders} \leftarrow [e]$ 
3:  $k \leftarrow 0$ 
4: while  $\text{orders}[k] \neq 1$  do
5:    $k \leftarrow k + 1$ 
6:   if  $\text{orders}[k - 1] \geq 16$  then
7:      $n \leftarrow \lfloor \text{orders}[k - 1] / 2 \rfloor$ 
8:   else
9:      $n \leftarrow \text{orders}[k - 1] - 1$ 
10:  end if
11:   $(R, S) \leftarrow \text{strat\_pts}[k - 1]$ 
12:  for  $i$  from 0 up to  $n - 1$  do
13:     $R \leftarrow \text{DBL}(R, A_1)$ 
14:     $S \leftarrow \text{DBL}(S, A_2)$ 
15:  end for
16:   $\text{strat\_pts}[k] \leftarrow (R, S)$ 
17:   $\text{orders}[k] \leftarrow \text{orders}[k - 1] - n$ 
18: end while
19:  $\_, I, 0_A, J, N \leftarrow \text{GluingCodomain}(\text{strat\_pts}[k])$  // Computing  $\Phi_1 : E_1 \times E_2 \rightarrow A_1$ 
20:  $\text{pts} \leftarrow [\text{GluingEvalSpecial}(P, I, N) \mid P \in \text{pts}]$ 
21: for  $i$  from 0 up to  $k - 1$  do
22:    $\text{strat\_pts}[i] \leftarrow \text{GluingEval}(\text{strat\_pts}[i], I, J, N)$ 
23:    $\text{orders}[i] \leftarrow \text{orders}[i] - 1$ 
24: end for
25:  $k \leftarrow k - 1$ 
26:  $\text{consts} \leftarrow \text{ThetaPrecomp}(0_A)$ 
27:  $j \leftarrow 0$ 
28: while  $k \geq 0$  and  $\text{orders}[k] \neq 0$  do
29:    $j \leftarrow j + 1$ 
30:   while  $\text{orders}[k] \neq 1$  do
31:      $k \leftarrow k + 1$ 
32:      $n \leftarrow \lfloor \text{orders}[k - 1] / 2 \rfloor$ 
33:     for  $i$  from 0 up to  $n - 1$  do
34:        $\text{strat\_pts}[k] \leftarrow \text{ThetaDBL}(\text{strat\_pts}[k - 1], \text{consts})$ 
35:        $\text{orders}[k] \leftarrow \text{orders}[k - 1] - n$ 
36:     end for
37:   end while
38:    $0'_A, I, 0_A \leftarrow \text{GenericCodomainWith8Torsion}(\text{strat\_pts}[k])$  // Computing  $\Phi_k : A_{k-1} \rightarrow A_k$ 
39:    $\text{pts} \leftarrow [\text{GenericEval}(P, I) \mid P \in \text{pts}]$ 
40:    $\text{consts} \leftarrow \text{ThetaPrecomp}(0_A, 0'_A)$ 
41:   for  $i$  from 0 up to  $k - 1$  do
42:      $\text{strat\_pts}[i] \leftarrow \text{GenericEval}(\text{strat\_pts}[i], I)$ 
43:      $\text{orders}[j] \leftarrow \text{orders}[j] - 1$ 
44:   end for
45:    $k \leftarrow k - 1$ 
46: end while
47:  $M \leftarrow \text{SplittingIsomorphism}(0_A)$ 
48:  $0_A \leftarrow M \cdot 0_A$ 
49:  $\text{pts} \leftarrow [M \cdot P \mid P \in \text{eval\_pts}]$ 
50:  $(A_3 : C_3), (A_4 : C_4) \leftarrow \text{ThetaToProduct}(0_A)$ 
51:  $\text{pts} \leftarrow [\text{ThetaProductPointToMontgomery}(P, 0_A) \mid P \in \text{pts}]$ 
52: Let  $E_3, E_4$  be the elliptic curves defined by Montgomery coefficients  $(A_3 : C_3), (A_4 : C_4)$ 
53: return  $E_3 \times E_4, \text{pts}$ 

```

Heuristics and failure cases

8.1. Heuristics on lattices and ideals

The termination of a few algorithms related to ideal lattices relies on mild heuristics. The first one guarantees [LatticeSampling](#)(Algorithm 2.14) terminates in a constant number of steps on average:

Heuristic 8.1. *The number of lattice points in an ellipsoid S centered at the origin is well approximated by the number of lattice points in a parallelogram $B \supset S$ multiplied by $\frac{\text{vol}(S)}{\text{vol}(B)}$.*

The next one guarantees [IdealGenerator](#)(Algorithm 2.17) always terminates quickly:

Heuristic 8.2. *Enumerating integer linear combinations of a quaternion ideal basis by increasing ℓ_1 norm finds a generator of the ideal in a small number of steps.*

8.2. Chains of $(2, 2)$ -isogenies

During key generation, signing, and verifying we compute several chains of isogenies of the form

$$E_1 \times E_2 \xrightarrow{\Phi_1} A_1 \xrightarrow{\Phi_2} A_2 \cdots A_{e-2} \xrightarrow{\Phi_{e-1}} A_{e-1} \xrightarrow{\Phi_e} E_3 \times E_4.$$

The algorithms we gave in Section 7.5 are not universal and may fail in two possible ways. We argue that these failures happen with negligible probability during an honest execution, and thus may be ignored during key generation/signing (we nevertheless catch them and restart the process). If they happen during verification, with overwhelming probability they indicate a malicious signature and thus lead to rejection.

The first failure case happens if we encounter a splitting before the final step of a chain, i.e., if A_k has product theta structure for $1 \leq k < e$. This failure can never happen during verification of an honest signature. To bound the failure probability in key generation/signing, we make the following assumption:

Heuristic 8.3. *The surfaces A_k encountered along the chains of $(2, 2)$ -isogenies computed in [SQIsign2D².KeyGen](#) and [SQIsign2D².Sign](#) behave like uniformly random superspecial principally polarized abelian surface.*

The number of products of supersingular elliptic curves $E_1 \times E_2$ is $O(p^2)$, and the number of superspecial principally polarized abelian surface is $O(p^3)$, thus the heuristic ensures the computation of a $(2, 2)$ -isogeny chain fails with probability $\tilde{O}(p^{-1})$.

The second failure case happens, during the first gluing isogeny $E_1 \times E_2 \rightarrow A$, when the base change matrix we compute in [ThetaChangeOfBasis](#) is 0. This happens only when the trace of the coordinate $X_1 \cdot X_2$ under our kernel is zero, where $(X_1 : Z_1)$ and $(X_2 : Z_2)$ are the Montgomery coordinates on E_1 and E_2 respectively.

Heuristic 8.4. *Let $E_1 \times E_2$ be a product of elliptic curves to which we apply a 2-dimensional isogeny computations during an honest execution of [SQIsign2D².KeyGen](#), [SQIsign2D².Sign](#), and [SQIsign2D².Verify](#). Then the trace of the product coordinate $X_1 \cdot X_2$ under the gluing kernel, behaves like an independent random element of a $\mathbb{F}_{p^2}$ -vector space of dimension one.*

Clearly the chance of encountering a zero trace is $O(p^{-2})$, and the computation only involves $O(1)$ two-dimensional isogenies: thus, the total failure probability for these second type of failures is in $O(p^{-2})$.

We note that this second failure may also happen (with negligible probability) during verification of an honest signature, because the gluing is computed from the other side in the verification compared to the signature.

8.3. Failures in SQIsign2D²

The [ImRanIso](#), [GenImRanIso](#) and [RandomEquivalentQuaternion](#) algorithms are the main algorithmic building block behind SQIsign2D² key generation and signature. These algorithms will fail in four places:

- (1) If the call to [FullRepresentInteger](#) fails in [ImRanIso](#),
- (2) If the call to [FullStrongApproximation](#) fails in [GenImRanIso](#),
- (3) If any of the calls to [Isogeny22Chain](#) fail,
- (4) If the call to [RandomEquivalentQuaternion](#) fails.

[Isogeny22Chain](#)'s failure was already discussed in Section 8.2 as it is not unique to these algorithms.

We treat (1) and (2) at once in 8.3.1, as both failure events are due to calls to [extend_Cornacchia](#). The failures of [RandomEquivalentQuaternion](#) are treated in 8.3.2. A common heuristic used in all of them is the following:

Heuristic 8.5. *Integers represented by a quadratic form behave like independent random integers of the same size with respect to factorization, coprimality and quadratic residuosity.*

This type of assumption is very common in number theory, and after accounting for obstructions that may arise in specific cases (e.g., some congruence properties), tends to be satisfied in practice and can be easily confirmed experimentally.

Some of these assumptions could be removed assuming variants of the Generalized Riemann Hypothesis (GRH), following [Wes22], but the price to pay would be an increase of parameters and significant loss in efficiency. We do not pursue this direction in SQIsign2D².

8.3.1. [FullRepresentInteger](#) and [FullStrongApproximation](#)

[FullRepresentInteger](#) was used in [ImRanIso](#). In [ImRanIso](#), the input M is approximately $p^{3/2}$.

[FullRepresentInteger](#) calls [extend_Cornacchia](#) to solve an equation of the form $x^2 + y^2 = M'$ for $M' = 4M - p(z^2 + w^2)$, where z, w are random in some intervals. This call succeeds if and only if:

- $M' = d \cdot M''$, where d is a positive integer such that all prime divisors of d are in `Prime_list` = $[2, 5, 13, 17, 29, 37, 41, 53, 61, 73, 89, 97]$;
- and M'' is a prime represented by $x^2 + y^2$;

`FullRepresentInteger` succeeds if the solution additionally satisfies $x \equiv w \pmod{2}$ and $y \equiv z \pmod{2}$ with probability $1/4$.

In `ImRanIso`, $M < B = p^{3/2}$, from Heuristic 8.5, M' behaves like a random integer. Thus we expect that the probability of M' satisfying the above two conditions is approximately

$$\frac{1}{2} \cdot \frac{1}{\ln B} \cdot \prod_{\ell \in \text{Prime_list}} (1 + 1/\ell + 1/\ell^2 + \dots) = \frac{1}{2} \cdot \frac{1}{\ln B} \cdot \prod_{\ell \in \text{Prime_list}} \frac{\ell}{\ell - 1} > \frac{3}{2} \cdot \frac{1}{\ln B} = \frac{1}{\ln p}$$

Thus every loop has probability of success approximately $1/(4 \cdot \ln p)$. Recall that

$$\text{QUAT_repres_num_trial} = \lceil \log_{1-1/(4 \cdot \ln p)}(2^{-64}) \rceil.$$

So the total failure probability of `FullRepresentInteger` is less than

$$\left(1 - \frac{1}{4 \ln p}\right)^{\text{QUAT_repres_num_trial}} < 2^{-64}.$$

The analysis in `GenImRanIso` is similar, where `FullStrongApproximation` is called with input M approximately p^2 . `FullStrongApproximation` succeeds if its subroutine `extend_Cornacchia` outputs an appropriate solution by means of the two conditions above. Similarly, one can show that the failure probability is less than

$$\left(1 - \frac{1}{3 \ln p}\right)^{\text{QUAT_strongappro_num_trial}} < 2^{-64}.$$

8.3.2. Response ideal sampling

In SQIsign2D², the degree q of the response isogeny is 2^a -adequate, i.e. q is odd, $q < 2^a$ and $3 \nmid q$. Before discussing security, we estimate the probability that a sampled value of q is admissible.

By Minkowski's theorem, the number of elements in a lattice Λ with norm less than R is approximately $\frac{\pi^2 R^4}{\text{Covol}(\Lambda)}$. We need to estimate the number of elements $\beta \in I$ whose norm is at most $2^a \text{Nrd}(I)$. Setting $R = \sqrt{2^a \text{Nrd}(I)}$, the number of such elements β is $\frac{2\pi^2 2^{2a}}{p} \approx \frac{2\pi^2 2^a}{f 3^b}$. Assume that $\mathcal{O}_R(I)^\times = \{\pm 1\}$. Then the number of distinct ideals equivalent to I of the form $I \frac{\bar{\beta}}{\text{Nrd}(I)}$ is approximately $\frac{\pi^2 2^a}{f 3^b}$.

However, the estimate that q is odd with probability $\frac{1}{2}$ is not correct. In fact, when searching for β , one may sample $\beta \in \mathcal{O}/\text{Nrd}(I)\mathcal{O}$. We believe that the distribution of norms mod ℓ should follow the distribution of determinants of uniformly random matrices $M \in M_2(\mathbb{Z}/\ell\mathbb{Z})$. Then the probability that $\frac{\text{Nrd}(\beta)}{\text{Nrd}(I)}$ is coprime to ℓ is $\frac{\#\text{GL}_2(\mathbb{Z}/\ell\mathbb{Z})}{\ell^4} = \frac{\ell^4 - 3(\ell - 1)^2(\ell + 1)}{\ell^4} = \frac{(\ell - 1)^2(\ell + 1)}{\ell^3}$.

TABLE 8.1. Estimated failure probabilities of finding q for all parameters.

	a	b	f	Failure probability
Level1-eff	131	78	1	$2^{-593.15}$
Level1-sec	137	84	1	$2^{-52.07}$
Level2-eff	161	96	61	$2^{-26.95}$
Level2-sec	168	101	11	$2^{-78.72}$
Level3-eff	263	156	1	$2^{-196636.81}$
Level3-sec	264	163	1	$2^{-179.82}$
Level5-eff	514	319	11	$2^{-109.66}$
Level5-sec	536	334	17	$2^{-20.74}$

Security analysis

In this chapter, we analyze the security of SQIsign2D² as in [XLZ+25]. The analysis in [XLZ+25] shows that the soundness is obtained from the hardness of computing endomorphism ring problem, the zero-knowledge property is obtained from some reliable assumptions and it is sufficient to obtain EUF-CMA security for SQIsign2D² scheme. While in this section we give a high-level overview of the security proofs and its main results, we refer the reader to [XLZ+25] for a more thorough treatment. Note that we analyze a version of SQIsign2D² where its subroutines do not fail: we address the impact of algorithm failures on the security of the protocol in Section 8.3.2.

9.1. Security reductions

9.1.1. Security of signature from Fiat-Shamir transform

EUFCMA Let $\Pi = (\text{KeyGen}, \text{Sign}, \text{Verify})$ be a signature scheme. The existential unforgeability under adaptive chosen-message attacks (EUF-CMA) of Π is defined by the following game between a challenger and an adversary $\mathcal{A}$.

The challenger generates a key pair $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$ and gives the public key pk to $\mathcal{A}$. The adversary $\mathcal{A}$ gets black-box query-access to a signing oracle $\mathcal{O}_{\text{sign}}$, which when queried on a message msg outputs a signature $\sigma \leftarrow \text{Sign}(\text{sk}, \text{msg})$. The adversary $\mathcal{A}$ wins the game if it can output a message msg^* and a signature σ^* such that $\text{Verify}(\text{pk}, \text{msg}^*, \sigma^*) = 1$ and msg^* has not been queried to $\mathcal{O}_{\text{sign}}$.

We define the advantage $\text{Adv}_{\Pi}^{\text{EUF-CMA}}(\mathcal{A})$ of $\mathcal{A}$ as the probability that it wins the game. We say that the signature scheme Π is EUF-CMA secure if every PPT adversary has negligible advantage in the security parameter λ .

The security of signature from Fiat-Shamir transform relies on the following properties:

Completeness. The completeness (correctness) property says that if both parties follow the protocol and if the prover uses a valid witness w such that $(x, w) \in R$, then the verifier will accept the proof with probability 1, that is, for all $(x, w) \in R$, we have $\Pr[V(x, a, c, r) = 1] = 1$

Special Soundness. A sigma protocol has *special soundness* if there exists an efficient extractor algorithm that, given two transcripts (x, a, c, r) and (x, a, c', r') such that $V(x, a, c, r) = V(x, a, c', r') = 1$ and $c \neq c'$, outputs a witness w such that $(x, w) \in R$.

Honest-verifier Zero-knowledge. A sigma protocol is *honest-verifier zero-knowledge* if there exists an efficient simulator $\mathcal{S}$ that, given x , outputs transcripts (x, a, c, r) that are

indistinguishable from transcripts generated in honest executions of the protocol on common input x .

Special Honest-verifier Zero-knowledge. A sigma protocol is *special honest-verifier zero-knowledge* if there exists an efficient simulator $\mathcal{S}$ that, given x and a challenge $c \in \mathcal{C}$, outputs transcripts (x, a, c, r) that are indistinguishable from honest transcripts with challenge c .

More precisely, for every $(x, w) \in R$ and every $c \in \mathcal{C}$, we require that $\mathcal{S}(x, c)$ is indistinguishable from the distribution of honest transcripts (x, a, c, r) generated by the protocol. If these distributions are statistically close, then the protocol is said to have *statistical honest-verifier zero-knowledge*; if they are only computationally indistinguishable, then it is said to have *computational honest-verifier zero-knowledge*.

Informally, completeness of the Σ -protocol guarantees the correctness of the resulting signature scheme. Its security, in particular EUF-CMA security, is typically established in the random oracle model under suitable assumptions, by relying on appropriate soundness or extractability properties of the underlying protocol together with the properties of the challenge distribution.

In the remainder of this chapter, we present a formal security analysis of the SQIsign2D² identification protocol. Since correctness is straightforward, our analysis focuses on establishing its special soundness and zero-knowledge properties.

9.1.2. Special soundness

The special soundness proof follows a structure analogous to those of SQIsignHD and SQIsign2D-East. The hard problem underlying the special soundness proof is known as Supersingular Endomorphism Problem:

Problem 9.1 (Supersingular Endomorphism Problem). *Given a supersingular elliptic curve E over $\mathbb{F}_{p^2}$ with p prime, find a non-trivial endomorphism of E that can be efficiently evaluated.*

From [PW24], the extraction of any non-scalar endomorphism $\text{End}(E_A)$ enables the complete reconstruction of $\text{End}(E_A)$. This allows for the efficient recovery of the secret ideal I_τ and its corresponding isogeny τ in polynomial time. Therefore, the proof reduces to demonstrating the extraction of a non-scalar endomorphism $\alpha \in \text{End}(E_A)$ from two valid conversations with the same commitment and different challenges.

Theorem 9.2. *The SQIsign2D² identification protocol has the special soundness property. That is, given two valid conversations with the same commitment and different challenges, one can extract a non-scalar endomorphism $\alpha \in \text{End}(E_A)$ that can be efficiently evaluated.*

Proof. Assume that the two valid conversations are $(E_{\text{com}}, K_{\text{cha}}, E_{\text{aux}}, U_2, V_2)$ and $(E_{\text{com}}, K'_{\text{cha}}, E'_{\text{aux}}, U'_2, V'_2)$ with $K_{\text{cha}} = R_{\text{com}} + [\text{com}]S_{\text{com}}$, $K'_{\text{cha}} = R_{\text{com}} + [\text{com}']S_{\text{com}}$ and $\langle K_{\text{cha}} \rangle \neq \langle K'_{\text{cha}} \rangle$. From K_{cha} and K'_{cha} one can reveal the challenge isogenies $\varphi : E_{\text{com}} \rightarrow E_{\text{chl}}$ and $\varphi' : E_{\text{com}} \rightarrow E'_{\text{chl}}$, respectively. Then the response isogenies σ and σ' are recovered with respect to the knowledge of $(E_{\text{aux}}, P_{\text{aux}}, Q_{\text{aux}}, E_{\text{chl}}, U_2, V_2)$ and $(E'_{\text{aux}}, P'_{\text{aux}}, Q'_{\text{aux}}, E'_{\text{chl}}, U'_2, V'_2)$, where $\{P_{\text{aux}}, Q_{\text{aux}}\}$ and $\{P'_{\text{aux}}, Q'_{\text{aux}}\}$ are the canonical bases of $E_{\text{aux}}[2^a]$ and $E'_{\text{aux}}[2^a]$, respectively. Therefore, one can extract an endomorphism $\alpha = \hat{\sigma}' \circ \varphi' \circ \hat{\varphi} \circ \sigma$, as illustrated in Figure 9.1.

It remains to prove that the endomorphism α is non-scalar. Assume that $\alpha = [n]$ for some integer n . Then $\alpha(R) = \infty$ for all $R \in E[3^b]$. It follows from $\gcd(q, 3^b) = 1$, $\gcd(q', 3^b) = 1$ that $\varphi' \circ \hat{\varphi} = [3^b]$, i.e., $\ker(\varphi) = \ker(\varphi')$. This leads to a contradiction. Hence, we conclude that α is a non-scalar endomorphism of E_A , which ends the proof. $\square$

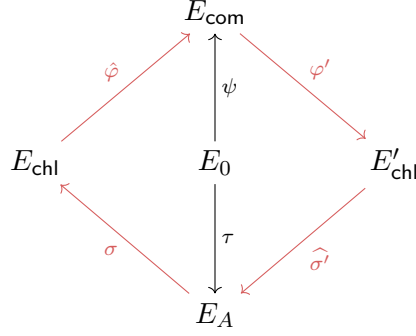


FIGURE 9.1. A sketch of the special soundness proof.

9.1.3. Zero knowledge

The zero-knowledge proof of the SQIsign2D² identification protocol parallels that of the SQIsign2D-East identification protocol. In the proof, we also define two random oracles, to simulate the response isogeny and the auxiliary isogeny, respectively.

Since the supersingular isogeny graph contains approximately $p/12$ isomorphism classes, the resulting commitment space is expected to be large enough to support pseudorandom commitment generation. This motivates the following assumption.

Assumption 9.3. *The commitment curve E_{com} returned by $\text{ImRanIso}(N_\psi, -)$ for $N_\psi \approx \sqrt{p}$ is computationally indistinguishable from an elliptic curve chosen uniformly at random among the supersingular j -invariants in characteristic p .*

Under the present parameter setting, this assumption captures the expected randomness of the commitment phase.

Definition 9.4. *Given an integer D , a Random Uniform Adequate Degree Isogeny Oracle (**RUADIO**) is an oracle taking as input a supersingular elliptic curve E and returning the tuple $(\sigma(P), \sigma(Q))$, where $\langle P, Q \rangle = E[D]$, $\sigma : E \rightarrow E'$ is a random isogeny and $\deg(\sigma) = q$ is D -adequate. Besides, the random oracle satisfies the following properties:*

- *The distribution of E' is computationally indistinguishable from that of an elliptic curve chosen uniformly at random in the supersingular isogeny graph;*
- *The conditional distribution of σ given E' is uniform among isogenies from E to E' whose degrees are D -adequate.*

Definition 9.5. *Given an integer D , a Fixed Degree Isogeny Oracle (**FIDIO**) is an oracle taking as input a supersingular elliptic curve E and an integer N and returning the tuple $(\varphi_{\text{aux}}(P), \varphi_{\text{aux}}(Q))$, where $\{P, Q\}$ is the canonical basis of $E[D]$ and $\varphi_{\text{aux}} : E \rightarrow E'$ is a uniformly random N -isogeny.*

Problem 9.6. *Let D be the integer defined in the parameter setting and E_A be the public curve. Define the distribution of the reduced norm of the response ideals I_σ on $\mathbb{Z}$ by $\mathcal{Q}$. Suppose that $\mathcal{D}_{\mathcal{U}}$ and $\mathcal{D}_{\mathcal{R}}$ represent the uniform distribution $\mathcal{U}_{\text{Iso}(E_A, q)} = \{\varphi : E_A \rightarrow * \mid \deg(\varphi) = D - q\}$ and the output distribution of **GenImRanIso**, respectively. Let $S = \{\varphi_{\text{aux}} : E_A \rightarrow * \mid \deg(\varphi_{\text{aux}}) = D - q\}$ be a set of size $M > \log(N_\tau)$, where either*

1. *S is sampled by first sampling $q \sim \mathcal{Q}$, then sampling φ_{aux} from $\mathcal{D}_{\mathcal{U}}$; or*
2. *S is sampled by first sampling $q \sim \mathcal{Q}$, then sampling φ_{aux} from $\mathcal{D}_{\mathcal{R}}$.*

*The problem is, given E_A, D, S , to distinguish between the two cases with a polynomial number of queries to $\mathcal{Q}$, **FIDIO** and $\mathcal{D}_{\mathcal{R}}$.*

As discussed in [NOC+25, Remark 5], a natural approach to addressing Problem 9.6 is to determine the endomorphism by reverse engineering the algorithms. This task is equivalent to solving Supersingular Endomorphism Problem, as defined in Problem 9.1. Consequently, we conjecture that Problem 9.6 is computationally hard. A rigorous analysis of the hardness of Problem 9.6 is left as future work.

Theorem 9.7. *Suppose Problem 9.6 is computationally hard and Assumption 9.3. Then, the SQIsign2D² identification protocol is special honest-verifier zero-knowledge (SHVZK) in the **RUADIO** and **FIDIO** models. That is, there exists a simulator $\mathcal{S}$ with access to **RUADIO** and **FIDIO**, such that the distribution of the accepting conversations generated by $\mathcal{S}$ is computationally indistinguishable from those of honest executions of the SQIsign2D² identification protocol.*

Proof. We proceed similarly as the zero-knowledge proof of SQIsign2D-East. The simulator $\mathcal{S}$ operates as follows: The simulator first adapts the **RUADIO** model to generate an efficient representation $(E_A, P_A, Q_A, E'_{\text{com}}, \sigma'(P_A), \sigma'(Q_A))$ of a q' -isogeny $\sigma' : E_A \rightarrow E'_{\text{chl}}$, where $\{P_A, Q_A\}$ is the canonical basis of $E_A[2^a]$ and q' is 2^a -adequate. Subsequently, the simulator $\mathcal{S}$ uniformly samples a 3^b -isogeny $\hat{\varphi}' : E'_{\text{chl}} \rightarrow E'_{\text{com}}$. Finally, the simulator constructs the $(2^a - q')$ -isogeny $\varphi'_{\text{aux}} : E_A \rightarrow E'_{\text{aux}}$ with the help of **FIDIO**. The resulting simulated transcript takes the form $(E'_{\text{com}}, \varphi', E'_{\text{aux}}, U'_2, V'_2)$, where U'_2, V'_2 are computed from φ'_{aux} and σ' .

Assume that the real transcript is $(E_{\text{com}}, \varphi, E_{\text{aux}}, U_2, V_2)$. In the following, we prove that the two transcripts are computationally indistinguishable.

From the first property of **RUADIO**, E'_{chl} is uniformly distributed in the supersingular isogeny graph. Since the isogeny $\hat{\varphi}' : E'_{\text{chl}} \rightarrow E'_{\text{com}}$ is chosen uniformly at random, the codomain E'_{com} of $\hat{\varphi}'$ is also uniformly random. Hence, E_{com} and E'_{com} are computationally indistinguishable. Furthermore, as φ, φ' are sampled from the same way, their distributions are the same.

The second property of **RUADIO** ensures that the response isogeny σ' is uniformly distributed among all isogenies of 2^a -adequate degree from E_A to E'_{chl} . Similarly, the response isogeny σ in the real transcript has the same distribution.

Thanks to the hardness of Problem 9.6, the auxiliary isogeny φ_{aux} in the real transcript is computationally indistinguishable from a random isogeny φ'_{aux} of degree $2^a - q$ starting from E_A .

Therefore, it remains to show that $(E_{\text{aux}}, U_2, V_2)$ and $(E'_{\text{aux}}, U'_2, V'_2)$ are computationally indistinguishable. Given the computational indistinguishability of $(\sigma, \varphi_{\text{aux}})$ and $(\sigma', \varphi'_{\text{aux}})$, it follows that the response $(E_{\text{aux}}, U_2, V_2)$ in the real identification protocol is computationally indistinguishable from $(E'_{\text{aux}}, U'_2, V'_2)$ in the simulated transcript. This completes the proof. $\square$

9.1.4. Security proof of SQIsign2D²

[FS86] provide a unified framework for constructing signature schemes from identification schemes via the Fiat–Shamir transform, and establish the security of this transformation in a high degree of generality.

Theorem 9.8. *Let Π be a non-trivial canonical Σ -protocol for a hard relation $\mathcal{R}$. Suppose that Π satisfies completeness, special soundness, and honest-verifier zero-knowledge. Then the Fiat–Shamir signature scheme derived from Π is existentially unforgeable under chosen-message attacks (EUF-CMA) in the random oracle model.*

Proof. This result is standard; see [FS86]. $\square$

SQIsign2D² is a signature scheme obtained by applying the Fiat–Shamir transform to the corresponding Σ -protocol. The underlying Σ -protocol satisfies completeness, special soundness (Theorem 9.2), and honest-verifier zero-knowledge (Theorem 9.7). Therefore, by Theorem 9.8, we obtain the following security result:

Theorem 9.9. *Assuming that the endomorphism ring computation problem is hard, assumption 9.3 that the commitment curve generated during honest signing is indistinguishable in the isogeny graph from a randomly chosen one, and that problem 9.6 is hard, then, in the RUADIO and FIDIO models, $SQIsign2D^2$ is EUF-CMA secure.*

Proof. Since the $SQIsign2D^2$ Σ -protocol satisfies completeness, special soundness by Theorem 9.2, and honest-verifier zero-knowledge by Theorem 9.7, the theorem follows directly from Theorem 9.8. $\square$

9.2. Resistance to known attacks

The parameters of $SQIsign2D^2$ satisfies the following conditions (other requirements are shown in 4):

- The prime $p = f \cdot 2^a \cdot 3^b - 1$, where f is a small positive integer and $a, b \approx \lambda$.
- The secret isogeny has degree $N_\tau < p^{1/4}$ and the commitment isogeny has degree $N_\psi \approx \sqrt{p}$.
- The challenge isogeny degree is 3^b with b as above.
- The challenge space has size 3^b with $3^b \approx 2^\lambda$.
- The response isogeny degree q is not fixed beforehand, but always satisfies $q < 2^a$.

In this section, we show that our parameter choices provide at least λ bits of classical security and $\lambda/2$ bits of quantum security against all known attacks on $SQIsign2D^2$. As in Section 1.5, we use the notation $\tilde{O}(g)$ to denote $O(g \cdot \ell(g))$ for some polylogarithmic function ℓ . For some of the attacks analyzed below, the hidden polylogarithmic factors have a non-negligible impact on the resulting concrete bit-security estimates.

9.2.1. The pure endomorphism ring problem and general isogeny finding

The hardness of the endomorphism ring problem is necessary for the security of all isogeny-based cryptography schemes. An efficient algorithm for computing the endomorphism ring of a supersingular elliptic curve would break essentially all isogeny-based cryptographic schemes, including $SQIsign2D^2$ [EHL+18; XLZ+25]. That is because an isogeny from E_0 to E_{pk} can be computed from the known endomorphism ring $\mathbf{End}(E_{pk})$ by computing the connecting ideal.

Computing the endomorphism ring of a supersingular elliptic curve is probabilistically polynomial-time equivalent to finding a non-scalar endomorphism [PW24]. Consequently, the core task can also be reduced to finding such a non-scalar endomorphism.

The fastest algorithm to find one non-scalar endomorphism is described in [EHL+20] which runs in $O(\log p)^2 p^{\frac{1}{2}})$ with low memory requirements. Grover algorithm using a quantum computer provides a $\tilde{O}(p^{\frac{1}{4}})$ time algorithm with low memory requirements. Since the main bottleneck is essentially a search over a space of size about $p^{\frac{1}{2}}$, Grover's algorithm gives a natural quantum speedup, reducing the running time to $O(p^{\frac{1}{4}})$ while preserving low memory usage. For any prime p , the full supersingular isogeny graph with its roughly $\frac{p}{12}$ isomorphism classes of supersingular elliptic curves over $\overline{\mathbb{F}}_p$ is connected.

Finding an isogeny between two given curves, i.e. the general isogeny problem, is equivalent to the endomorphism computation problem [EHL+20],

Collision search to find a path between two given elliptic curves in $\tilde{O}(p^{\frac{1}{2}})$ time since the isogeny graph is connected with roughly $\frac{p}{12}$ points. Delfs and Galbraith gives an improved algorithm requiring significantly less memory but still the same time [DG16]. So the general isogeny problem has $\tilde{O}(p^{\frac{1}{2}})$ classical complexity and $\tilde{O}(p^{\frac{1}{4}})$ quantum complexity.

9.2.2. Key recovery

In SQIsign2D², the secret key is a random isogeny of unknown prime degree starting from E_0 , whose degree satisfies $N_\tau < p^{1/4}$. To recover the secret key, an attacker has no better option than to enumerate all admissible secret degrees and the corresponding cyclic isogenies. For each prime degree N , there are $N + 1$ cyclic isogenies of degree N . Hence, the number of candidate secret keys is approximately

$$\sum_{\substack{\ell < p^{1/4}, \ell \text{ prime} \\ \left(\frac{3}{\ell}\right) = -1}} (\ell + 1) \approx \frac{2p^{1/2}}{\log p}.$$

This estimate only counts the number of candidates to be enumerated; it is not yet the total running time of the key-recovery attack. For each candidate, the attacker must still compute and test the corresponding N -isogeny. A direct computation of an N -isogeny costs $O(N^{1/2})$ time. The attacker may use the KLPT algorithm to reduce the cost of this step. Even with this optimization, however, each trial still incurs at least polylogarithmic, and in particular logarithmic, overhead in p , coming from the KLPT procedure, the required finite-field arithmetic, and the verification step. Therefore, even under the conservative lower bound that each candidate costs only $\Omega(\log p)$ time, the total key-recovery time is at least $O\left(\frac{2p^{1/2}}{\log p} \cdot (\log p)\right) = O(p^{1/2})$.

Thus, choosing a public parameter $p > 2^{2\lambda}$ gives $p^{1/2} > 2^\lambda$, so exhaustive key recovery requires more than 2^λ operations. Consequently, SQIsign2D² achieves λ -bit security against this key-recovery attack.

9.2.3. Signature forgery

Forgery attacks require solving the endomorphism ring problem with addition information. The attacker is given a target curve E , together with a collection of isogenies of different degrees starting from E . If all these isogenies have smooth degree, then the addition information does not contribute to solve the endomorphism ring problem: random smooth degree isogenies can be sampled from any given curve, and hence the problem is still hard.

In SQIsign2D², the attacker can get some isogenies of non-smooth degree. This prevents us to directly judge whether the problem with addition isogenies is as difficult as the standard one. The reason is that, without additional information about the endomorphism ring of the starting curve, sampling isogenies of non-smooth degree is generally difficult. Indeed, a recent isogeny-based cryptographic construction [BBC+25] directly relies on precisely this hardness assumption.

Nevertheless, isogenies of non-smooth degree are unlikely to reduce the difficulty of the endomorphism ring problem. Conceptually, they do not appear to reveal any information that cannot already be obtained from smooth-degree isogenies. In fact, existing algorithms for computing the endomorphism ring of a given curve typically use only smooth-degree isogenies. Therefore, the best known attacks on the endomorphism ring problem with some known isogenies simply ignore this additional information and attempt to compute the endomorphism ring of E directly. It follows that the complexity of the best known attacks with addition information remains essentially the same as that of the endomorphism ring problem.

9.2.4. Attacking the Fiat-Shamir transform

As in any Fiat-Shamir signature scheme, an attacker may also attempt to forge signatures by attacking the security properties of the hash function used in the Fiat-Shamir transform, namely SM3 in our instantiation. In SQIsign2D², we use a hash function denoted by `Hash`, which takes as input $\text{pk} \parallel j(E_{\text{com}}) \parallel \text{msg}$ and outputs a value truncated to e_{chl} bits. This provides at least $b \log_2 3$ bits of classical security against preimage and second-preimage attacks, and $\frac{b}{2} \log_2 3$ bits of classical security against collision attacks.

Given a valid signature σ on a message msg , an attacker may mount a second-preimage attack on the hash function in order to find another message msg' . This would yield the same challenge value in the SQIsign2D² identification protocol, and hence σ would also be a valid signature on msg' . This gives rise to a forgery attack of complexity $O(3^b)$, which is slightly below the target security level $O(2^\lambda)$.

To close this security gap, we instantiate **HASH** by iterating the underlying standard hash function, SM3 in our case, $2^{\lambda - b \log_2 3}$ times. This technique, commonly referred to as grinding, raises the attack cost to the target level $O(2^\lambda)$: finding a second preimage for **HASH** requires $O(3^b)$ trials, and each trial requires at least $2^{\lambda - b \log_2 3}$ evaluations of the underlying hash function. The total cost is therefore $O(2^\lambda)$.

Attacks against the Fiat–Shamir transform itself should also be considered. Although finding a collision requires only $O(2^{\frac{b}{2} \log_2 3})$ work, this does not appear to yield a more efficient forgery attack against SQIsign2D². Indeed, before a collision search can be performed, the curve E_{com} must first be fixed, whereas the collision attack described above does not allow the adversary to control E_{com} .

The signer could then claim to have signed msg , while later asserting that the signature was intended for msg' where $\text{HASH}(\text{pk} \parallel j(E_{\text{com}}) \parallel \text{msg}) = \text{HASH}(\text{pk} \parallel j(E_{\text{com}}) \parallel \text{msg}')$. Such an attack only requires finding a fixed-prefix collision, and can therefore be carried out in time $O(2^{\frac{b}{2} \log_2 3})$. However, this type of repudiation attack lies outside the standard unforgeability guarantees of digital signatures. Moreover, it applies to any Fiat–Shamir signature scheme with the same challenge space size.

9.3. Concrete security of parameter sets

As shown in section 9.2, for security parameter $\lambda \in \{128, 160, 256, 512\}$, our parameter choices provide at least λ bits of classical security and $\lambda/2$ bits of quantum security. Table 9.1 provides the concrete security bits for each parameter set.

TABLE 9.1. Concrete security bits for each parameter set.

	Classical	Quantum
Level1-eff	128	64
Level1-sec	135	67
Level2-eff	160	80
Level2-sec	166	83
Level3-eff	256	128
Level3-sec	262	131
Level5-eff	512	256
Level5-sec	535	268

Bibliography

- [AAA+25] Marius A. Aardal, Gora Adj, Diego F. Aranha, Andrea Basso, Isaac Andrés Canales Martínez, Jorge Chávez-Saab, Maria Corte-Real Santos, Pierrick Dartois, Luca De Feo, Max Duparc, Jonathan Komada Eriksen, Tako Boris Fouotsa, Décio Luiz Gazzoni Filho, Basil Hess, David Kohel, Antonin Leroux, Patrick Longa, Luciano Maino, Michael Meyer, Kohei Nakagawa, Hiroshi Onuki, Lorenz Panny, Sikhar Patranabis, Christophe Petit, Giacomo Pope, Krijn Reijnders, Damien Robert, Francisco Rodríguez Henríquez, Sina Schaeffler, and Benjamin Wesolowski. *SQIsign: Algorithm Specifications and Supporting Documentation*. Version 2.0. SQIsign Team. Feb. 5, 2025. URL: <https://sqisign.org/spec/sqisign-20250205.pdf> (cit. on pp. 5, 8, 12).
- [SIKE] Reza Azarderakhsh, Matthew Campagna, Craig Costello, Luca De Feo, Basil Hess, Aaron Hutchinson, Amir Jalali, David Jao, Koray Karabina, Brian Koziel, Brian LaMacchia, Patrick Longa, Michael Naehrig, Geovandro Pereira, Joost Renes, Vladimir Soukharev, and David Urbanik. *Supersingular Isogeny Key Encapsulation*. <http://sike.org>. Oct. 10, 2020 (cit. on p. 5).
- [Bab86] László Babai. “On Lovász’lattice reduction and the nearest lattice point problem”. In: *Combinatorica* 6.1 (1986), pp. 1–13 (cit. on p. 31).
- [BKL+02] Paulo SLM Barreto, Hae Y Kim, Ben Lynn, and Michael Scott. “Efficient algorithms for pairing-based cryptosystems”. In: *Annual international cryptology conference*. Springer. 2002, pp. 354–369 (cit. on p. 76).
- [BBC+25] Andrea Basso, Giacomo Borin, Wouter Castryck, Maria Corte-Real Santos, Riccardo Invernizzi, Antonin Leroux, Luciano Maino, Frederik Vercauteren, and Benjamin Wesolowski. “PRISM: simple and compact identification and signatures from large prime degree isogenies”. In: *IACR International Conference on Public-Key Cryptography*. Springer. 2025, pp. 300–332 (cit. on p. 111).
- [BDF+25] Andrea Basso, Pierrick Dartois, Luca De Feo, Antonin Leroux, Luciano Maino, Giacomo Pope, Damien Robert, and Benjamin Wesolowski. “SQIsign2D–West”. In: *Advances in Cryptology – ASIACRYPT 2024*. Ed. by Kai-Min Chung and Yu Sasaki. Singapore: Springer Nature Singapore, 2025, pp. 339–370 (cit. on p. 5).
- [BDL+20] Daniel J Bernstein, Luca De Feo, Antonin Leroux, and Benjamin Smith. “Faster computation of isogenies of large prime degree”. In: *Open Book Series* 4.1 (2020), pp. 39–55 (cit. on p. 21).
- [Can87] David G Cantor. “Computing in the Jacobian of a hyperelliptic curve”. In: *Mathematics of computation* 48.177 (1987), pp. 95–101 (cit. on p. 23).

- [CD23] Wouter Castryck and Thomas Decru. “An Efficient Key Recovery Attack on SIDH”. In: *Advances in Cryptology – EUROCRYPT 2023*. Ed. by Carmit Hazay and Martijn Stam. Cham: Springer Nature Switzerland, 2023, pp. 423–447 (cit. on p. 5).
- [Coh13] Henri Cohen. *A course in computational algebraic number theory*. Springer Science & Business Media, 2013 (cit. on pp. 28, 31, 35).
- [Cor07] Giuseppe Cornacchia. *Su di un metodo per la risoluzione in numeri interi dell’equazione...* 1907 (cit. on p. 37).
- [CEM+24] Maria Corte-Real Santos, Jonathan Komada Eriksen, Michael Meyer, and Krijn Reijnders. “AprèsSQI: Extra fast verification for SQISign using extension-field signing”. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2024, pp. 63–93 (cit. on p. 16).
- [CJL+17] Craig Costello, David Jao, Patrick Longa, Michael Naehrig, Joost Renes, and David Urbanik. “Efficient compression of SIDH public keys”. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2017, pp. 679–706 (cit. on p. 16).
- [Dar25] Pierrick Dartois. “Fast computation of 2-isogenies in dimension 4 and cryptographic applications”. In: *Journal of Algebra* 683 (2025), pp. 449–514 (cit. on p. 88).
- [DLR+24] Pierrick Dartois, Antonin Leroux, Damien Robert, and Benjamin Wesolowski. “SQISignHD: New Dimensions in Cryptography”. In: *Advances in Cryptology – EUROCRYPT 2024*. Ed. by Marc Joye and Gregor Leander. Cham: Springer Nature Switzerland, 2024, pp. 3–32 (cit. on p. 5).
- [DMP+24] Pierrick Dartois, Luciano Maino, Giacomo Pope, and Damien Robert. “An algorithmic approach to (2, 2)-isogenies in the theta model and applications to isogeny-based cryptography”. In: *International Conference on the Theory and Application of Cryptology and Information Security*. Springer. 2024, pp. 304–338 (cit. on pp. 87, 88, 90).
- [DKL+20] Luca De Feo, David Kohel, Antonin Leroux, Christophe Petit, and Benjamin Wesolowski. “SQISign: Compact Post-quantum Signatures from Quaternions and Isogenies”. In: *Advances in Cryptology – ASIACRYPT 2020*. Ed. by Shiho Moriai and Huaxiong Wang. Cham: Springer International Publishing, 2020, pp. 64–93 (cit. on pp. 5, 40).
- [DLL+23] Luca De Feo, Antonin Leroux, Patrick Longa, and Benjamin Wesolowski. “New Algorithms for the Deuring Correspondence: Towards Practical and Secure SQISign Signatures”. In: *Advances in Cryptology – EUROCRYPT 2023*. Ed. by Carmit Hazay and Martijn Stam. Cham: Springer Nature Switzerland, 2023, pp. 659–690 (cit. on pp. 22, 38, 40).
- [DG16] Christina Delfs and Steven D Galbraith. “Computing isogenies between supersingular elliptic curves over $\mathbb{F}_p$ ”. In: *Designs, Codes and Cryptography* 78.2 (2016), pp. 425–440 (cit. on p. 110).
- [Die75] Ulrich Dieter. “How to calculate shortest vectors in a lattice”. In: *Mathematics of Computation* 29.131 (1975), pp. 827–833 (cit. on p. 30).
- [DF25] Max Duparc and Tako Boris Fouotsa. “SQIPrime: A Dimension 2 Variant of SQISignHD with Non-smooth Challenge Isogenies”. In: *Advances in Cryptology – ASIACRYPT 2024*. Ed. by Kai-Min Chung and Yu Sasaki. Singapore: Springer Nature Singapore, 2025, pp. 396–429 (cit. on p. 5).
- [Ebe07] David Eberly. “The Laplace expansion theorem: Computing the determinants and inverses of matrices”. In: *Geometric Tools, LLC, Scottsdale, Ariz, USA* (2007) (cit. on pp. 28, 35).

- [EHL+18] Kirsten Eisenträger, Sean Hallgren, Kristin Lauter, Travis Morrison, and Christophe Petit. “Supersingular isogeny graphs and endomorphism rings: reductions and solutions.” In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Cham: Springer International Publishing, 2018 (cit. on pp. 8, 110).
- [EHL+20] Kirsten Eisenträger, Sean Hallgren, Chris Leonardi, Travis Morrison, and Jennifer Park. “Computing endomorphism rings of supersingular elliptic curves and connections to path-finding in isogeny graphs”. In: *Open Book Series* 4.1 (2020), pp. 215–232 (cit. on p. 110).
- [EPG+20] Andres Erbsen, Jade Philipoom, Jason Gross, Robert Sloan, and Adam Chlipala. “Simple high-level code for cryptographic arithmetic: With proofs, without compromises”. In: *ACM SIGOPS Operating Systems Review* 54.1 (2020), pp. 23–30 (cit. on p. 64).
- [FS86] Amos Fiat and Adi Shamir. “How to prove yourself: Practical solutions to identification and signature problems”. In: *Conference on the theory and application of cryptographic techniques*. Springer. 1986, pp. 186–194 (cit. on p. 109).
- [FPL23] FPLLL development team. *fpLLL, a lattice reduction library, Version 5.5.0*. 2023. URL: <https://github.com/fplll/fplll> (visited on 04/10/2026) (cit. on p. 30).
- [FST10] David Freeman, Michael Scott, and Edlyn Teske. “A taxonomy of pairing-friendly elliptic curves”. In: *Journal of cryptology* 23.2 (2010), pp. 224–280 (cit. on p. 76).
- [FR94] Gerhard Frey and Hans-Georg Rück. “A remark concerning m-divisibility and the discrete logarithm in the divisor class group of curves”. In: *Mathematics of computation* 62.206 (1994), pp. 865–874 (cit. on p. 20).
- [JD11a] David Jao and Luca De Feo. “Towards Quantum-Resistant Cryptosystems from Supersingular Elliptic Curve Isogenies”. In: *Post-Quantum Cryptography*. Ed. by Bo-Yin Yang. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 19–34 (cit. on p. 5).
- [JD11b] David Jao and Luca De Feo. “Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies”. In: *International workshop on post-quantum cryptography*. Springer. 2011, pp. 19–34 (cit. on p. 25).
- [Kan97] Ernst Kani. “The number of curves of genus two with elliptic differentials”. In: *Journal für die reine und angewandte Mathematik* 1997.485 (1997), pp. 93–122 (cit. on p. 43).
- [Lic69] Stephen Lichtenbaum. “Duality theorems for curves over p-adic fields”. In: *Inventiones mathematicae* 7.2 (1969), pp. 120–136 (cit. on p. 20).
- [LWX+24] Kaizhan Lin, Weize Wang, Zheng Xu, and Chang-An Zhao. “A Faster Software Implementation of SQIsign”. In: *IEEE Transactions on Information Theory* 70.9 (2024), pp. 6679–6689 (cit. on p. 19).
- [LR23] David Lubicz and Damien Robert. “Fast change of level and applications to isogenies”. In: *Research in Number Theory* 9.1 (2023), p. 7 (cit. on p. 24).
- [MMP+23] Luciano Maino, Chloe Martindale, Lorenz Panny, Giacomo Pope, and Benjamin Wesolowski. “A Direct Key Recovery Attack on SIDH”. In: *Advances in Cryptology – EUROCRYPT 2023*. Ed. by Carmit Hazay and Martijn Stam. Cham: Springer Nature Switzerland, 2023, pp. 448–471 (cit. on p. 5).
- [Mil04] Victor S. Miller. “The Weil Pairing, and Its Efficient Calculation”. In: *Journal of Cryptology* 17.4 (Sept. 2004), pp. 235–261 (cit. on pp. 19, 76).

- [Mil86] James S Milne. “Jacobian varieties”. In: *Arithmetic geometry*. Springer, 1986, pp. 167–212 (cit. on p. 23).
- [MN90] François Morain and Jean-Louis Nicolas. “On Cornacchia’s algorithm for solving the diophantine equation $u^2 + dv^2 = m$ ”. In: *Projet 1000* (1990), a1 (cit. on p. 37).
- [NOC+25] Kohei Nakagawa, Hiroshi Onuki, Wouter Castryck, Mingjie Chen, Riccardo Invernizzi, Gioella Lorenzon, and Frederik Vercauteren. “SQIsign2D-East: A New Signature Scheme Using 2-Dimensional Isogenies”. In: *Advances in Cryptology – ASIACRYPT 2024*. Ed. by Kai-Min Chung and Yu Sasaki. Singapore: Springer Nature Singapore, 2025, pp. 272–303 (cit. on pp. 5, 109).
- [NS09] Phong Q Nguyen and Damien Stehlé. “An LLL algorithm with quadratic complexity”. In: *SIAM Journal on Computing* 39.3 (2009), pp. 874–903 (cit. on pp. 29, 30).
- [NV10] Phong Q Nguyen and Brigitte Vallée. *The LLL algorithm*. Springer, 2010 (cit. on p. 29).
- [OU73] Frans Oort and Kenji Ueno. “Principally Polarized Abelian Varieties of Dimension 2 Or 3 Are Jacobian Varieties”. In: *Journal of the Faculty of Science University of Tokyo Section 1-mathematics Astronomy Physics Chemistry* 20.3 (1973), pp. 377–381 (cit. on p. 23).
- [PW24] Aurel Page and Benjamin Wesolowski. “The Supersingular Endomorphism Ring and One Endomorphism Problems are Equivalent”. In: *Advances in Cryptology – EUROCRYPT 2024*. Ed. by Marc Joye and Gregor Leander. Cham: Springer Nature Switzerland, 2024, pp. 388–417 (cit. on pp. 9, 107, 110).
- [PZ24] Patrick Pelissier and Paul Zimmermann. *The DPE library*. GitLab repository. 2024. URL: <https://gitlab.inria.fr/zimmerma/dpe/> (visited on 04/10/2026) (cit. on p. 27).
- [PS18] C Petit and S Smith. “An improvement to the quaternion analogue of the l-isogeny problem”. In: *Presentation at MathCrypt*. MathCrypt. 2018 (cit. on p. 40).
- [Phi18] Jade Philipoom. “Correct-by-construction finite field arithmetic in Coq”. PhD thesis. Massachusetts Institute of Technology, 2018 (cit. on p. 64).
- [PH03] Stephen Pohlig and Martin Hellman. “An improved algorithm for computing logarithms over $GF(p)$ and its cryptographic significance (corresp.)” In: *IEEE Transactions on information Theory* 24.1 (2003), pp. 106–110 (cit. on p. 19).
- [Rei23] Krijn Reijnders. “Effective pairings in isogeny-based cryptography”. In: *International Conference on Cryptology and Information Security in Latin America*. Springer. 2023, pp. 109–128 (cit. on p. 76).
- [Ren18] Joost Renes. “Computing isogenies between Montgomery curves using the action of $(0, 0)$ ”. In: *International conference on post-quantum cryptography*. Springer. 2018, pp. 229–247 (cit. on p. 22).
- [Rob23] Damien Robert. “Breaking SIDH in Polynomial Time”. In: *Advances in Cryptology – EUROCRYPT 2023*. Ed. by Carmit Hazay and Martijn Stam. Cham: Springer Nature Switzerland, 2023, pp. 472–503 (cit. on p. 5).
- [Rob24] Damien Robert. *Fast pairings via biextensions and cubical arithmetic*. Cryptology ePrint Archive, Paper 2024/517. 2024. URL: <https://eprint.iacr.org/2024/517> (cit. on p. 76).
- [Sta07] Katherine E Stange. “The Tate pairing via elliptic nets”. In: *International Conference on Pairing-Based Cryptography*. Springer. 2007, pp. 329–348 (cit. on p. 76).

- [Tat62] John Tate. “Duality theorems in Galois cohomology over number fields”. In: *Proc. Internat. Congr. Mathematicians (Stockholm, 1962)*. 1962, pp. 288–295 (cit. on p. 20).
- [Vél71] Jacques Vélú. “Isogénies entre courbes elliptiques”. In: *Comptes-Rendus de l’Académie des Sciences* 273 (1971), pp. 238–241 (cit. on p. 21).
- [Wei40] André Weil. “Sur les fonctions algébriques a corps de constantes fini”. In: *CR Acad. Sci. Paris* 210.592-594 (1940), p. 149 (cit. on p. 19).
- [Wes22] Benjamin. Wesolowski. “The supersingular isogeny path and endomorphism ring problems are equivalent.” In: *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 2022 (cit. on pp. 8, 104).
- [XLZ+25] Zheng Xu, Kaizhan Lin, Chang-An Zhao, and Yi Ouyang. “SQIsign2D 2: New SQIsign2D Variant by Leveraging Power Smooth Isogenies in Dimension One”. In: *International Conference on the Theory and Application of Cryptology and Information Security*. Springer. 2025, pp. 341–371 (cit. on pp. 5, 12, 106, 110).
- [ZSP+18] Gustavo HM Zanon, Marcos A Simplicio, Geovandro CCF Pereira, Javad Doliskani, and Paulo SLM Barreto. “Faster key compression for isogeny-based cryptosystems”. In: *IEEE Transactions on Computers* 68.5 (2018), pp. 688–701 (cit. on p. 16).